



Object-Oriented Systems Engineering Method (OOSEM v2)

August, 2026



An OMG Managed Community

Sanford Friedenthal
safriedenthal@gmail.com

Copyright © 2021-2026 by Sanford Friedenthal

Licensed under the Creative Commons Attribution 4.0 International License.

*To view a copy of this license, visit <http://creativecommons.org/licenses/by/4.0/> or
send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.*

Changes from previous version

- This is the initial version of OOSEM v2 which is based on OOSEM v1

Abstract

The object-oriented systems engineering method (OOSEM) is an MBSE method to specify, architect, analyze, and verify systems. OOSEM applies a scenario-driven decomposition approach coupled with the principle of separation of concerns to develop the system model. This content provides an overview of how OOSEM v2 leverages SysML v2 to provide more efficient modeling and improved traceability across the system model.. This work is intended to support a collaborative effort between the OMG Systems Modeling Community (SMC) and the INCOSE OOSEM Working Group.

Acknowledgements

- The material from this course acknowledges the following contributions:
 - “A Practical Guide to SysML” by Friedenthal, Moore, and Steiner (3rd Edition - Chapter 17)
 - INCOSE OOSEM Working Group
 - OOSEM v1 Tutorial material developed by Joe Wolfrom and Sandy Friedenthal, and reviewed by the INCOSE OOSEM Working Group
 - Current OOSEM v2 effort
 - OMG Systems Modeling Community (SMC) User WG
 - S. Friedenthal, A. Muxen paper A SysML V2 Modeling Pattern for Recursive System Decomposition presented at iNCOSE IS, June 2026

Acknowledgement

- The SysML v2 diagrams in this slide set were created using Dassault Systèmes Cameo Systems Modeler

Produced with

Magic / Cameo Systems Modeler

Presentation Objectives

- Understand how to use OOSEM to guide a team through the complexities of system development
 - Concepts
 - Key activities
 - The modeling artifacts that are produced that constitute views of the system model
 - Does not cover the detailed modeling patterns

Agenda

- Introduction
- Contrasting SysML v2 with SysML v1
- OOSEM Overview
- Manage System Development
- Analyze Stakeholder Needs
- Analyze System requirements
- Define Logical Architecture
- Synthesize Candidate Physical Architectures
- Optimize and Evaluate Alternatives
- Verify System Design
- Manage Requirements Traceability
- Summary

Contrasting SysML v2 with SysML v1

SysML v2 Objectives

- **Increase adoption and effectiveness of MBSE with SysML v2 to support the evolving MBSE practice by enhancing...**
 - Precision and expressiveness of the language
 - Consistency and integration among language concepts
 - Interoperability with other engineering models and tools
 - Usability by model developers and consumers
 - Extensibility to support domain specific applications
 - Migration path for SysML v1 users and implementors
- **Enable more rigorous and agile systems engineering**

Key Elements of SysML v2

- New Metamodel that is not constrained by UML
 - Preserves most of UML modeling capabilities with a focus on systems modeling
 - Grounded in formal semantics
- Robust visualizations based on flexible view & viewpoint specification
 - Graphical, Tabular, Textual
- Standardized API to access the model

SysML v2 to v1

Terminology Mapping (partial)

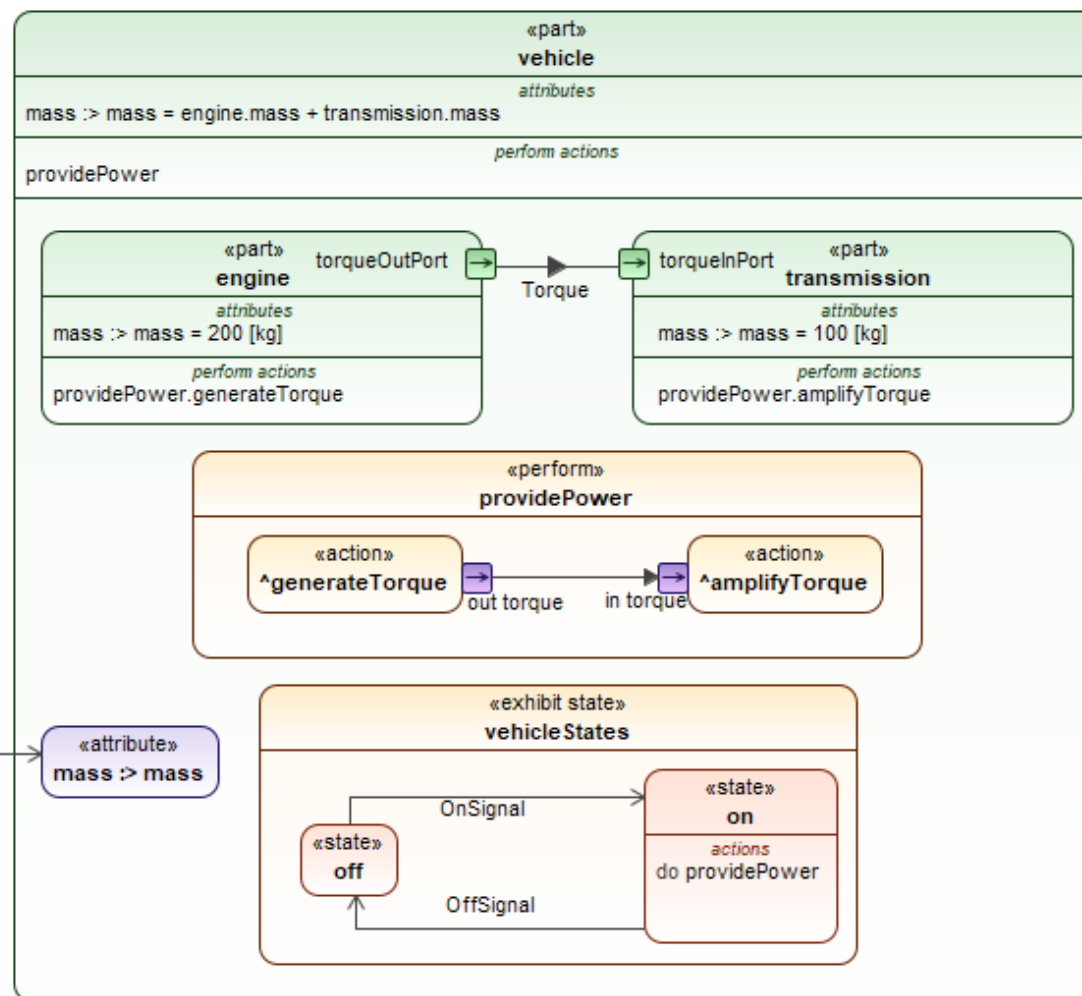
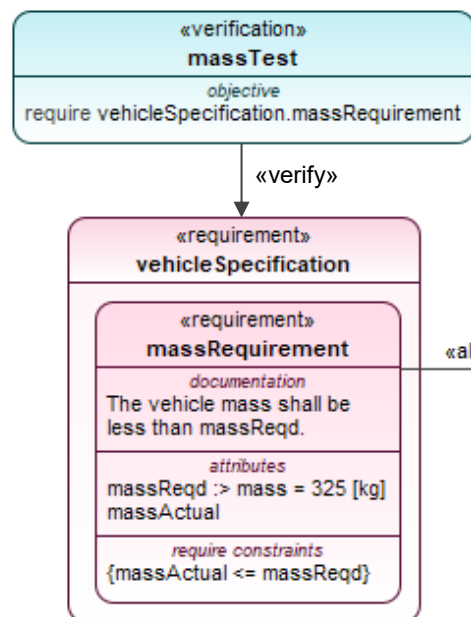
SysML v2	SysML v1
part / part def	part property / block
attribute / attribute def	value property / value type
port / port def	proxy port / interface block
action / action def	action / activity
state / state def	state / state machine
constraint / constraint def	constraint property / constraint block
requirement / requirement def	requirement
connection / connection def	connector / association block
view / view def	view

SysML v2 applies a consistent pattern of definition and usage
define once, and use many times
each usage can be adapted to its context
enables decomposition and specialization

Vehicle Example

Produced with

Magic | Cameo Systems Modeler



```
package vehicle_example {
  view vehicle_example;
  private import SI::*;
  item def OnSignal;
  item def OffSignal;
  attribute def Torque;

  part vehicle {
    attribute mass := ISQBase::mass = engine.mass + transmission.mass;
    perform providePower;
    part engine {
      attribute mass := ISQBase::mass = 200 [kilogram];
      port torqueOutPort {
        out attribute torque : Torque;
      }
      perform providePower.generateTorque;
    }
    part transmission {
      attribute mass := ISQBase::mass = 100 [kilogram];
      port torqueInPort {
        in attribute torque : Torque;
      }
      perform providePower.amplifyTorque;
    }
    connect engine.torqueOutPort to transmission.torqueInPort {
      flow of Torque from source.torque to target.torque;
    }
    exhibit state vehicleStates {
      state off;
      state on {
        do providePower;
      }
      transition off_to_on first off accept OnSignal then on;
      transition on_to_off first on accept OffSignal then off;
    }
  }

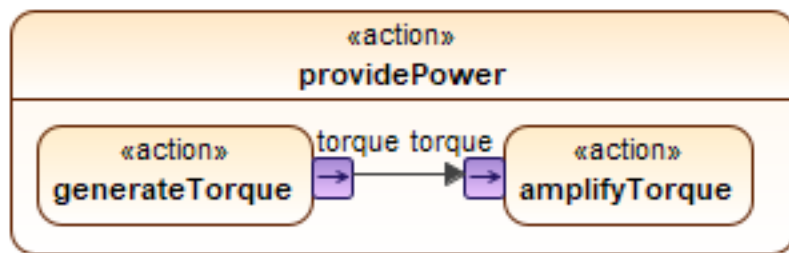
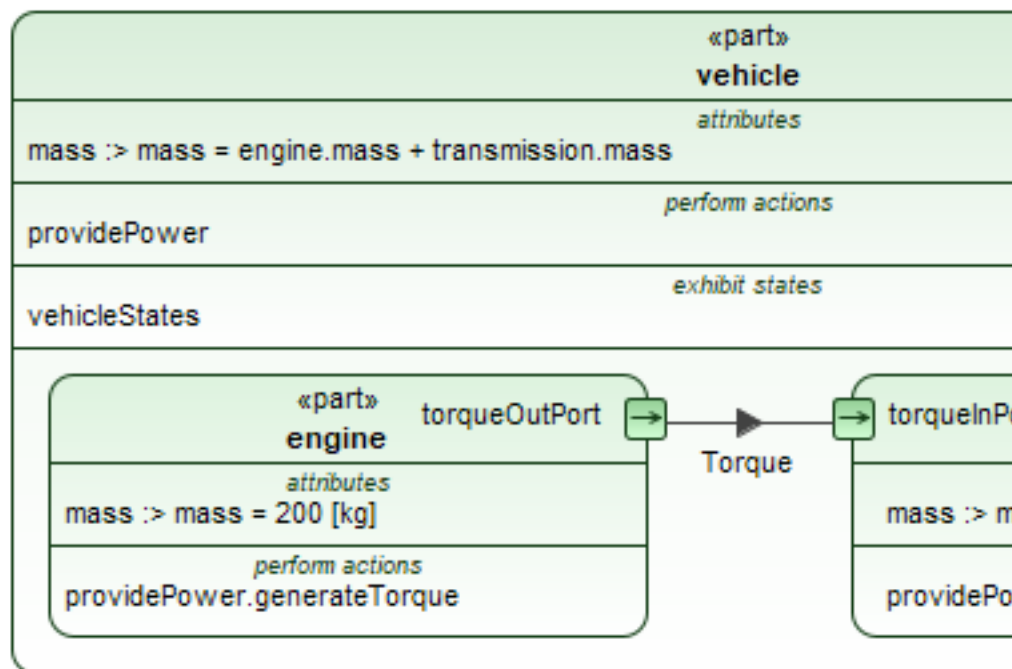
  action providePower {
    action generateTorque { out attribute torque; }
    action amplifyTorque { in attribute torque; }
    flow generateTorque.torque to amplifyTorque.torque;
  }

  requirement vehicleSpecification {
    requirement massRequirement {
      doc /* The vehicle mass shall be less than massReqd. */
      attribute massReqd := mass = 325 [kilogram];
      attribute massActual;
      require constraint {massActual <= massReqd}
    }
  }

  allocate vehicleSpecification.massRequirement to vehicle.mass;
  verification massTest {
    objective {
      verify vehicleSpecification.massRequirement;
    }
  }
}
```

Vehicle Example

Graphical/Textual Correspondence



```

7  part vehicle {
8      attribute mass :> ISQBase::mass = engine.mass + transmission.mass;
9      perform providePower;
10  exhibit state vehicleStates {↔}
18  part engine {
19      attribute mass :> ISQBase::mass = 200 [kg];
20  port torqueOutPort {
21      out attribute torque : Torque;
22  }
23  perform providePower.generateTorque;
24  }
25  part transmission {
26      attribute mass :> ISQBase::mass = 100 [kg];
27  port torqueInPort {
28      in attribute torque : Torque;
29  }
30  perform providePower.amplifyTorque;
31  }
32  connect engine.torqueOutPort to transmission.torqueInPort {↔}
35  }
36  action providePower {
37      action generateTorque { out attribute torque; }
38      action amplifyTorque { in attribute torque; }
39      flow generateTorque.torque to amplifyTorque.torque;
40  }

```

Proc



Magic | Cameo Systems Modeler

Comparing SysML v2 with SysML v1

- **Simpler to learn and use**
 - Systems engineering concepts designed into metamodel versus added-on
 - Consistent definition and usage pattern
 - More consistent terminology
 - Ability to decompose parts, actions, ...
- **More precise**
 - Textual syntax and expression language
 - Formal semantic grounding
 - Requirements as constraints
 - Integrated structure and behavior
- **More expressive**
 - Specialization of behavior, requirements, ..
 - Variant modeling
 - Analysis case and trade-off analysis
 - Individuals, snapshots, time slices
 - More robust quantitative properties (e.g., vectors, ..)
 - Simple geometry
 - Query/filter expressions
 - Metadata
- **More extensible**
 - Simpler language extension capability
 - Based on model libraries
- **More interoperable**
 - Standardized API

These enhancements can be leveraged by OOSEM

OOSEM v2 Overview

Systems Engineering Value Statement

Reference: INCOSE SE Vision 2035

- **Systems engineering aims to ensure the pieces work together to achieve the objectives of the whole**
 - *Architect balanced solutions that satisfy diverse stakeholder needs for capability, dependability, sustainability, social acceptability, and ease of use*
 - *Adapt to evolving technology and requirements*
 - *Manage complexity and risk*



What is a good systems engineering methodology?

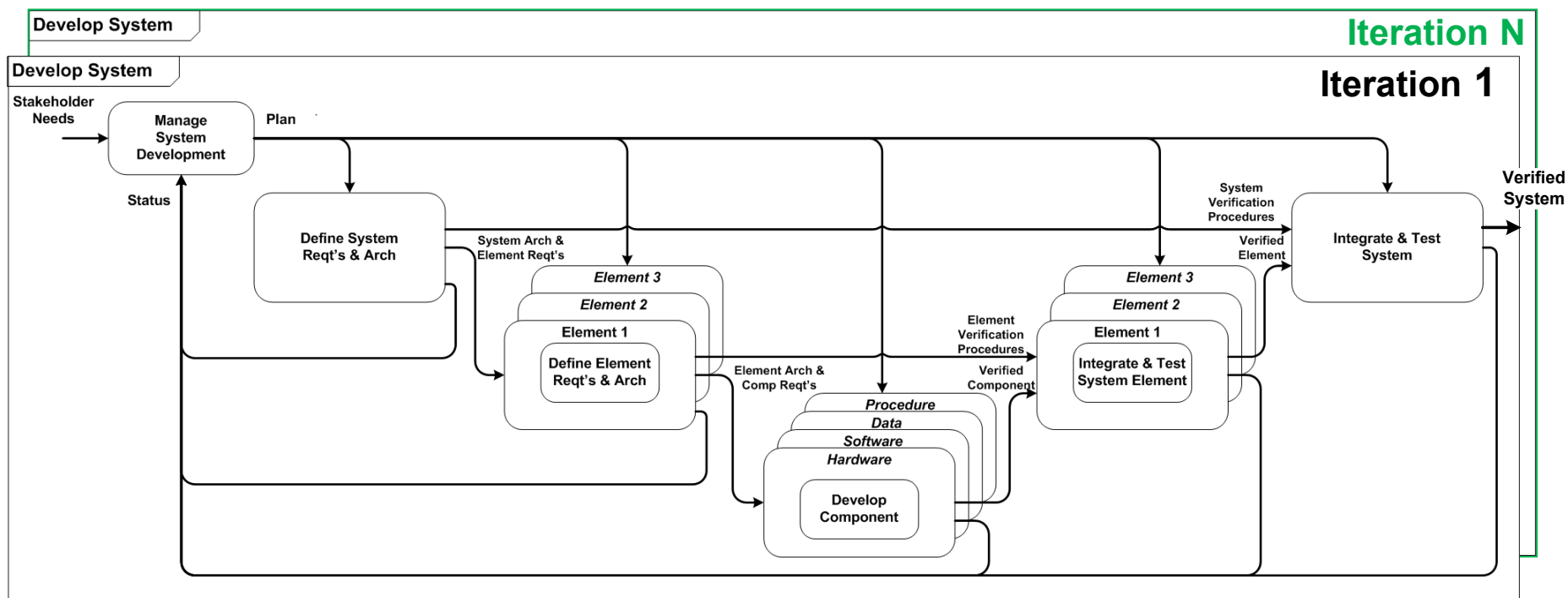
- Guides a team to develop a balanced solution that addresses stakeholder needs
- Efficiently implements the **systems engineering** process
 - Consistent with industry standard development and lifecycle processes
 - Produces an integrated system model as a primary artifact of the SE process to support specifying, designing, analyzing, and verifying the system
- Deals with increasing system complexity and the ability to more rapidly respond to change
 - Separation of concerns
 - Enables concurrent activities
 - Enables reuse
- Tailorable to a wide range of considerations including the application domain, project size, project phase, new development vs legacy, experience of team, etc.
- Uses a standard modeling language that is not overly constrained by a specific tool

Object-oriented Systems Engineering Method (OOSEM)

- Goal: Provide a model-based systems engineering (MBSE) method to support
 - Efficient and rigorous implementation of standard SE processes
 - Integration with software and hardware design, verification, and analysis
 - Flexibility to accommodate changing requirements, design evolution, and variability
 - Use as a [reference methodology](#) that can be adapted and tailored to the application needs
- Approach
 - Top-down decomposition coupled with bottoms-up design
 - Scenario-driven
 - Separation of concerns to deal with complexity
 - Results in a system model that captures and maintains traceability among the specification, design, analysis, and verification artifacts
 - Leverage agile management practices that support continuous system integration
 - Leverage SysML as the standard modeling language

System Development Process

Achieving Productivity, Quality, and Agility



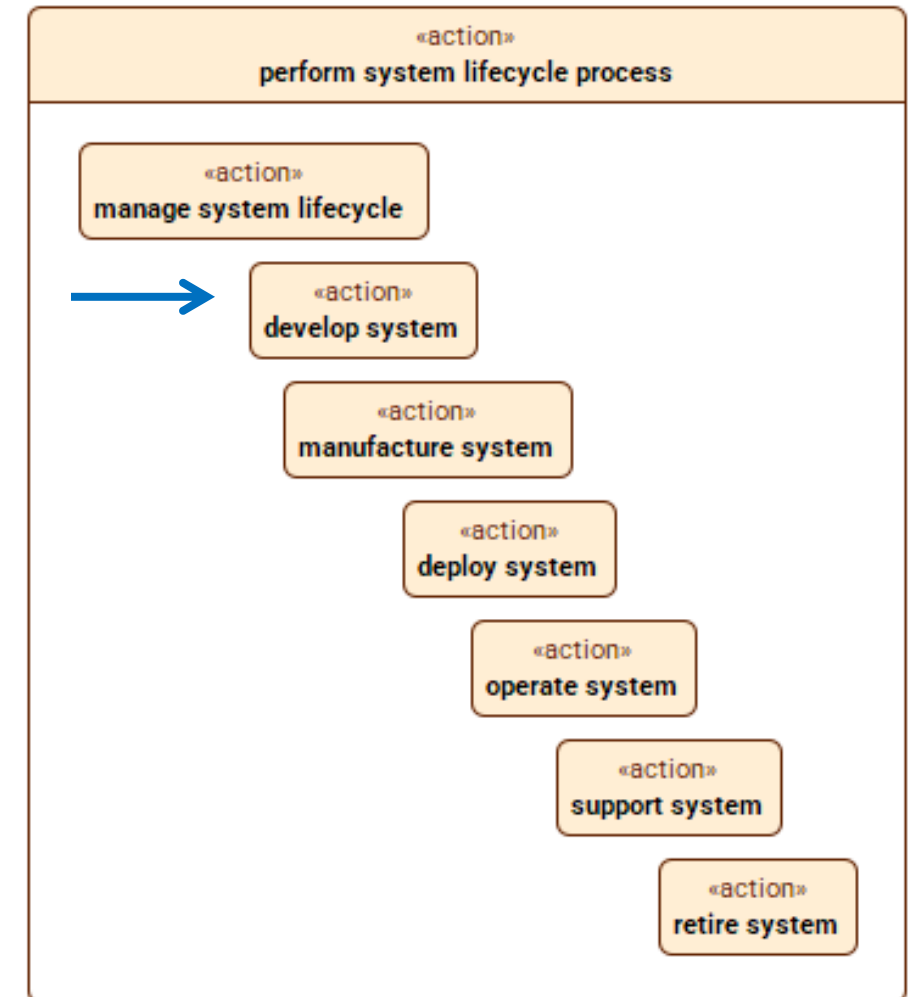
Cross-disciplinary teams
are essential to improve
communications

A recursive Vee process

Applied iteratively
and incrementally
throughout the
system lifecycle

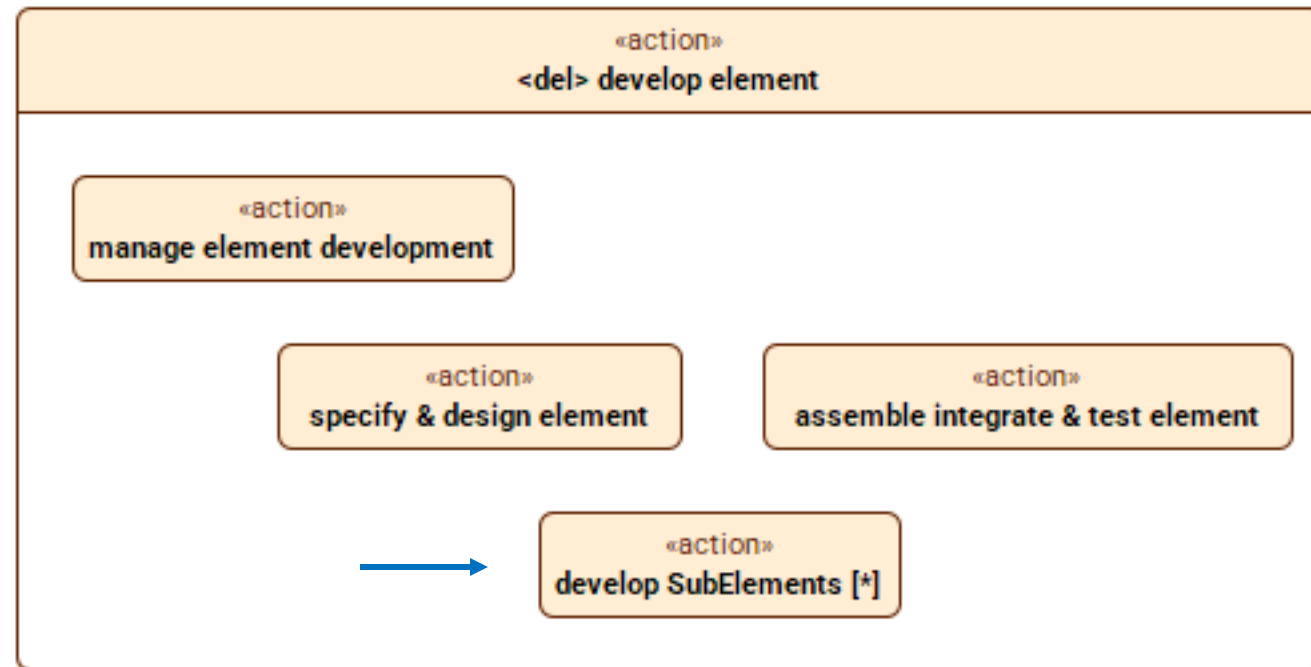
System Lifecycle Processes and the Development Process

- *Develop system* process includes processes to develop operational system and enabling systems (e.g., manufacturing system, support system, ...) and how they integrate with the other system lifecycle processes



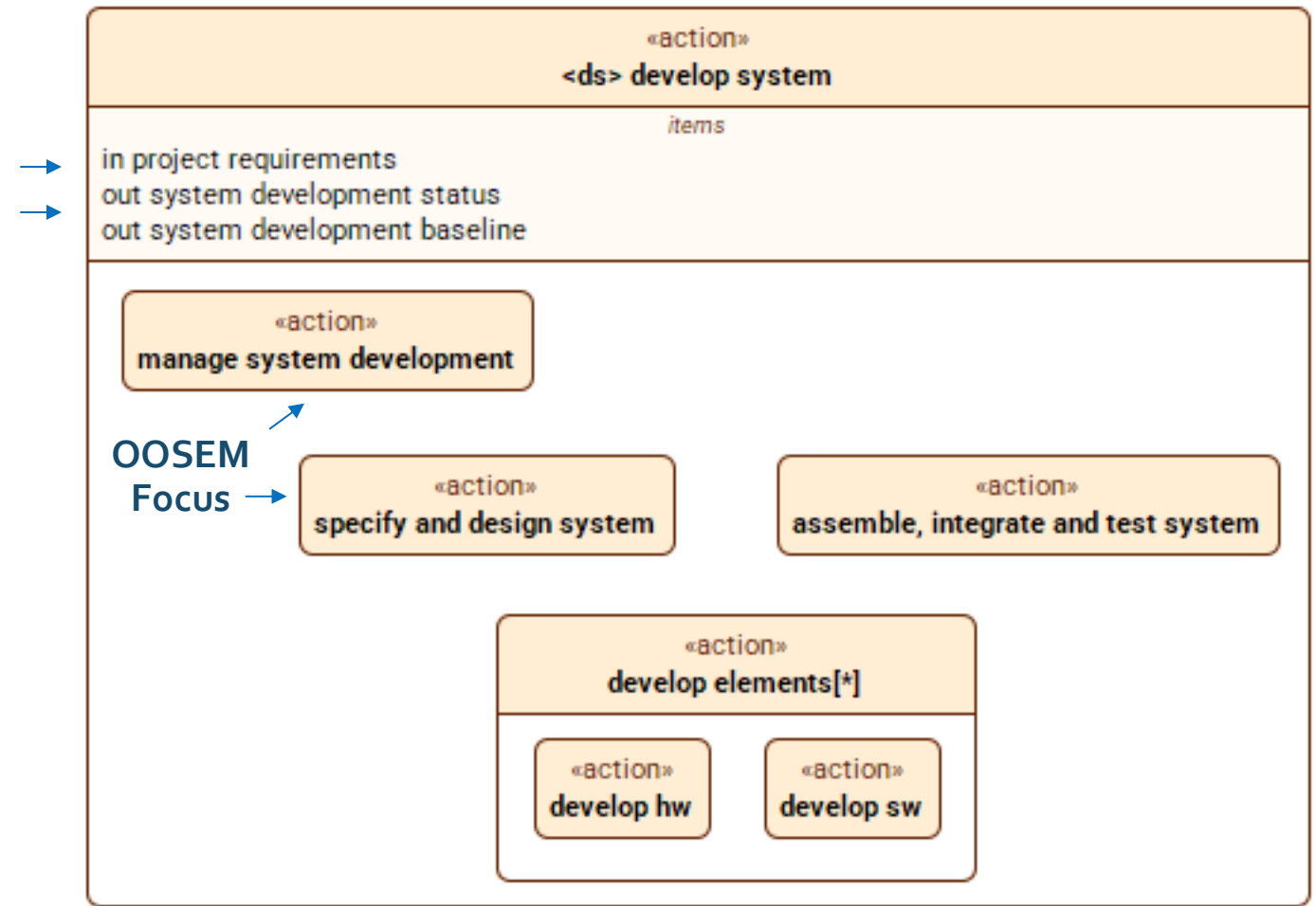
Recursive Development Process

- Can be applied to develop any kind of element including system of systems, system, hardware, software, ..
- Applies recursively to sub elements down to leaf level where detailed design and implementation is performed



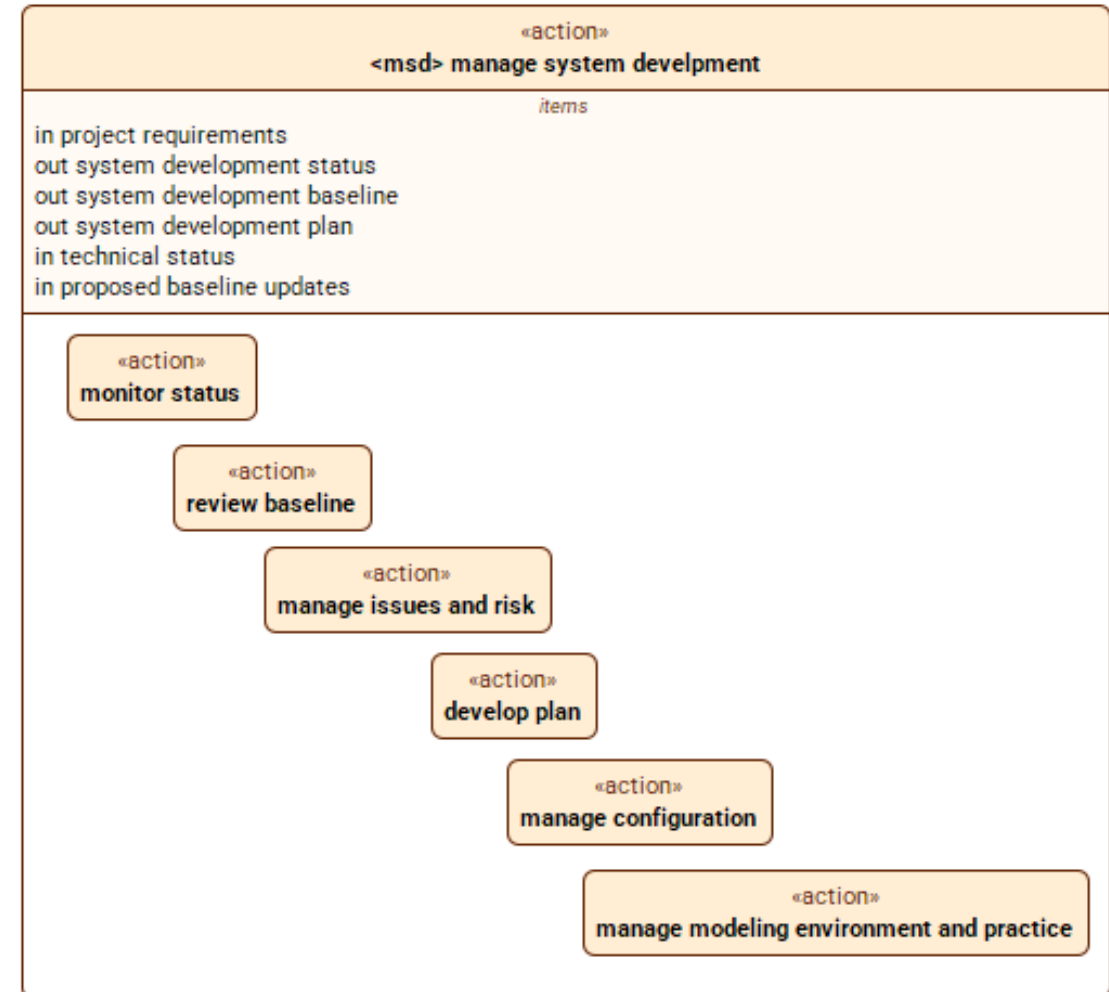
Develop System Process

- *Develop System* process is a specialization of the *Develop Element* process
- Inputs
 - *Project requirements* (e.g., statement of work)
- Outputs
 - *System development baseline* that includes the technical data package, and deliverable hardware and software used to verify and validate system requirements



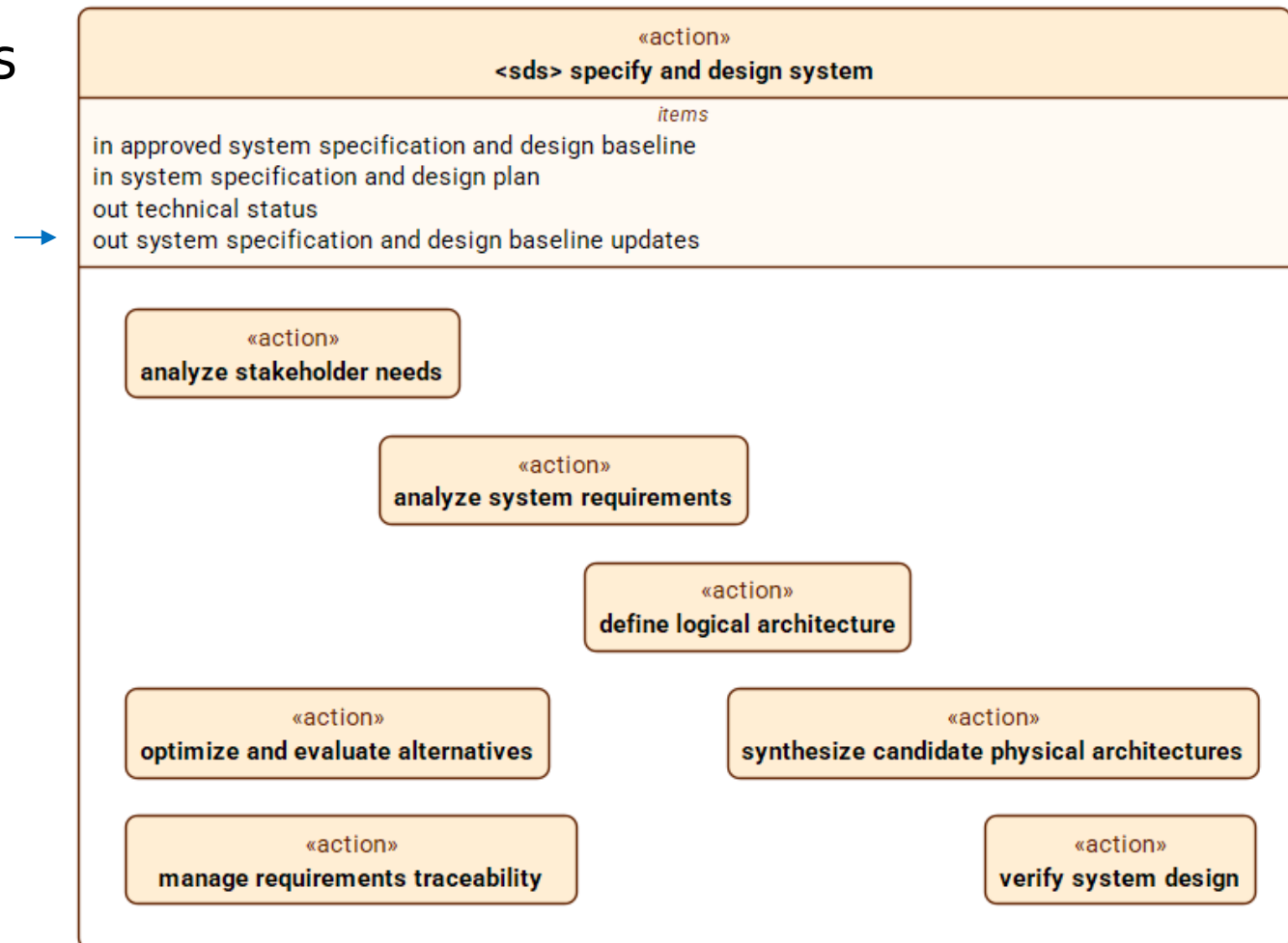
Manage System Development Process

- Manages the technical processes by monitoring technical status, reviewing the baseline, managing the configuration, managing risk, and providing incremental plan updates



Specify and Design System Process

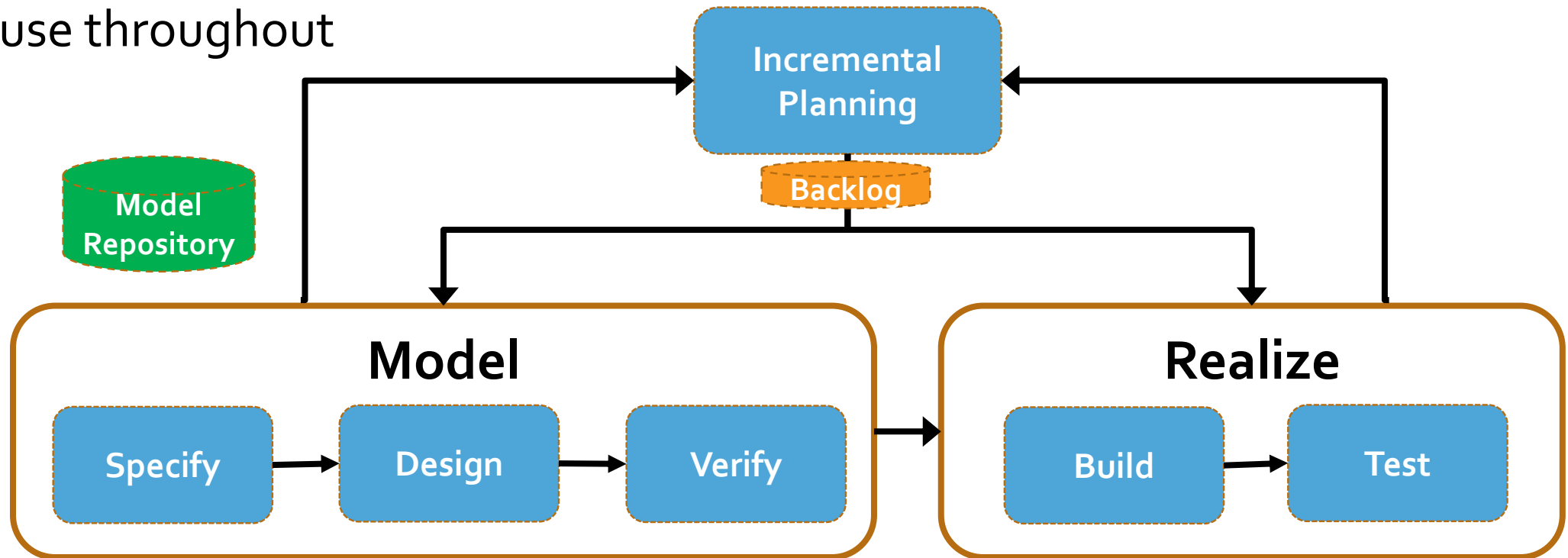
- Primary OOSEM technical process
 - Transforms the set of stakeholder needs and as-is system to the to-be system specification and design
- The system specification and design baseline **is** the ***system model***



MBSE Process

Verify Before Build

- Incrementally develop a model of the system before building the actual system
- Increments can be use case driven (e.g., functional threads) and/or risk driven
- Leverage reuse throughout



Facilitates Rigor and Agility

Tailoring the Reference Methodology

- OOSEM must be tailored to meet the needs of an organization or project
- Tailoring must consider lifecycle phase, project size, application domain, design level, breadth of trade space, team experience, cost and schedule constraints, etc.
- Typical tailoring includes:
 - Identifying which processes & modeling artifacts are needed and their level of detail
 - Aligning terminology of processes and artifacts
 - Defining the project lifecycle process and the associated sequencing of actions
 - Allocating actions and artifacts to organization structure, roles and/or work packages
- Emphasis shifts with the phase of development
 - Top level mission requirements, architecture design and physical sizing trade-offs in conceptual design phase
 - Detailed architecture trade-offs and component requirements in later design phases

Summary

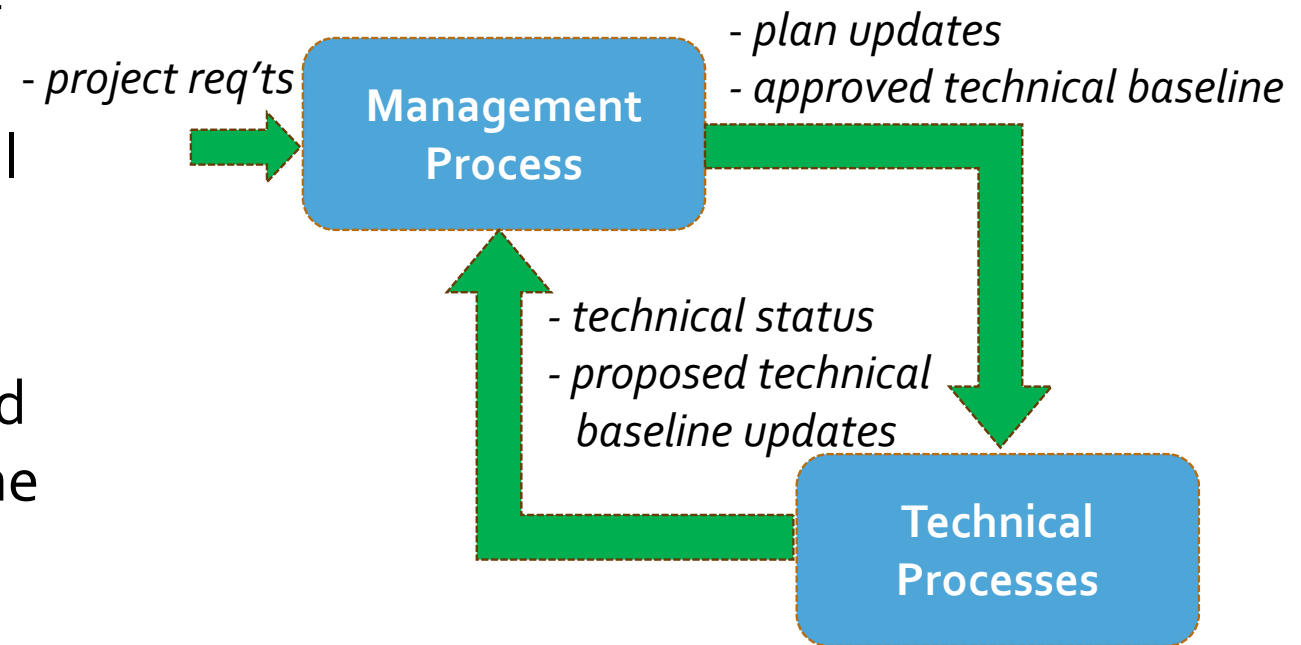
- OOSEM is an MBSE method that provides a top-down, scenario-driven process to analyze, specify, design, and verify a system
 - System model is a primary artifact
 - Controlled as part of the technical baseline
- Provides a way to deal with system design and development complexity using separation of concerns as a fundamental tenet
 - Black box, white box
 - Logical, physical
 - Distribution of functionality, data and control
 - Operational system, enabling systems
- Should be tailored to satisfy project objectives and constraints

Manage System Development

The Technical Management Activities

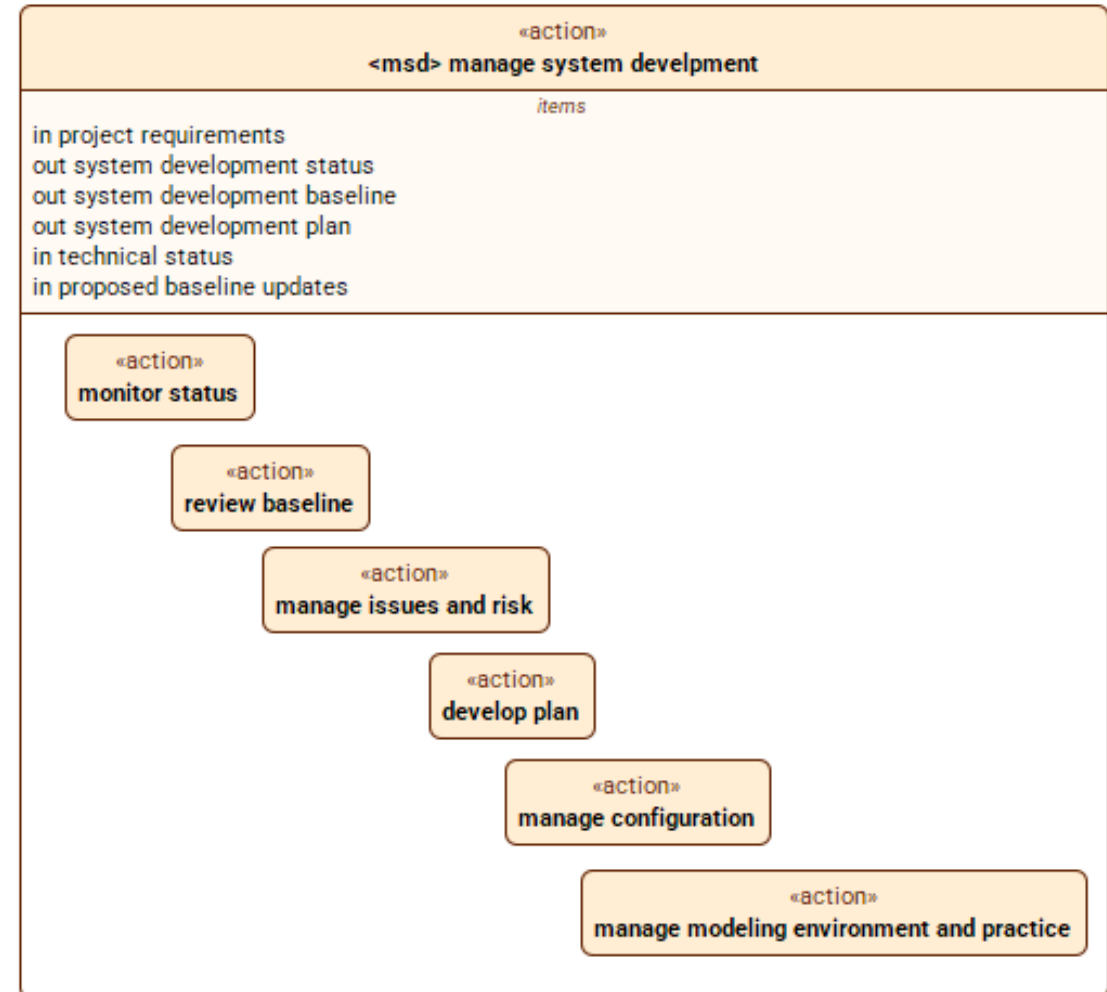
Management & Technical Processes

- Project requirements (e.g., statement of work) is a primary input to management process
- Management process monitors technical status and provides incremental plan updates
- Technical processes execute the plan and propose updates to the technical baseline
- Management process reviews and approves updates to the technical baseline
- A feedback and control process



Manage System Development Process

- Manages the technical processes by monitoring technical status, reviewing the baseline, managing the configuration, managing risk, and providing incremental plan updates



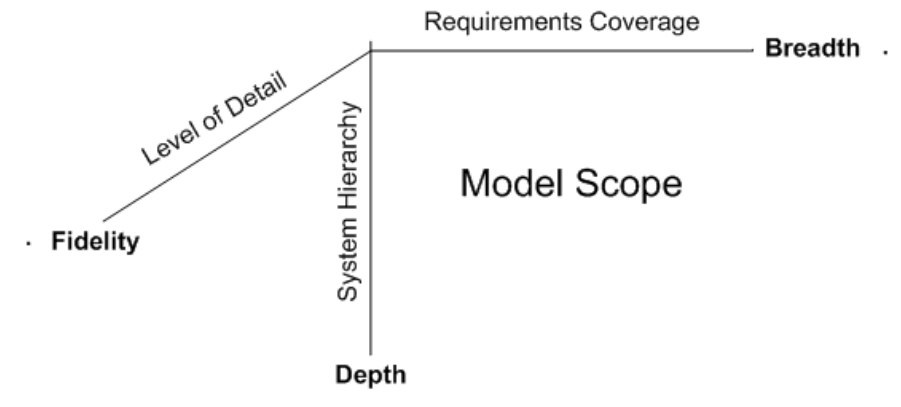
Plan the Modeling Effort

Plan the Modeling Effort

- Develop long term plan that includes key project milestones, deliverables, and estimated resources
- At each milestone
 - Update status and long-term plan
 - Prepare incremental plan to next milestone
- Plan the modeling effort to support the project objectives and constraints
 - Define model objectives
 - Scope the model to meet objectives
 - Identify initial source data to mine and populate the model
 - (i.e., reference models, legacy data including requirements components, interfaces)
 - Select & tailor the method and modeling guidelines
 - Select the tools and establish the modeling infrastructure
 - Define the schedule of modeling activities and artifacts
 - Staff and organize the modeling effort
 - Provide training in the language, method, and tools

Model Objectives and Scope

- Model objectives and success criteria
 - Align with project objectives and phase
 - Identify artifacts needed from the model
 - *Who are the consumers and how are they using the model ?*
 - Identify relevant project metrics
- Model scope supports model objectives
 - Define breadth, depth and fidelity of model
 - Breadth relates to requirements coverage
 - Depth relates to level of system design hierarchy
 - Fidelity relates to level of detail of the model
 - Balance scope with project resources, expertise, and constraints



Modeling Artifacts

- Identify modeling artifacts for system, subsystem, components
 - Black box specification
 - Logical/physical architecture
 - Analysis
 - Verification cases/verification results
 - Traceability to requirements
- Maintain model as part of the technical baseline under CM control
- Support project deliverables
 - Generate documentation directly from the model

Model Development Milestones

- Modeling artifact maturity defined for each project milestone
- Integrate model artifact development milestones into project schedule
- Track against plan like any other project deliverable

Milestone	Artifact	Maturity Level				
		SRR	SDR	PDR	CDR	TRR
System	Black box specification	H				
	Logical architecture	M	H			
	Physical architecture	L	M	H		
	Analysis					
	Test cases	L	M		H	
	Verification results					L
Element	Black box specification	M	H			
	Logical architecture		M	H		
	Physical architecture		L	M	H	
	Analysis					
	Test cases		L	M	H	
	Verification results					M
Component	Black box specification		M	H		
	Design		L	M	H	
	Analysis					
	Test cases		L	M	H	
	Implementation					H
	Verification results					H

Summary

- The modeling plan should be part of an integrated plan to achieve project objectives within project constraints
- Identify key project milestones and resources for long term plan and develop detailed plans for each increment
- The plan for the modeling effort should include the modeling objectives, scope, tailored method, artifact delivery milestones, responsibilities, ...

Setup Model

Set Up Model

- Establish modeling conventions and standards to facilitate consistent representation and style across the model
- Establish and maintain the model organization



Defining Model Conventions and Standards

- Ensure consistency throughout the model
 - Naming conventions
 - Modeling patterns
 - Diagram guidelines
 - User defined extensions
 - Model annotation guidelines for element descriptions, rationale, and comments
 - Custom reports and document templates
 - Model review checklists
 - Model metrics

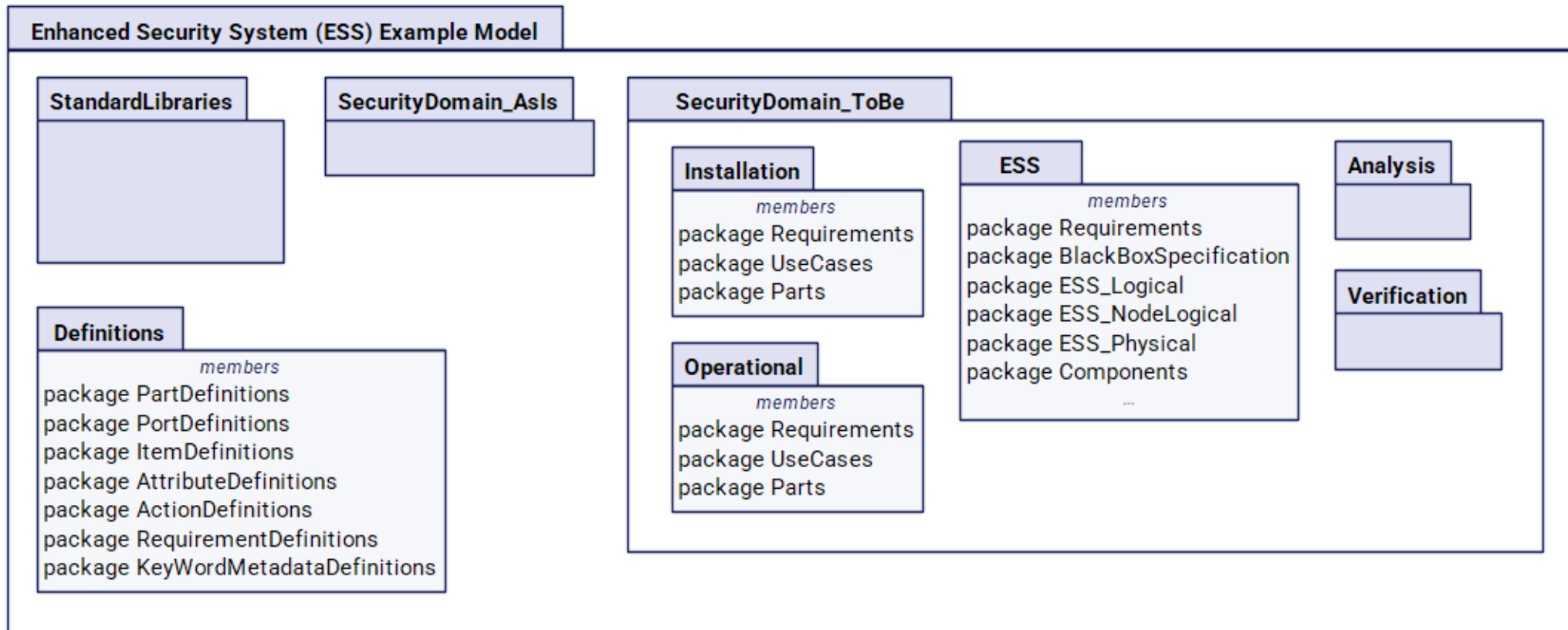
Motivation for Model Organization

- Models often have thousands of model elements and hundreds of diagrams
- Model organization is essential to aid in managing the model
 - navigating and querying the model
 - managing changes to the model
 - controlling access to the model
 - reusing the model

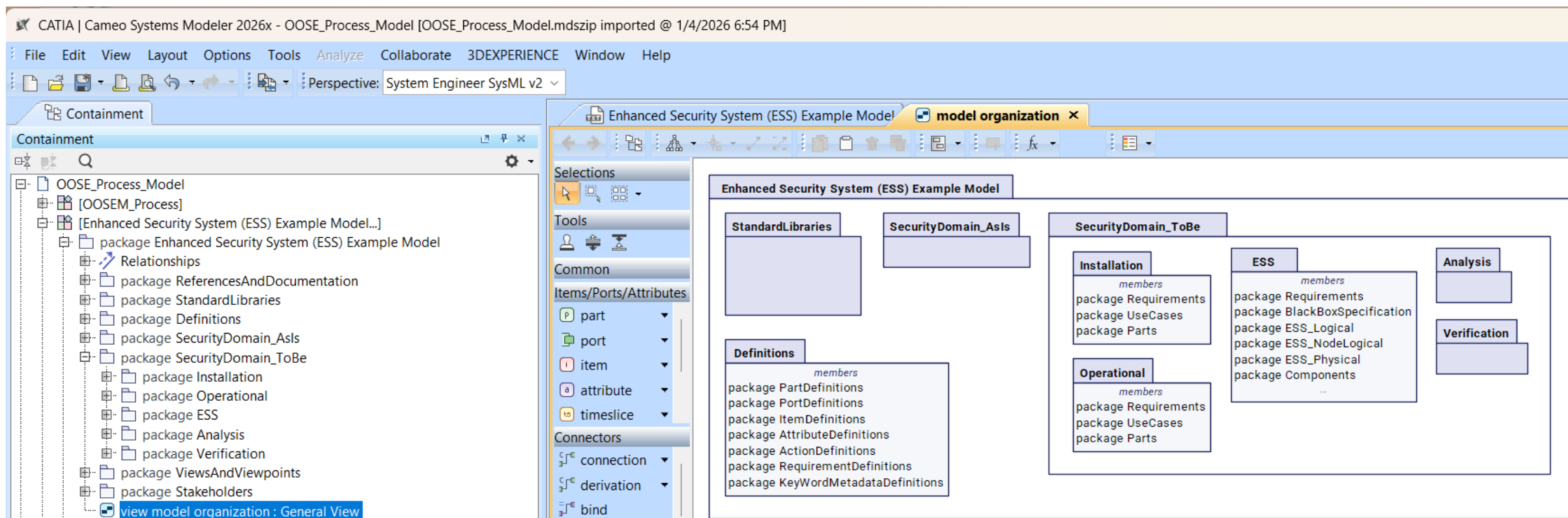
Model Organization

- Packages are used to organize the model
 - Groups model elements into a name space
 - Package hierarchy represented in tool browser
 - Often used to support configuration management (check-in/out)
- Model can be organized in multiple ways
 - Align with methodology

Enhanced Security System (ESS) Example Model Organization



ESS Model Organization



Summary

- Naming conventions, modeling patterns, custom reports are defined to ensure consistency across the model
- Model organization must be established, maintained, and communicated across the team
 - Essential for model management, access control, reuse, queries and automation

Manage the Configuration

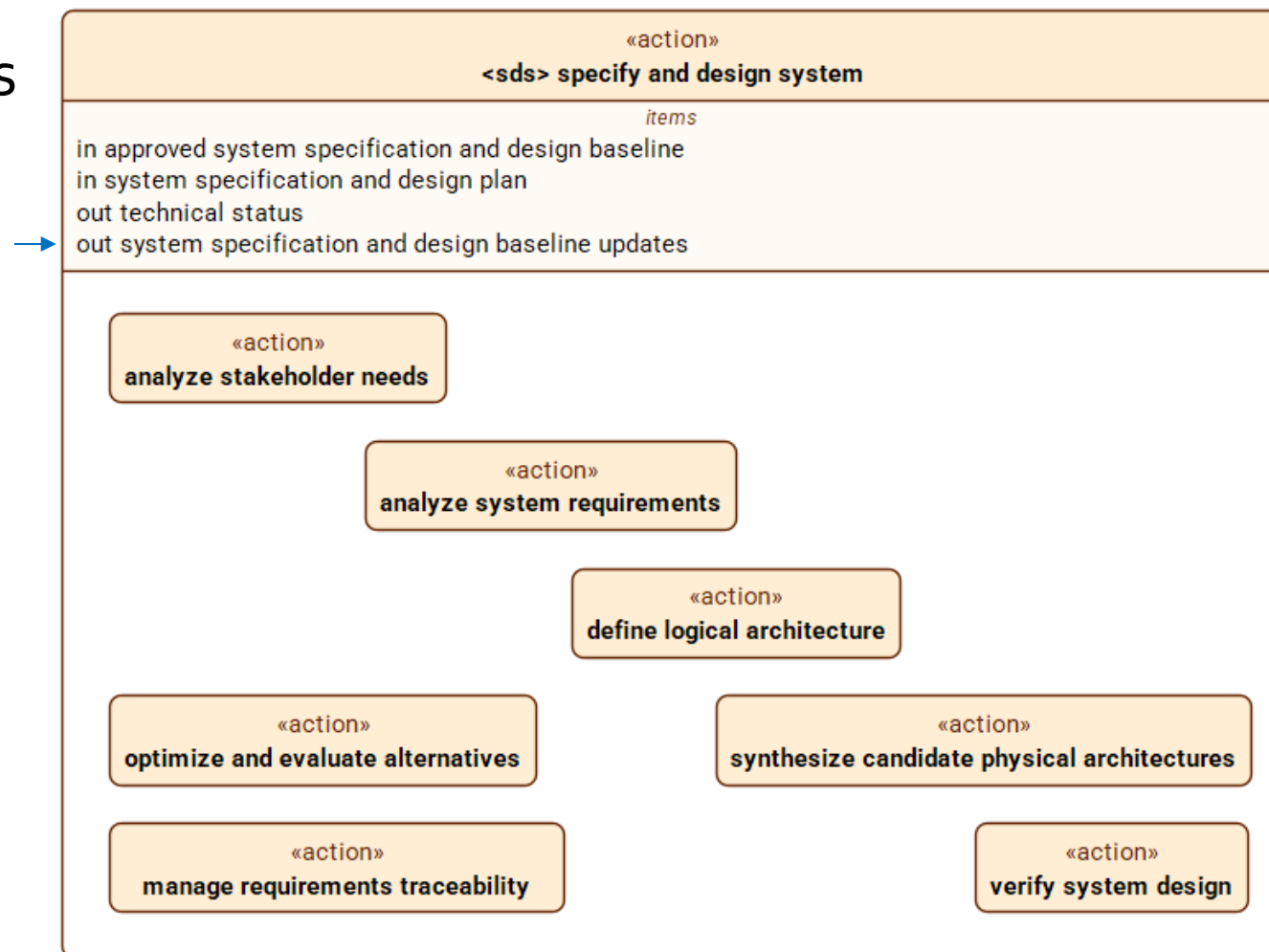
Configuration Management Considerations

- Project usages are part of model organization
 - A project can use another project that it depends on
- Check in/check out
 - Controls access (e.g., read/write privileges)
- Branch / merge
 - Create branch to modify current baseline (e.g., black box, gray box, white box)
 - Assess impact of change on current baseline and merge into trunk to create updated baseline
 - Make incremental changes
 - Verify requirements are satisfied prior to merge
- Implement appropriate level of CM based on design maturity
 - Lighter-weight change control during early phases when change occurs frequently
 - Disciplined change control process for more mature designs

Specify and Design System

Specify and Design System Process

- Primary OOSEM technical process
 - Transforms the set of stakeholder needs and as-is system to the to-be system specification and design
- The system specification and design baseline **is** the ***system model***



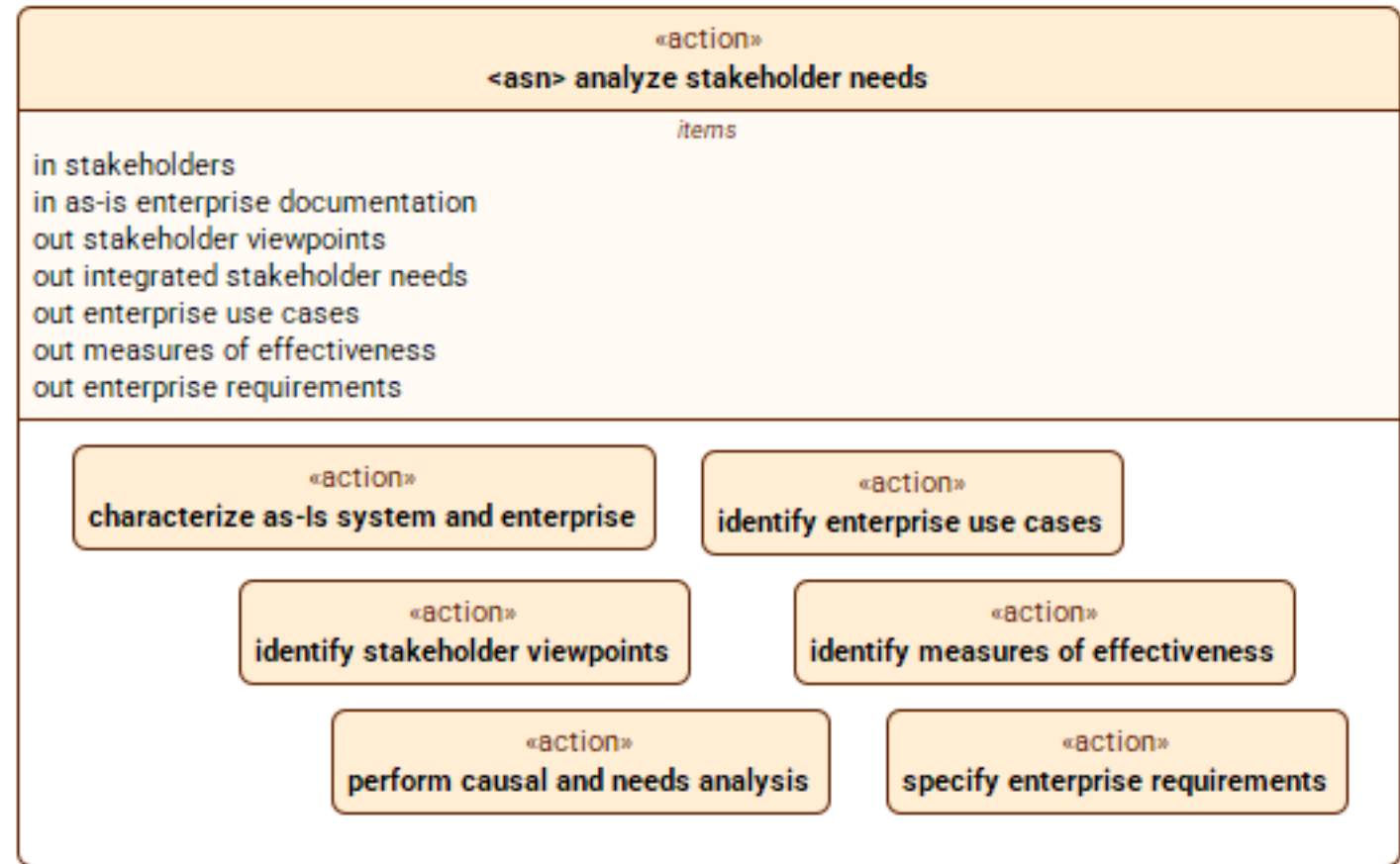
Analyze Stakeholder Needs

Motivation

- Understanding the problem is a pre-requisite to developing an effective solution

Analyze Stakeholder Needs

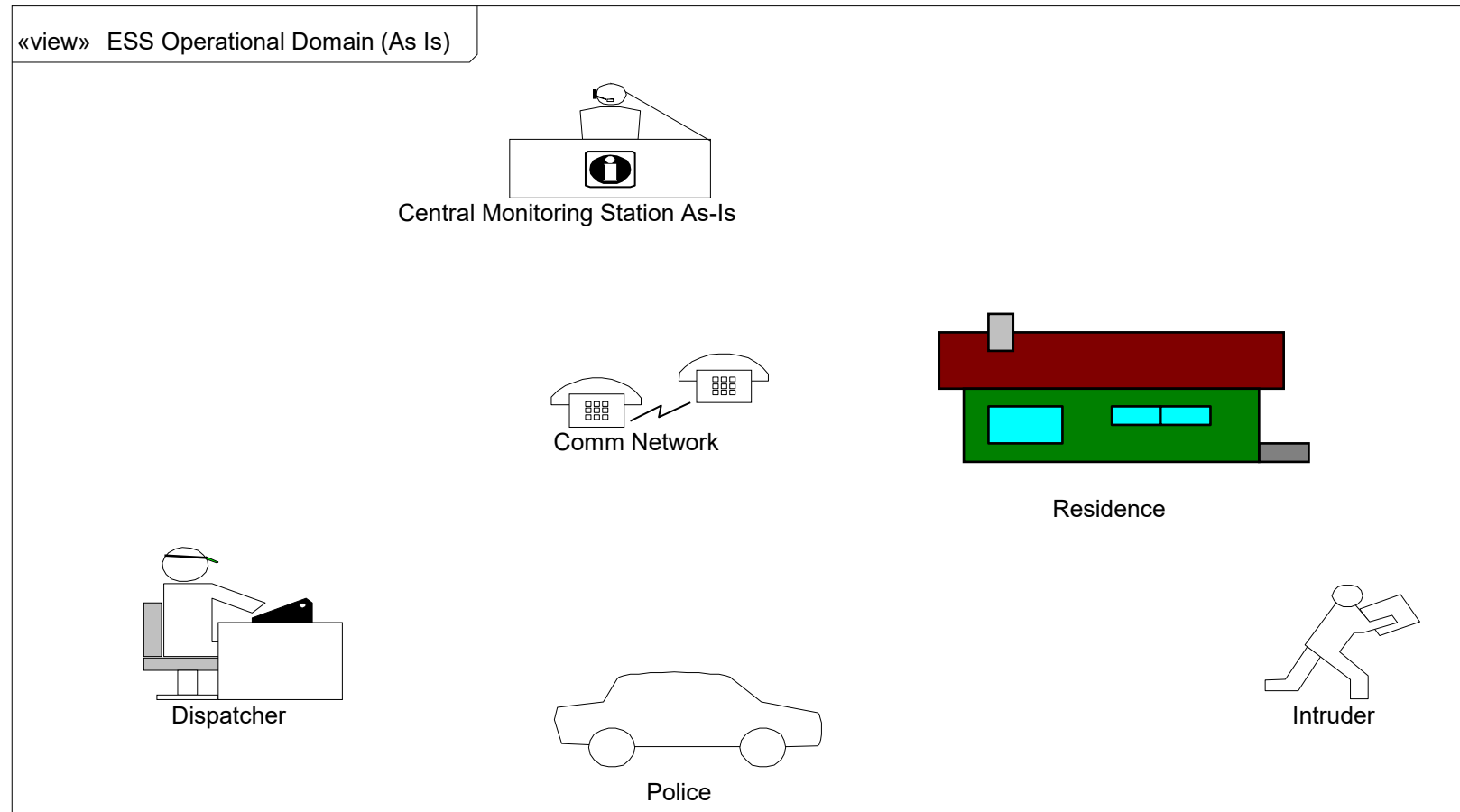
- Characterize as-is system/enterprise and elicit stakeholder needs
 - Current limitations and capabilities
 - Potential for reuse
- Specify enterprise level
 - stakeholder needs
 - use cases
 - measures of effectiveness
 - enterprise requirements



Security System Example - Background

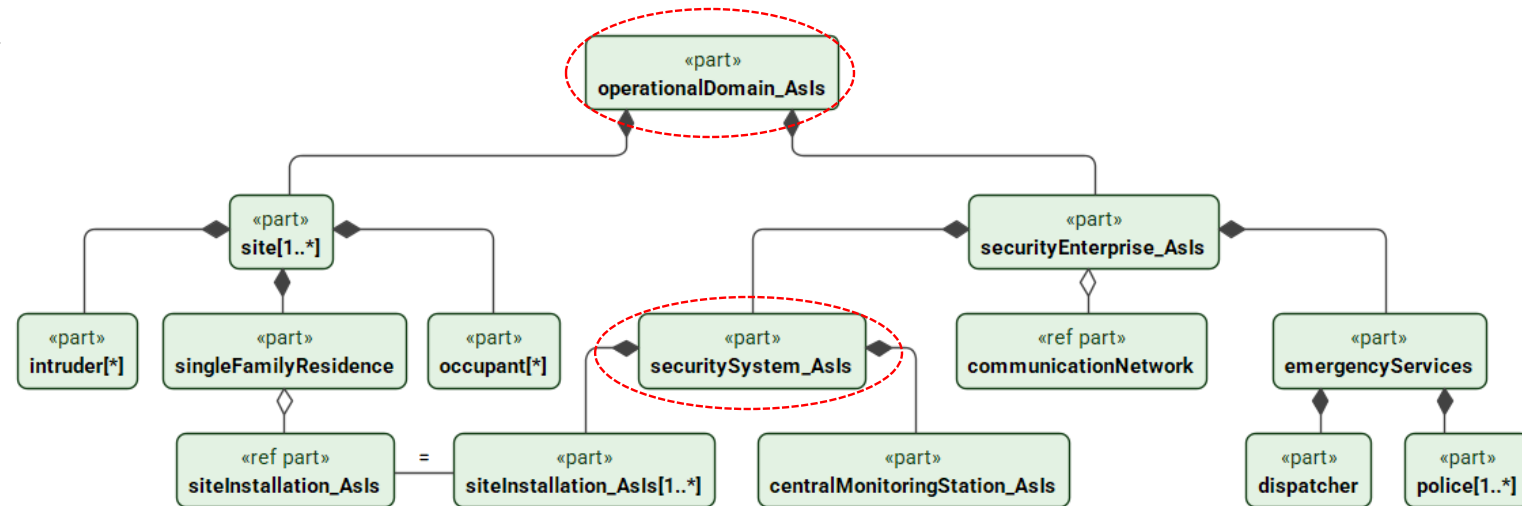
- Security Inc. has been in business for several years providing residential security systems
- Current system provides break-in detection, alarms, and reporting to a central system
- Company has been losing market share to competitors
- Security Inc. is initiating development of an **Enhanced Security System (ESS)** to increase its business

As-Is Operational Domain Concept



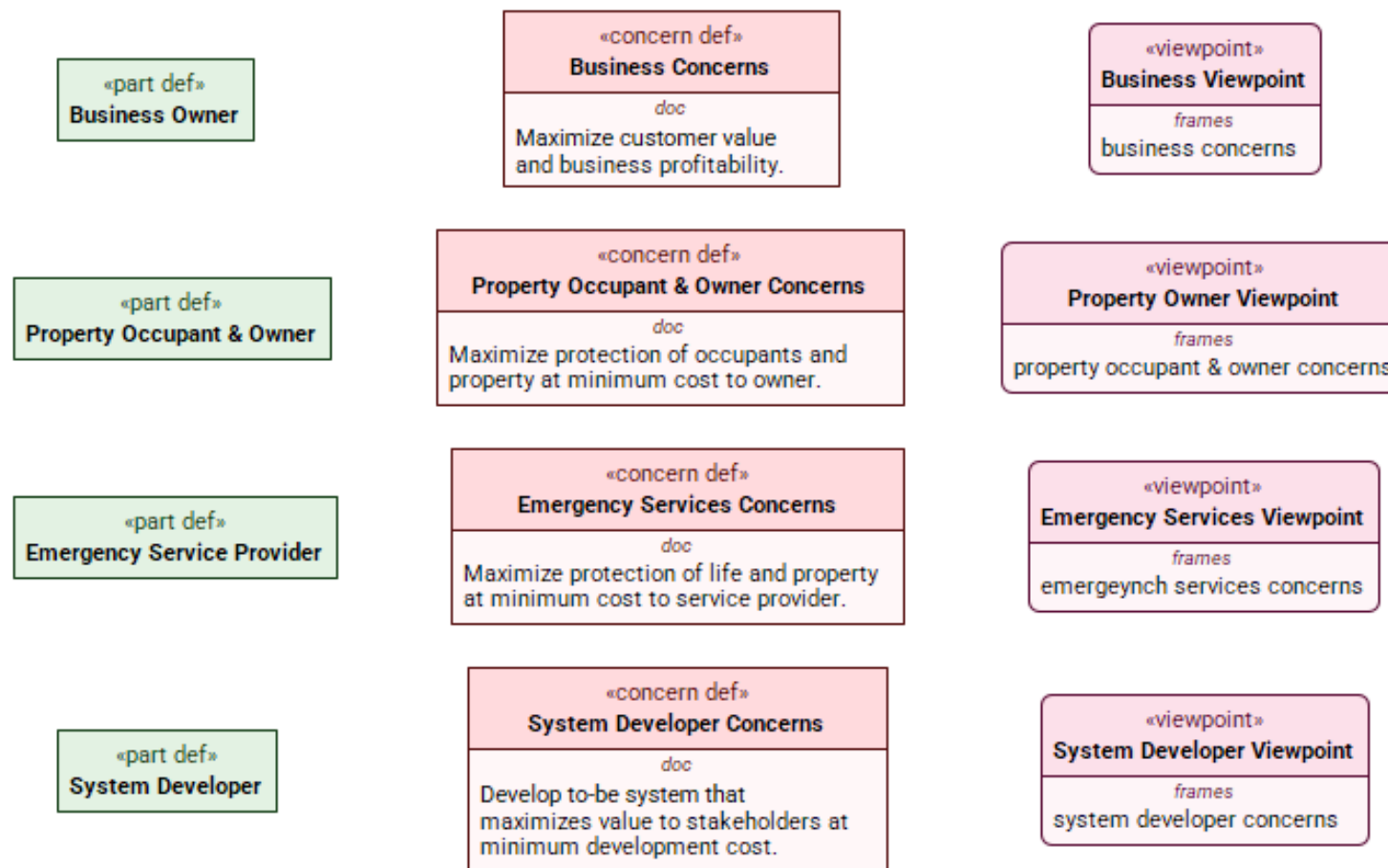
Characterize As-is System and Enterprise

- Model the current system sufficient to understand its limitations, capabilities, and areas for potential improvement
- The 'as-is' operational domain represents the 'system-of-interest' and external systems and users of the system (i.e. system context)
- The system of interest is the system being modified which may require changes to external systems



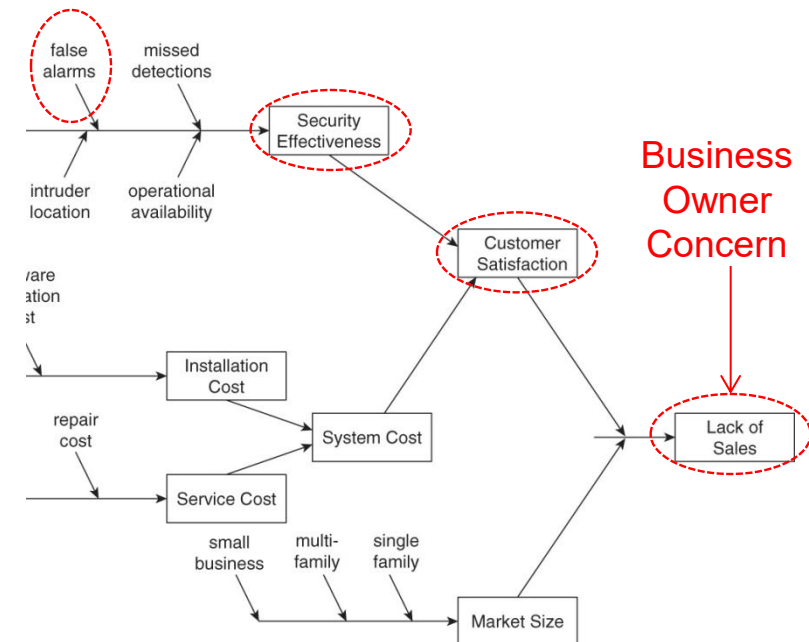
Identify Stakeholder Viewpoints

- Viewpoints reflect the concerns of each stakeholder (e.g. what they care about)

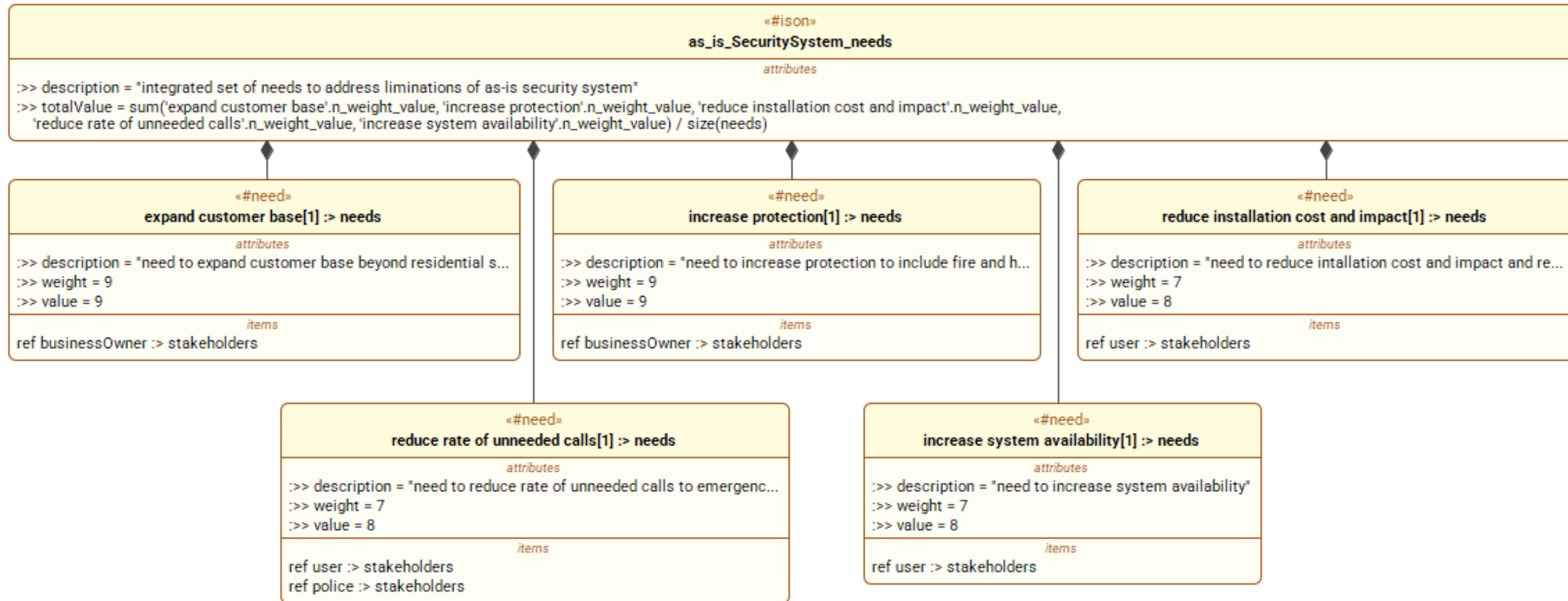


Perform Causal Analysis

- Assess current enterprise limitations and potential improvement areas
- Fishbone diagram represents cause-effect relationships among key system parameters and MOE's
- Performed for each primary stakeholder and their concerns
- Example:
 - High 'False Alarms' adversely impacts 'Security Effectiveness' which in turn impacts 'Customer Satisfaction', which leads to a 'Lack of Sales'



Capture Stakeholder Needs



Show in tabular format

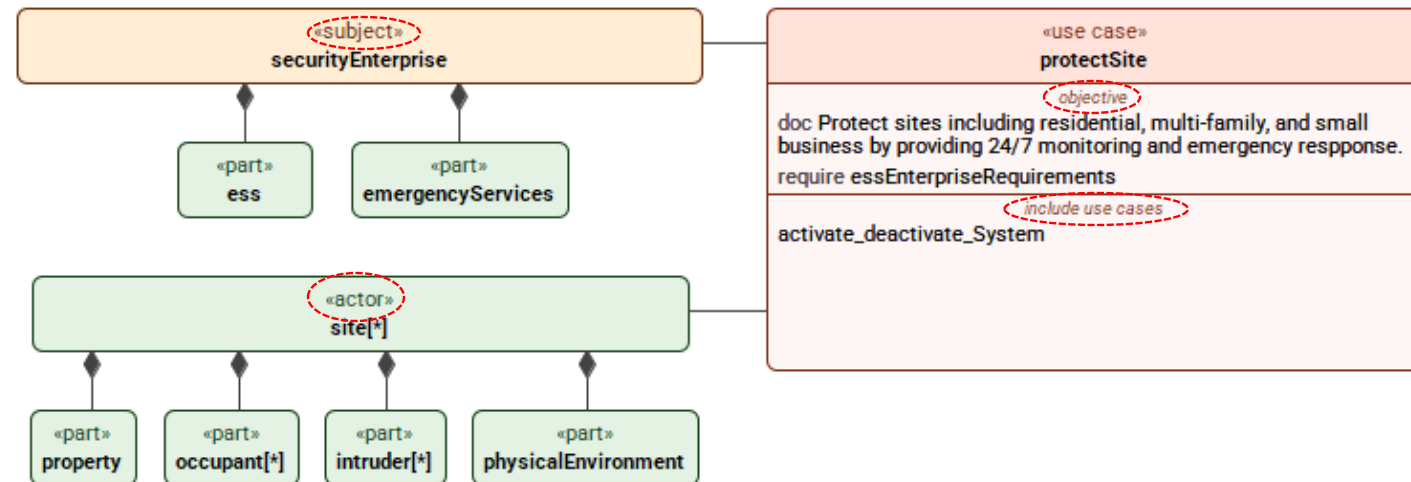
INCOSE Identify Measures of Effectiveness (MOE)

- MOEs reflect stakeholder value
 - Include availability, emergency response time, operational cost
 - Derived from the causal analysis and stakeholder needs
 - Evaluation function establishes relative weighting for each moe
- MOE values established via analysis
- Critical system attributes are derived from MOE's and assigned to the system and its parts

```
calc evaluationFunction {  
  in attribute availability :Real;  
  in attribute operationalCost :Real;  
  attribute w1 :Real;  
  attribute w2 :Real;  
  attribute w3 :Real;  
  attribute w4 :Real;  
  return attribute effectiveness :Real =  
    w1*availability  
    + w2*emergencyResponseTime  
    + w3*operationalCost;  
}
```

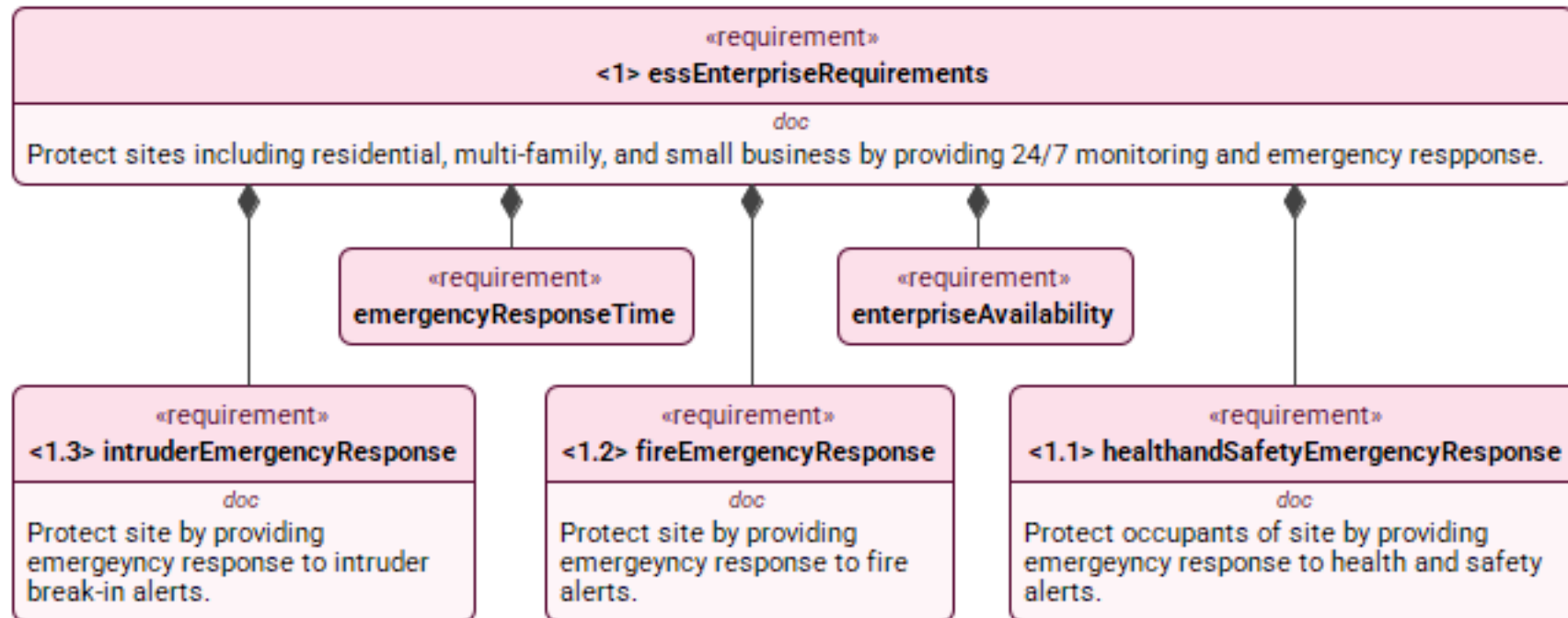
Define Enterprise Use Cases

- Defines enterprise goals to serve the *primary actors*
- Use case participants
 - The subject is the enterprise
 - The actors are external to the enterprise
- The use case objective can include requirements
- A use case can include other use cases
- The use case scenarios specify ESS black-box functional requirements



Specify Enterprise Requirements

- Address stakeholder needs and limitations of the 'as-is' enterprise
- Requirements generally rendered in a requirements table



Summary

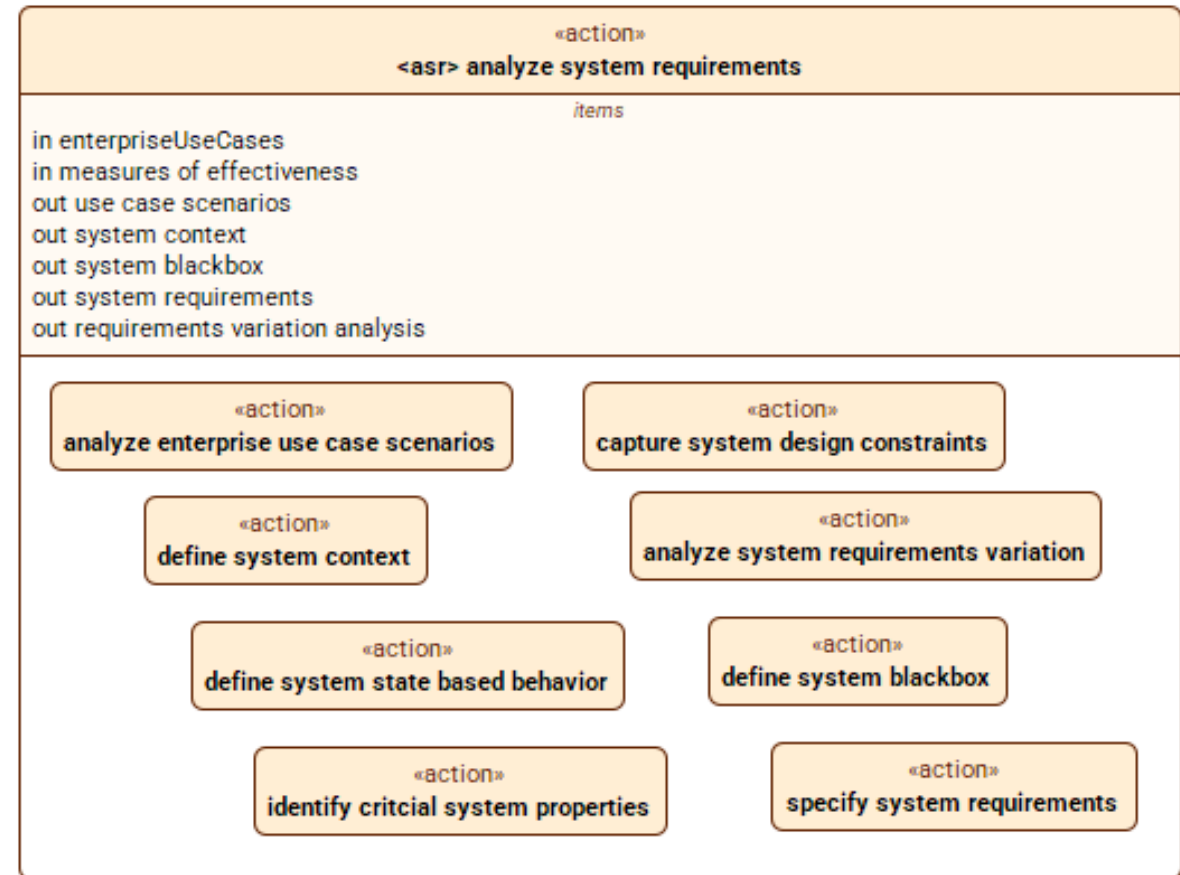
- Model as-is system and enterprise
- Analyze as-is system and enterprise to understand the problem and elicit stakeholder needs
 - Causal analysis, stakeholder viewpoints, stakeholder needs
- Specify capabilities and requirements to address limitations of the as-is enterprise
 - Enterprise use cases
 - Measures of effectiveness
 - Enterprise requirements

Analyze System Requirements

- Specify the system requirements in a concise manner
 - Every extra requirement costs
 - Every missed requirement costs
 - Ambiguity costs
- Deficiencies in system requirements propagate to large numbers of derived element and component requirements

Analyze System Requirements

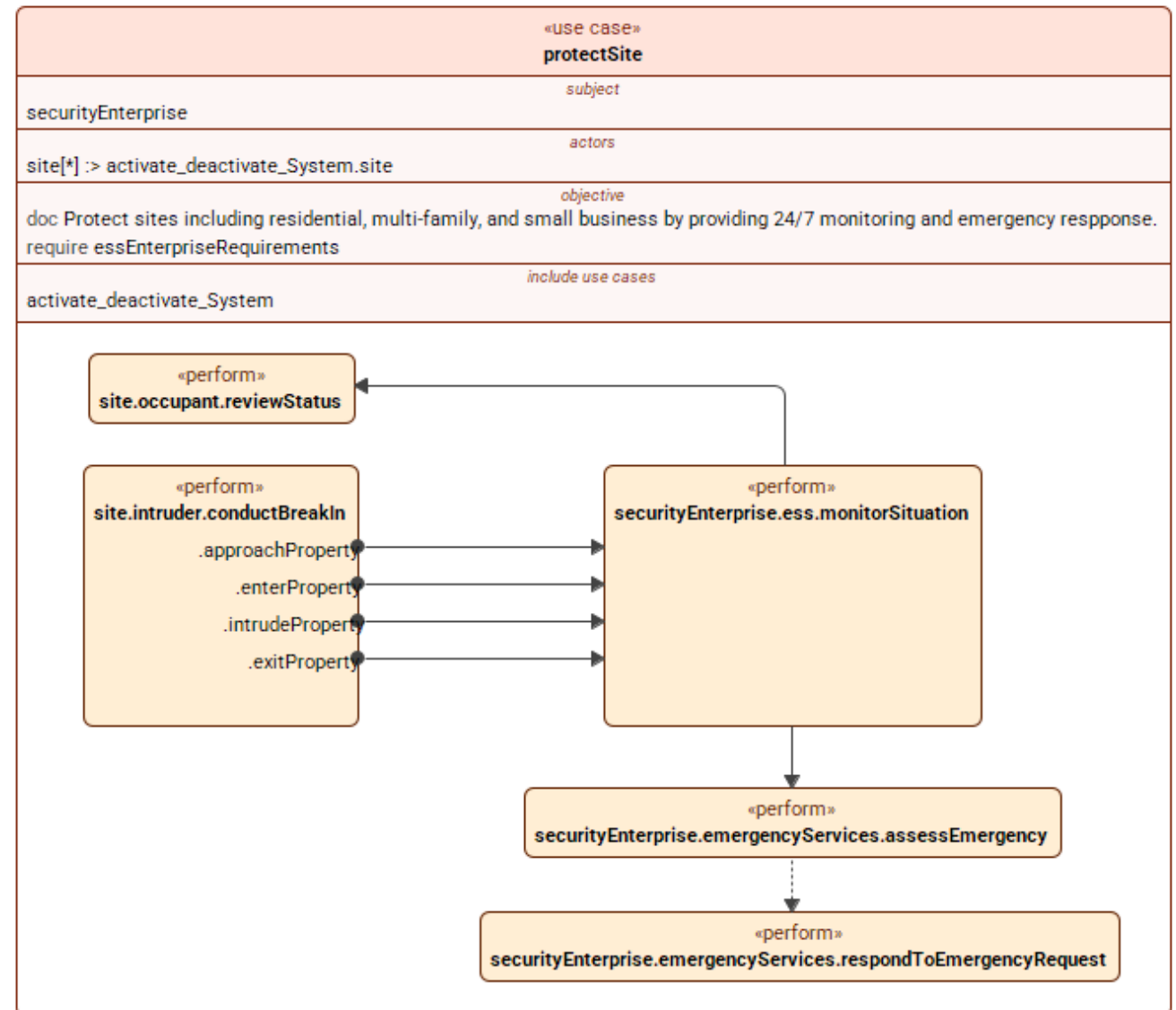
- Perform use case scenario analysis
- Define the system context
- Define the system black box that will support satisfying the enterprise requirements
 - Attributes (e.g., critical performance, physical, and quality characteristics)
 - Ports (i.e., interface requirements)
 - Actions (i.e., functional & i/o requirements)
 - States (i.e., states and modes)
 - Items (e.g., data and energy stores)
- System requirements constrain black box features (e.g., mass <= massMax)



Use Case Scenario

Protect Site

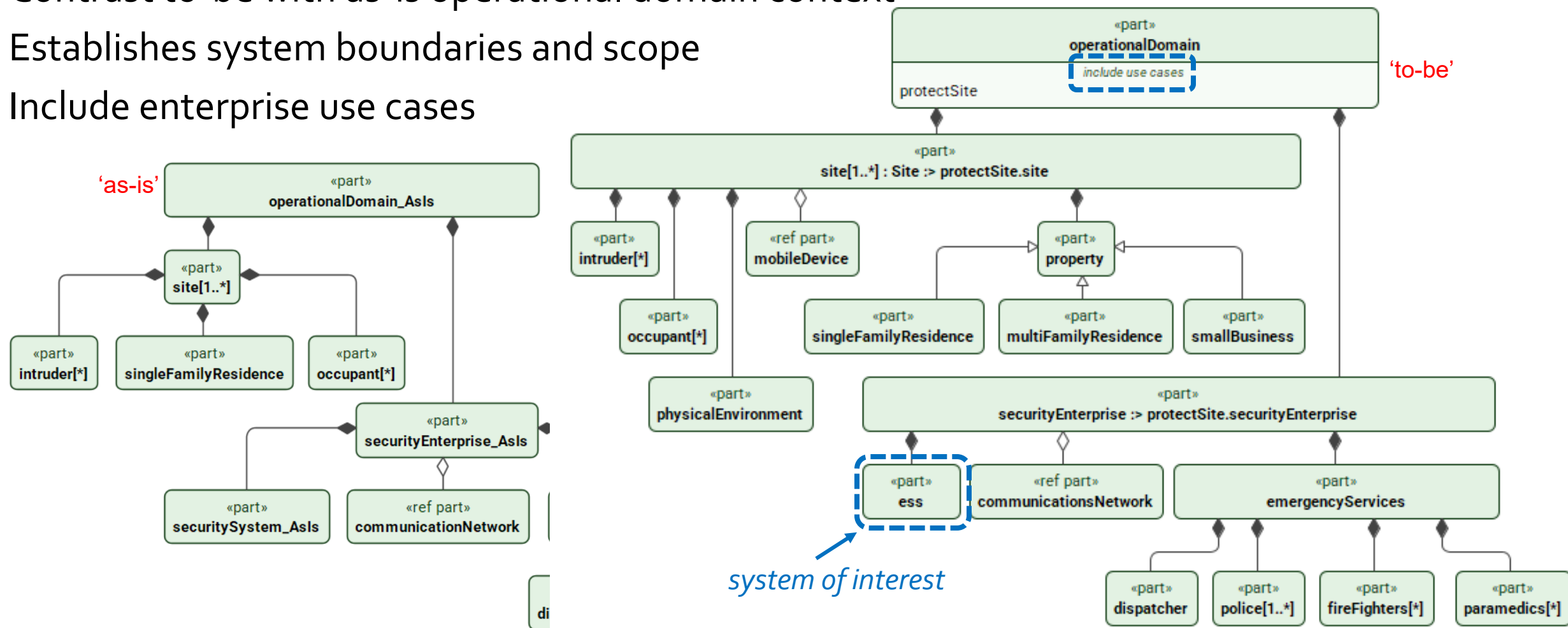
- Action flow specifies how system interacts with external systems and users for each use case scenario
- Swim lanes depict the system, external systems and users (**not shown**)
- Each swim lane contains the actions the entity must perform
- Output connected to input by a flow
- Off nominal paths can also be specified using decision nodes



Show using nested swim lanes

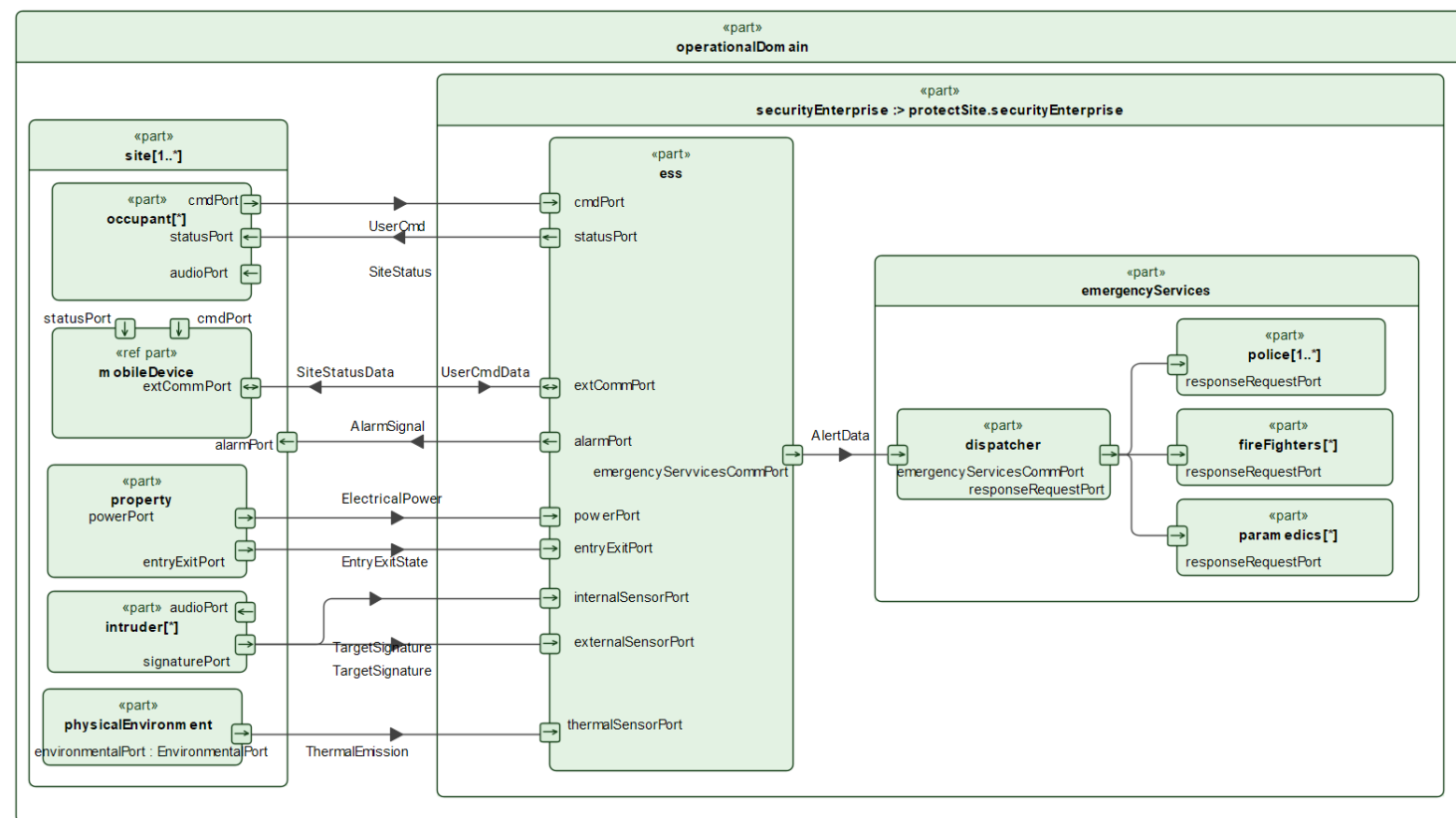
Operational Domain as System Context

- Contrast to-be with as-is operational domain context
- Establishes system boundaries and scope
- Include enterprise use cases

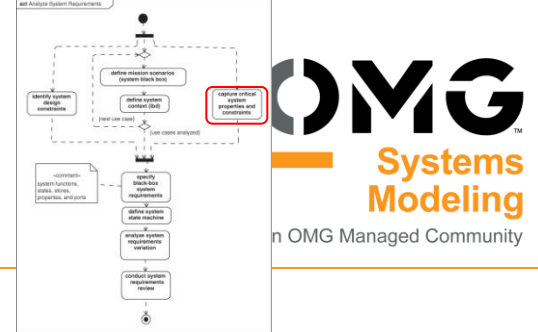


ESS Interconnection with External Systems

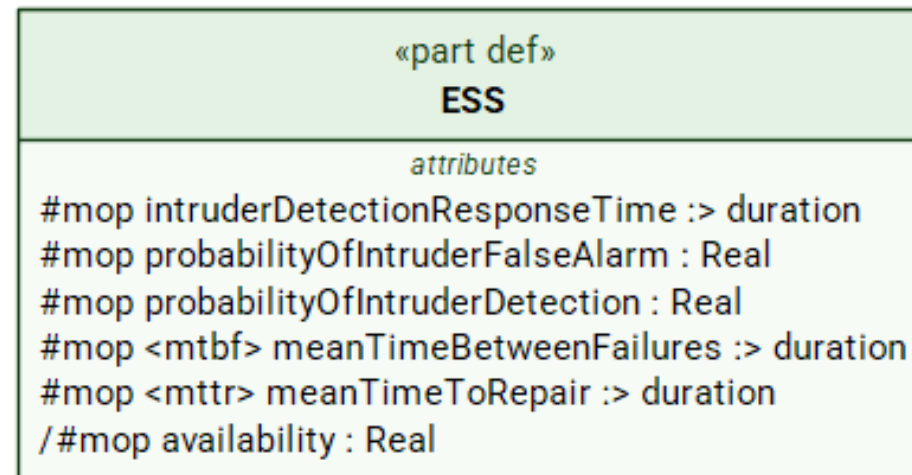
- Specifies the required interfaces between the system of interest (i.e. ess) and the external systems and/or users
 - Inputs/Outputs from activity diagrams are allocated to flows on connections
 - Ports specify the external interfaces



Capture Critical System Properties

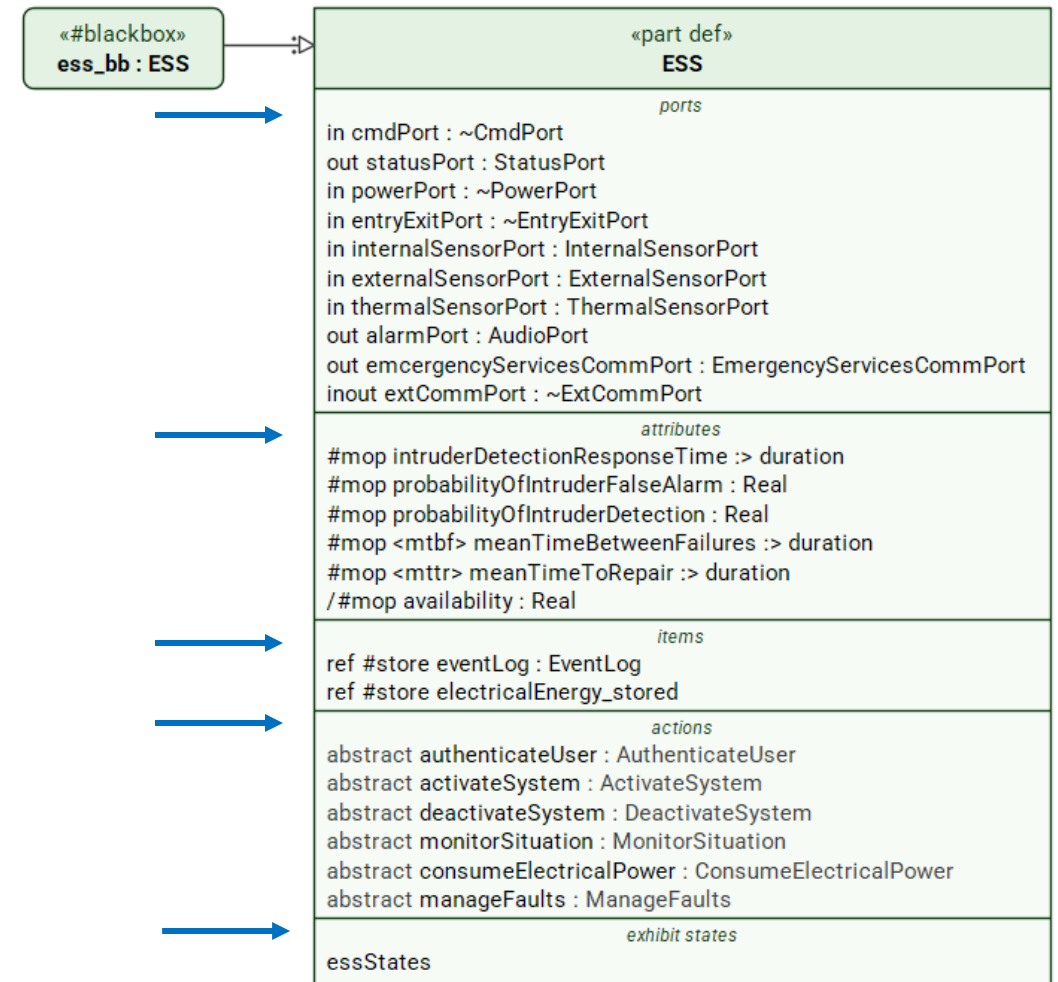


- Refactor the features of the part ess into the part definition ESS
- Critical performance, physical, and other non-functional requirements are captured as attributes of the system (e.g., intruder detection response time)
 - Derived from analysis models for each MOE



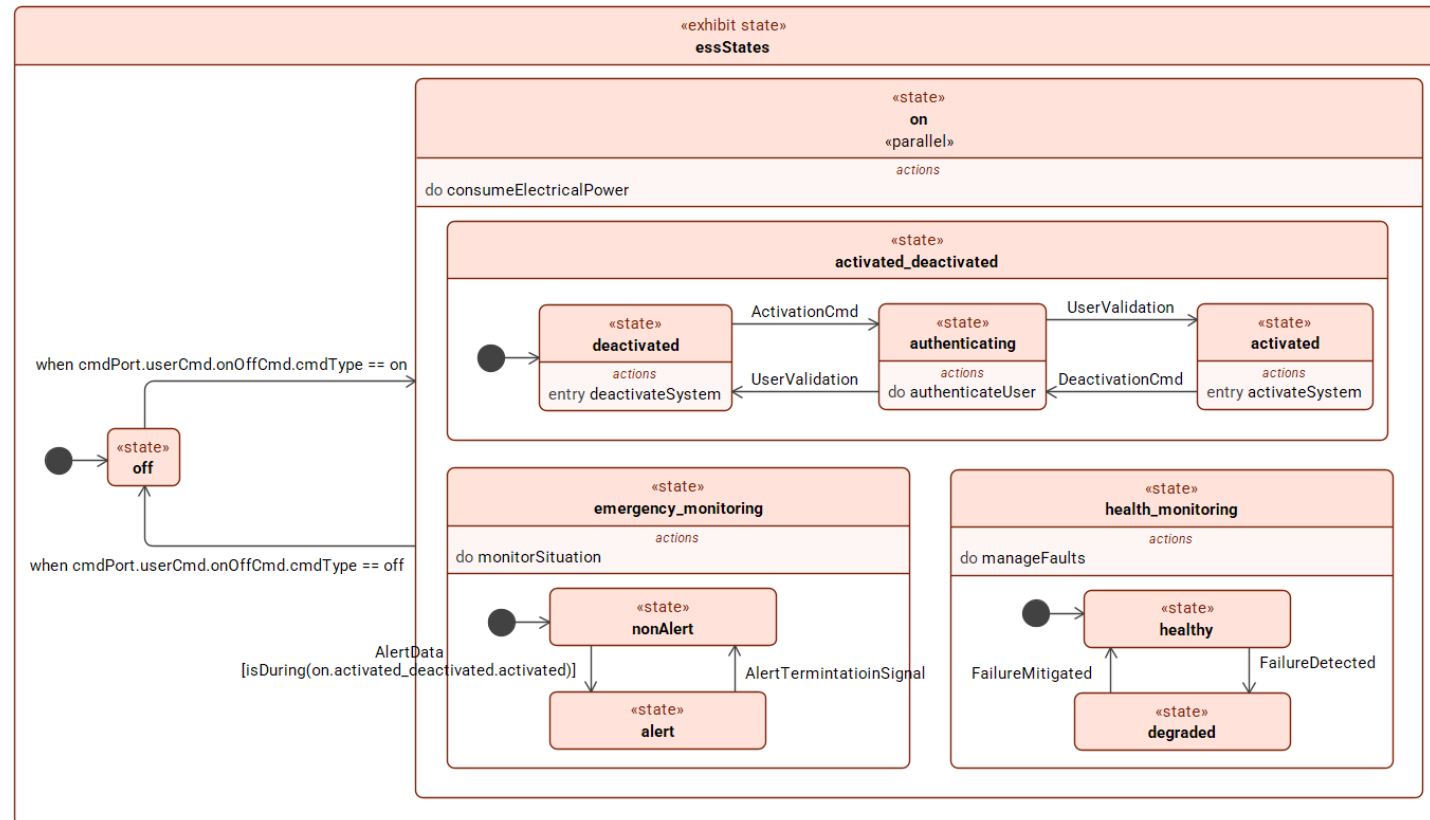
Define ESS Black Box

- Based on the scenario analysis, interconnection, and engineering analyses
- System's externally observable behavior and characteristics
- Black-box specification features include:
 - performance, physical, and quality attributes
 - ports (i.e. interfaces)
 - items that must be stored
 - actions (i.e. functions)
 - states

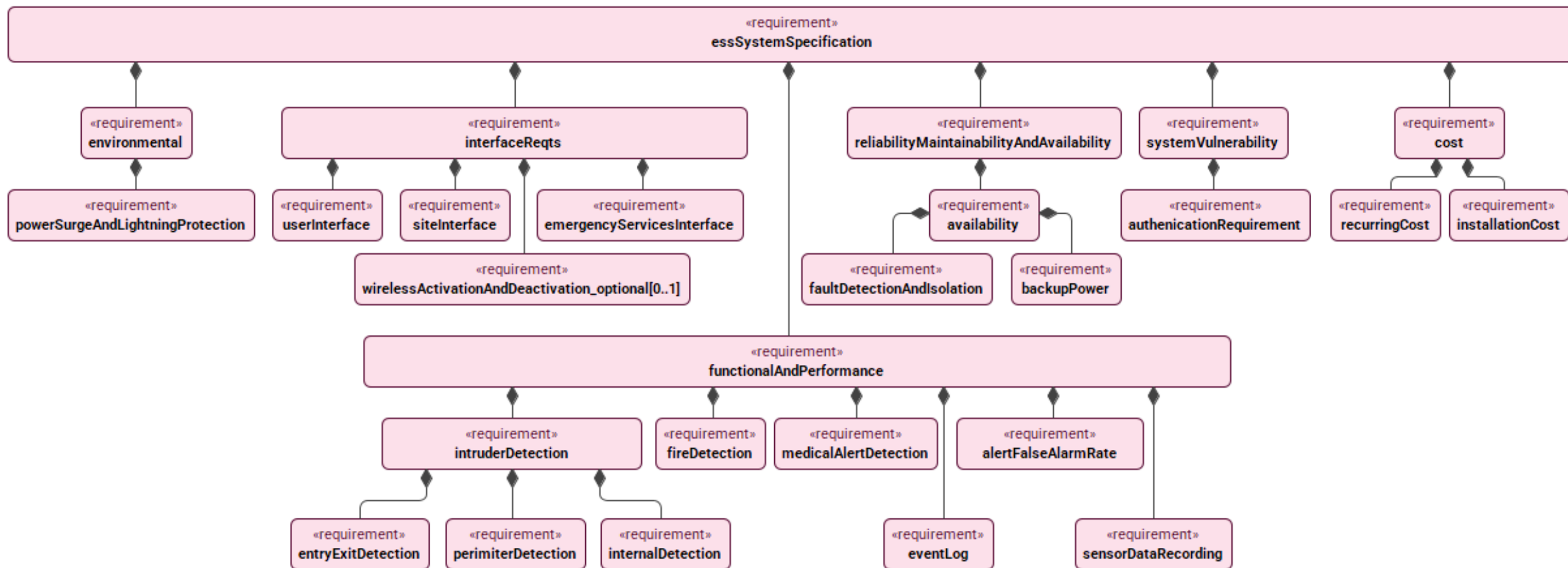


ESS States

- Defines how the system responds to events and enable actions
- Represents the composite behavior to support all scenarios

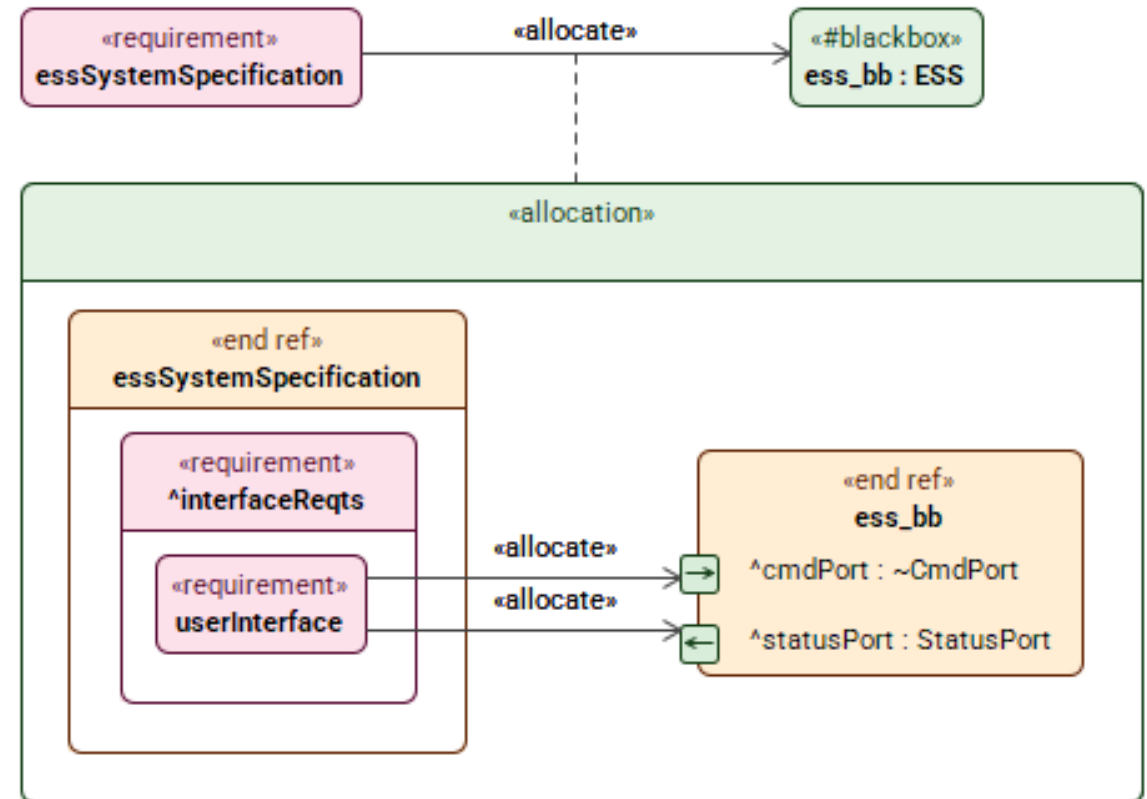


ESS Requirements



Requirements Allocation

- Allocate requirements to black box
 - Functional req'ts to actions
 - Interface req'ts to ports
 - Performance, physical, and quality characteristics/req'ts to attributes
 - States and modes req'ts to states
 - Persistent store req'ts to item stores
- Typically depicted in an allocation matrix
- White box satisfies requirements



This example shows the allocation of the user interface requirement to the cmd Port and the status Port

Analyze System Requirements Variation

- Requirements Variation Analysis is an evaluation of the probability that a requirement will change due to:
 - Changing interfaces
 - System growth
 - Increased performance needs
 - New functionality
 - COTS or technology upgrades
 - Changes in physical location and/or physical environment

Analyze System Requirements Variation (cont.)

- Assess each system black box feature
 - Define Probability of change for each requirement (High, Medium, Low)
 - Identify corresponding black box features
 - Evaluate sensitivity of MOEs to feature and requirements variation
 - Provide input to risk analysis
 - Risk measure = probability of change * criticality
- Mitigation
 - Ensure design can accommodate requirements variation

Identify System Design Constraints

- Design constraints are imposed on the solution space
 - hardware, software, data, operational procedures, interfaces, etc.
 - Example: the system must use xyz COTS hardware or software
- Should validate with applicable stakeholders that each constraint is needed
- Capture as a requirement (e.g. system shall use part xyz) with rationale
- Constraints are imposed on the physical architecture

Identify System Off Nominal Behavior

- Identify off-nominal behavior associated with each action
 - Example:
 - Nominal behavior to detect intruder
 - Off-nominal behavior is missed detection
- Off-nominal behavior is input to analysis that determines potential causes (e.g., sensor failure) and mitigation strategies (e.g., fault monitoring approach)

Summary

- System 'black-box' contains features the design must realize
 - Actions and item stores derived from scenario analysis
 - State-based behavior derived from scenario analysis to specify when actions must be enabled
 - Ports derived from system interconnection and scenario analysis
 - Performance, physical & quality attributes derived from engineering analysis
- Requirements allocated to black box features
- Additional black box analysis and assessment
 - Off-nominal behavior analysis
 - Requirements variation analysis (e.g., risk methodology) can contribute to a more robust and flexible design
 - Design constraints identified, validated, & satisfied by the architecture

Define Logical Architecture

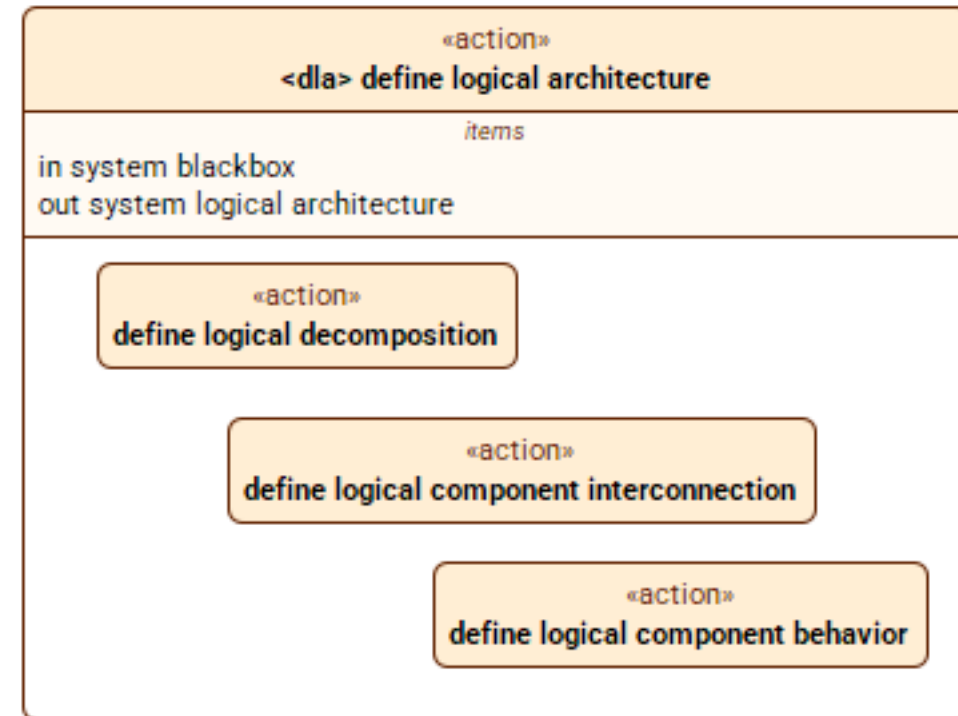
Logical Architecture

Motivation

- Partitions system design functionality and properties into logical components
 - Provides a shared understanding of what the system is and can do
- Provides an intermediate and stable abstraction layer between the requirements and the physical design
 - Supports trade studies of alternative physical architectures
 - Mitigates impact of change in requirements
 - Mitigates impact of change in physical design/technology
 - Supports design evolution
 - Can serve as a basis for developing a product line

Define Logical Architecture

- Decompose the system into logical components
- Define the interconnection between the logical components
- Define how the logical components interact to realize the system black box behavior



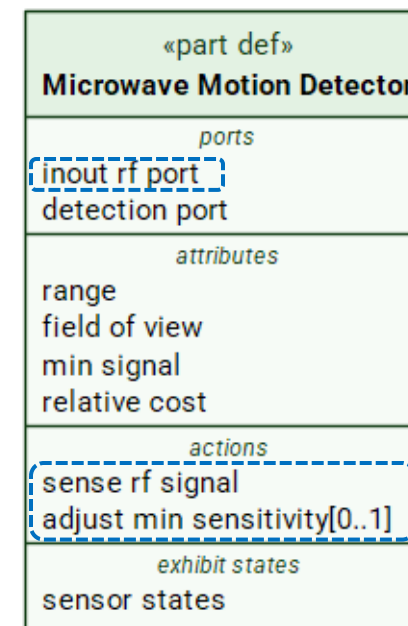
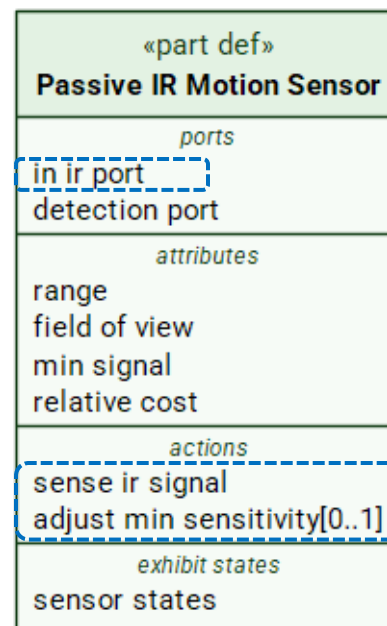
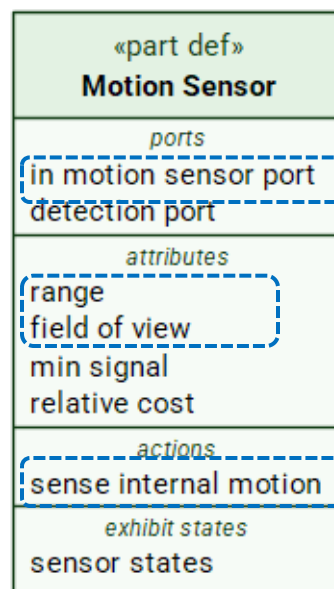
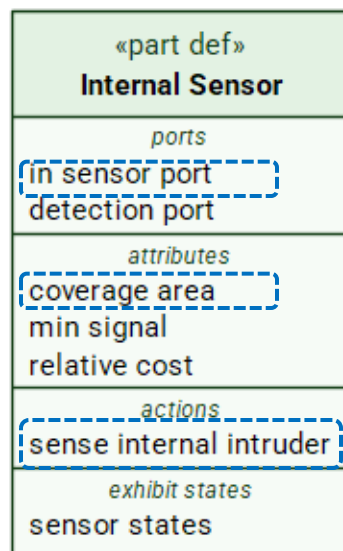
Logical vs. Physical Components

- On a spectrum from abstract to concrete
 - Increasing technology/implementation constraints on ports, actions, states, and attributes
 - Logical components are allocated to physical components (allocation can be m to n)

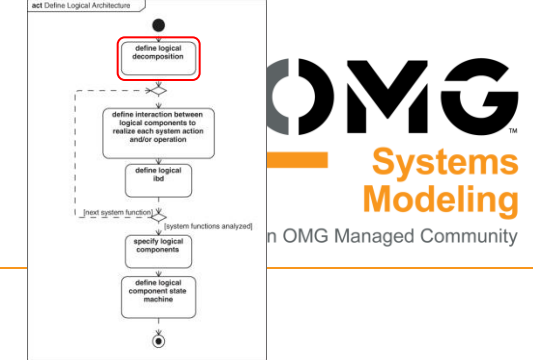
Logical Component



Physical Component



Define Logical Decomposition

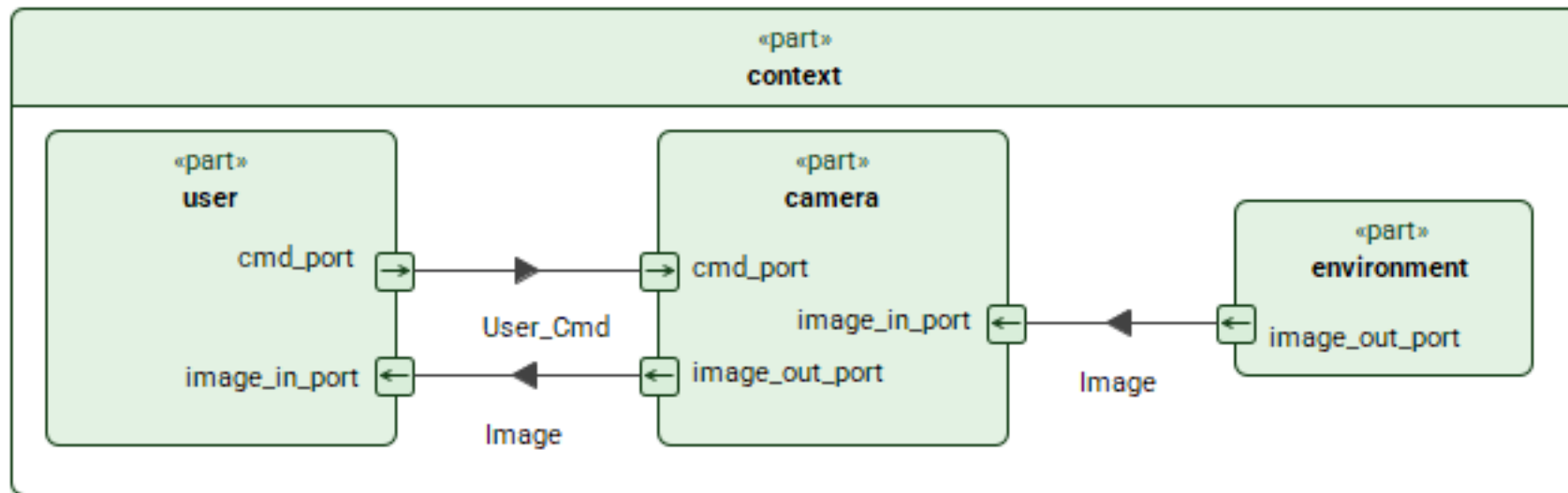


- ESS_logical specializes the ESS system black box
 - Inherits the black box specification features
- Use heuristics to decompose ESS_logical into logical components
 - **external interface mgr**: interface/connect to each external system, user, or environment
 - **input-output manager**: manage each functional input and output
 - **internal resource manager**: manage each internal resource
 - e.g., stored items, system states, internal connections, ..

Define Logical Decomposition

Camera Example (1 of 4)

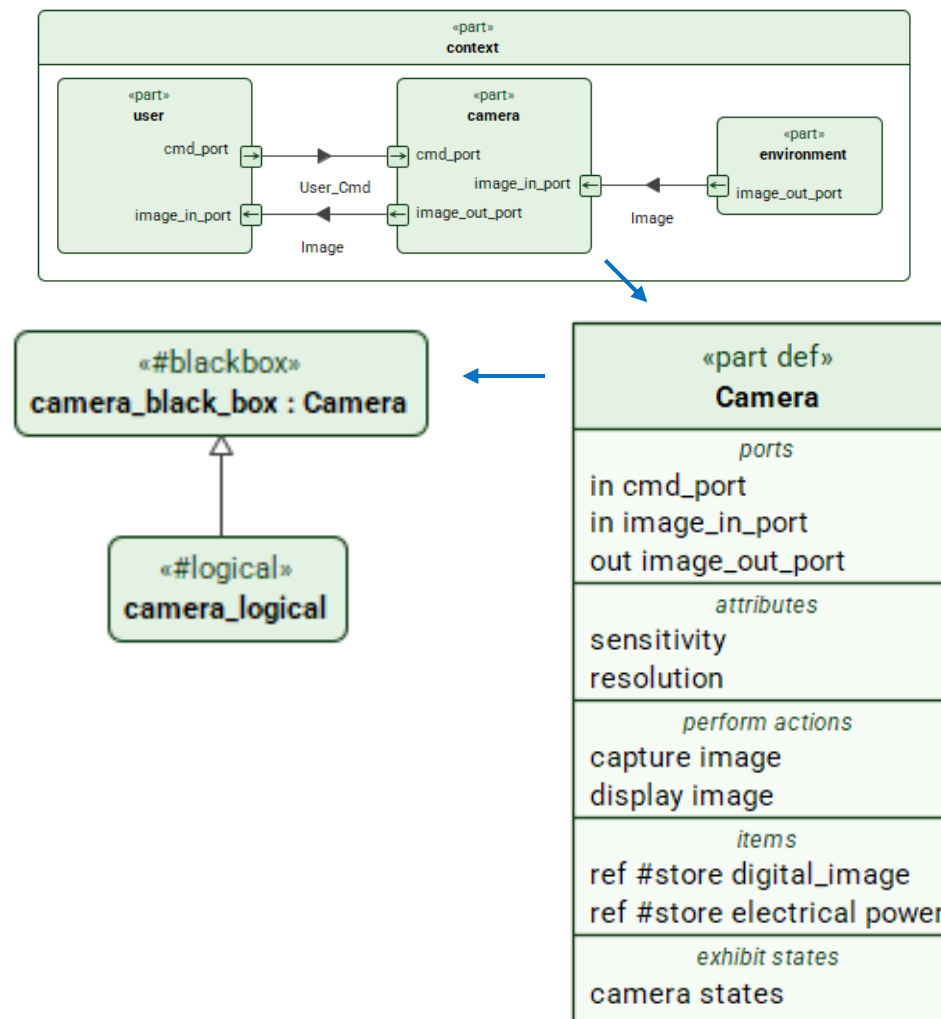
- Specify how the system interconnects with external systems, users, and environments as part of Analyze System Requirements



Define Logical Decomposition

Camera Example (2 of 4)

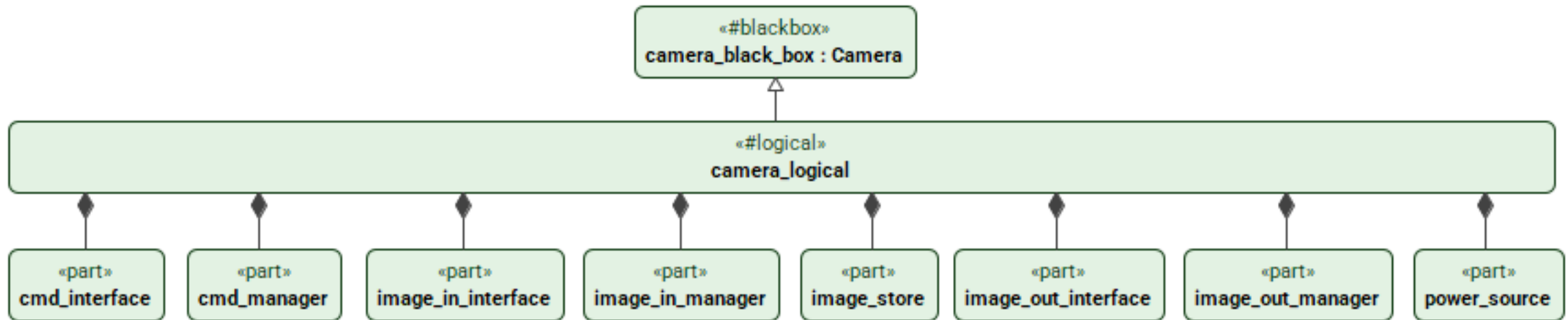
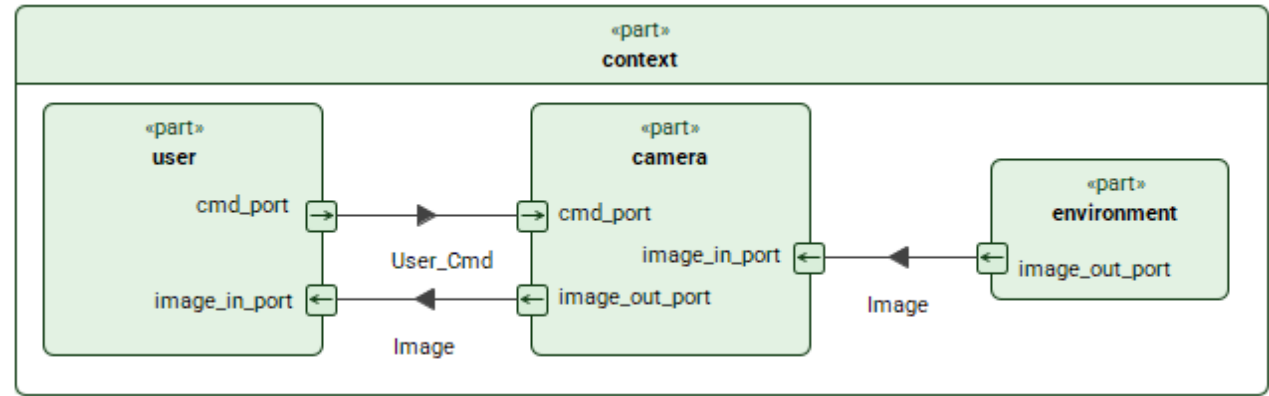
- The part definition Camera is a reusable element that contains the features of the part camera in the context
- The part camera_black_box defined by Camera is the specification
- The part camera_logical subsets camera_black_box and inherits its features



Define Logical Decomposition

Camera Example (3 of 4)

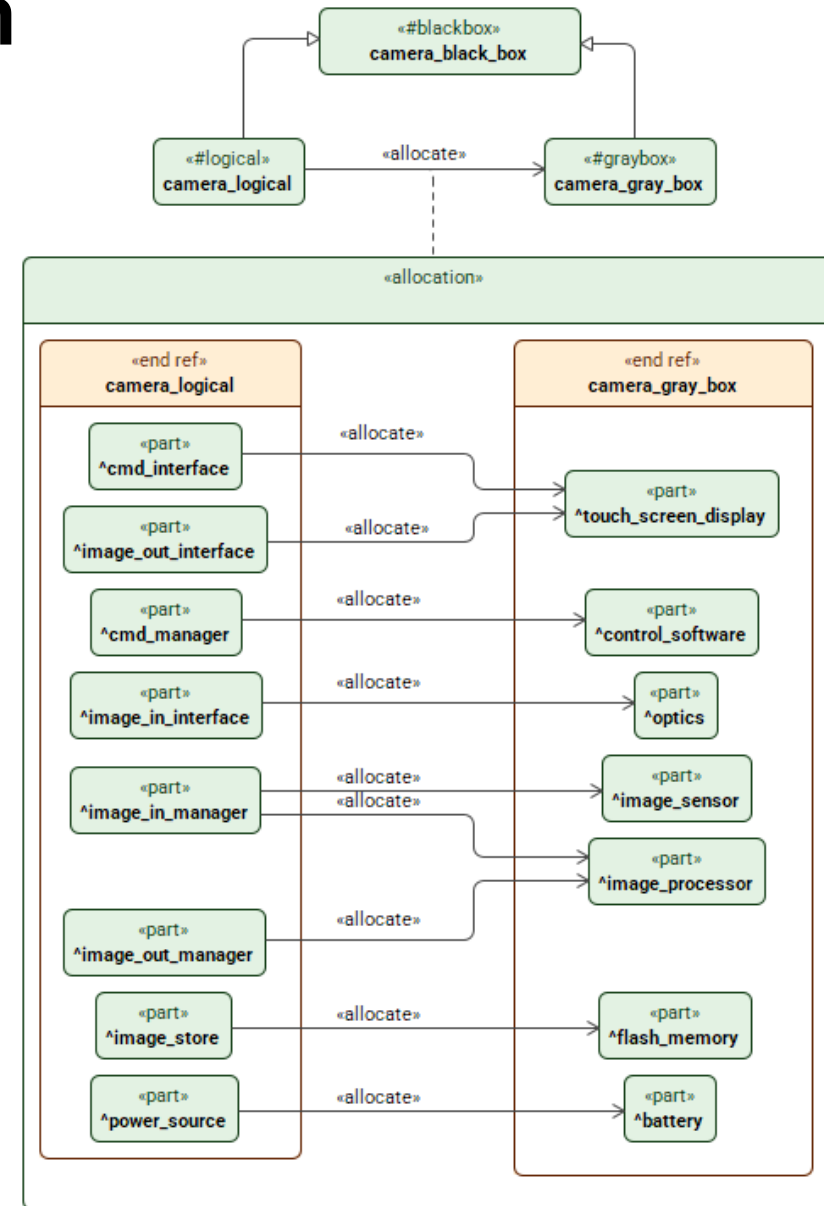
- Identify logical components
 - external interface components
 - Input-output managers
 - internal resource managers (stored power, stored image)



Logical to physical allocation

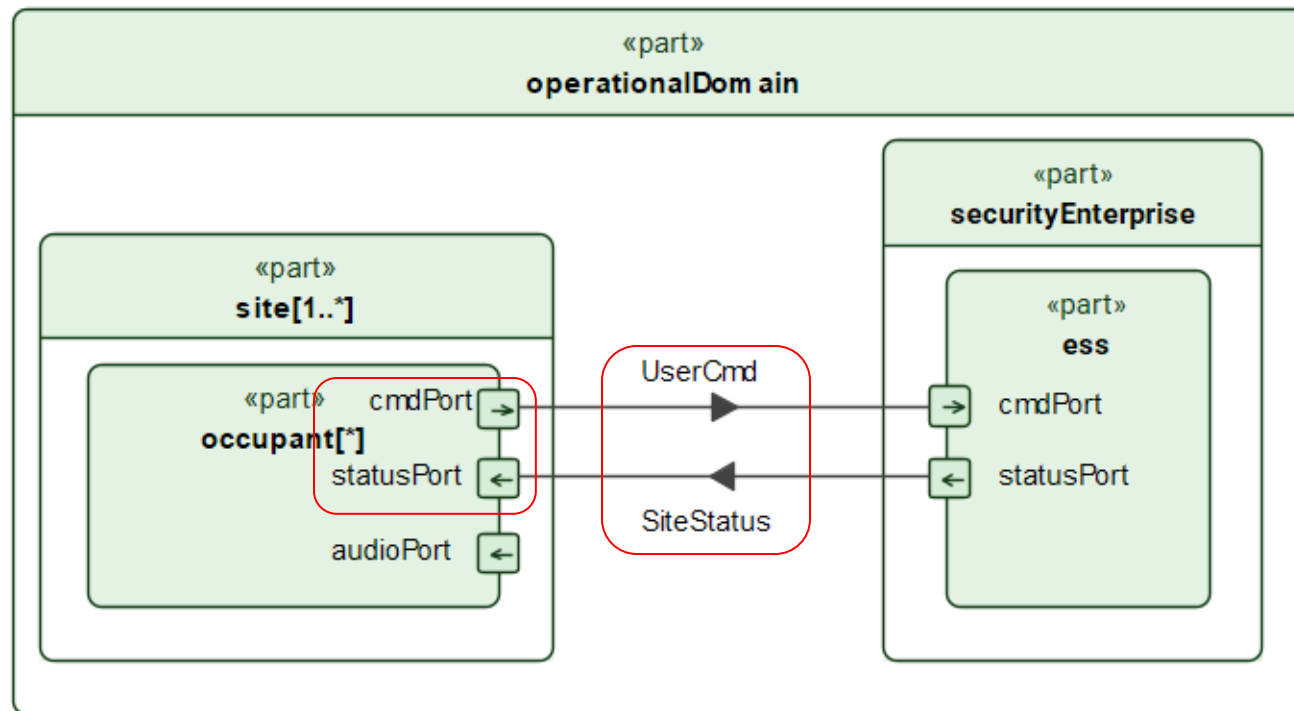
Camera Example (4 of 4)

- Logical to physical allocation occurs in the follow-on activity (*synthesize candidate physical architectures*)



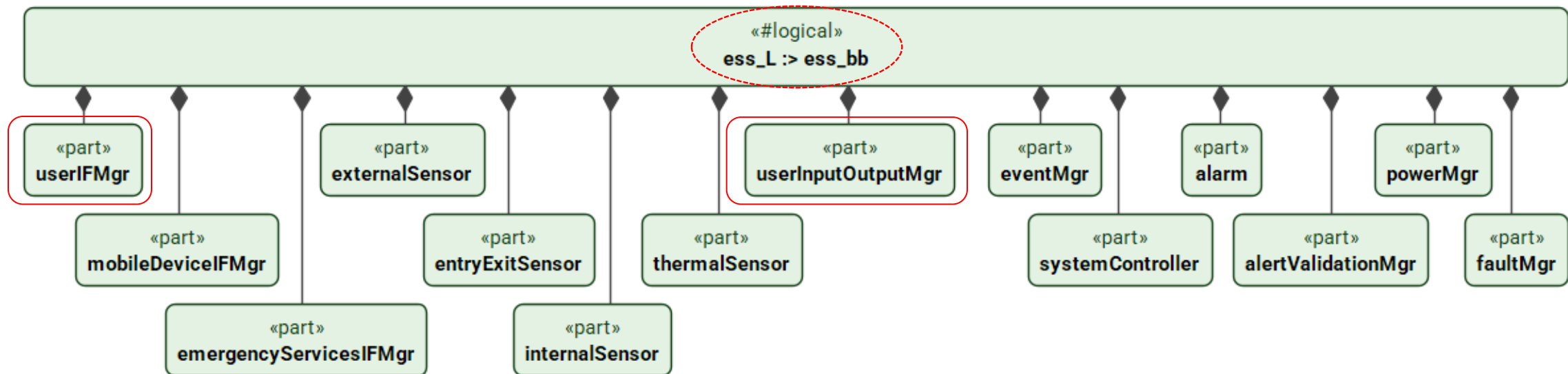
ESS Logical Decomposition

- Example of ESS flows and external interface from System Context that drives logical decomposition

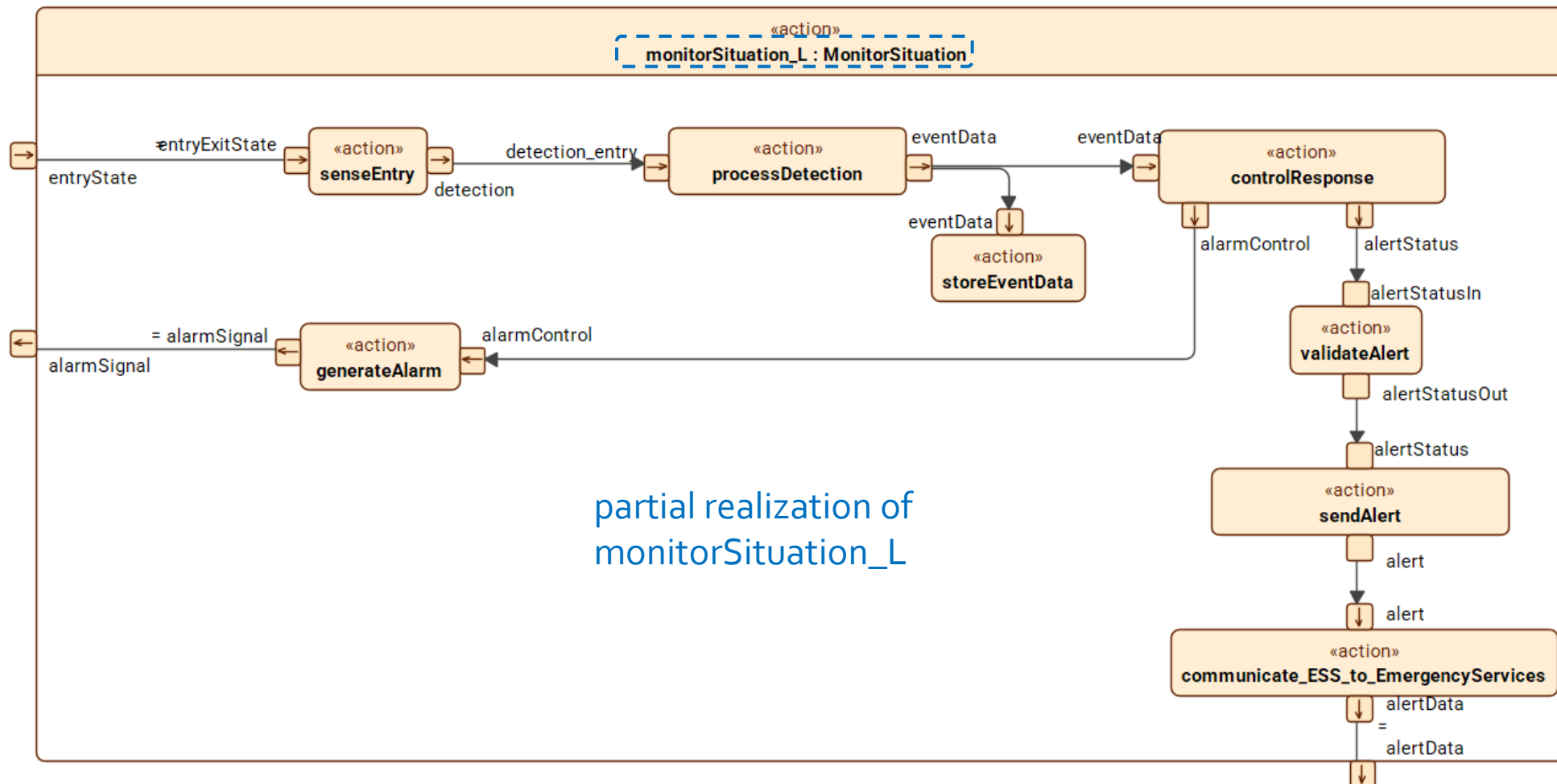


ESS Logical Decomposition (cont.)

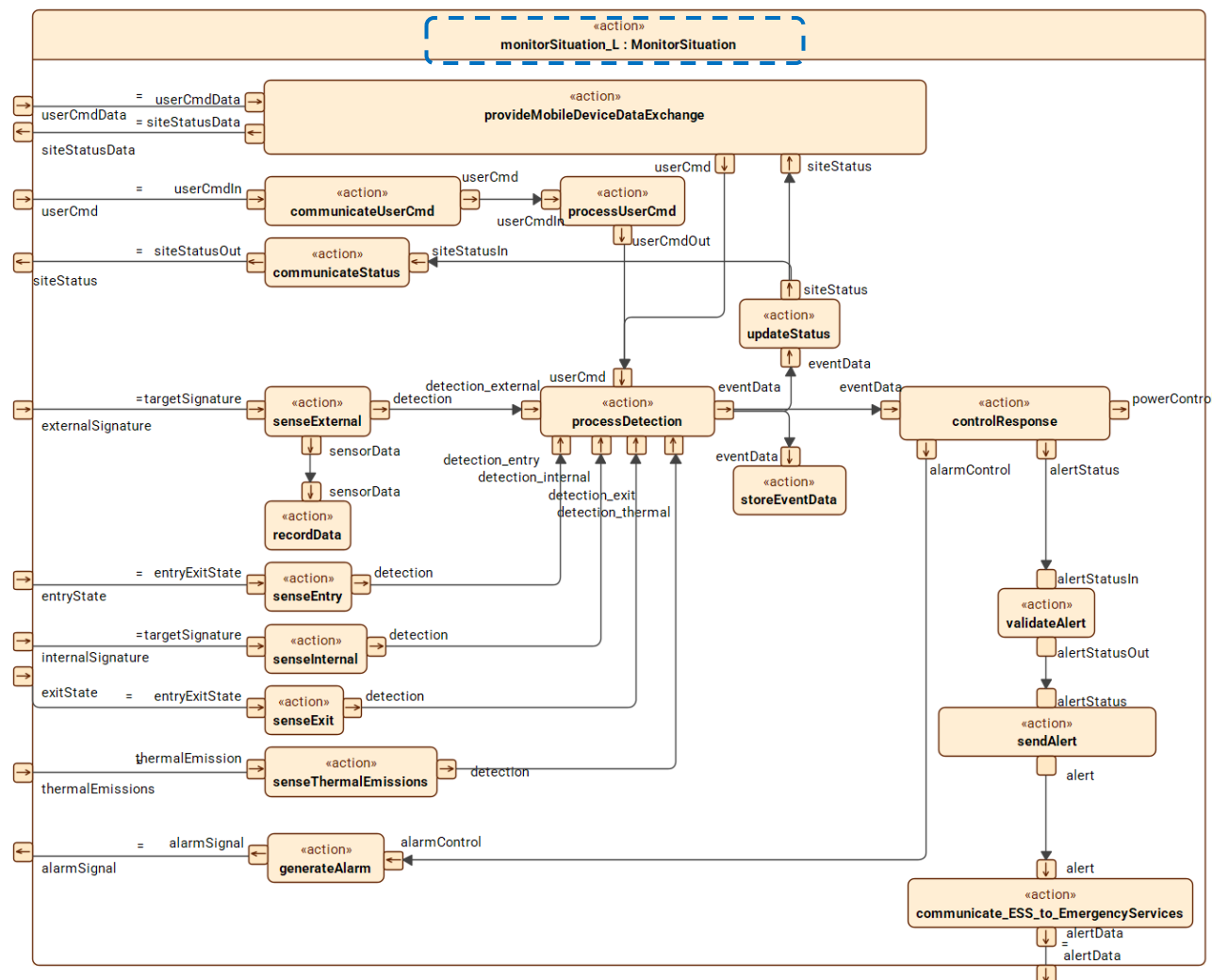
- Logical ess (ess_L) specializes ess black box
- Logical components support the following
 - Provide functionality to manage input User Cmd and output Site Status
 - Provide functionality to interface with occupant cmd port and status port



Define Action Flow (logical) for monitor situation

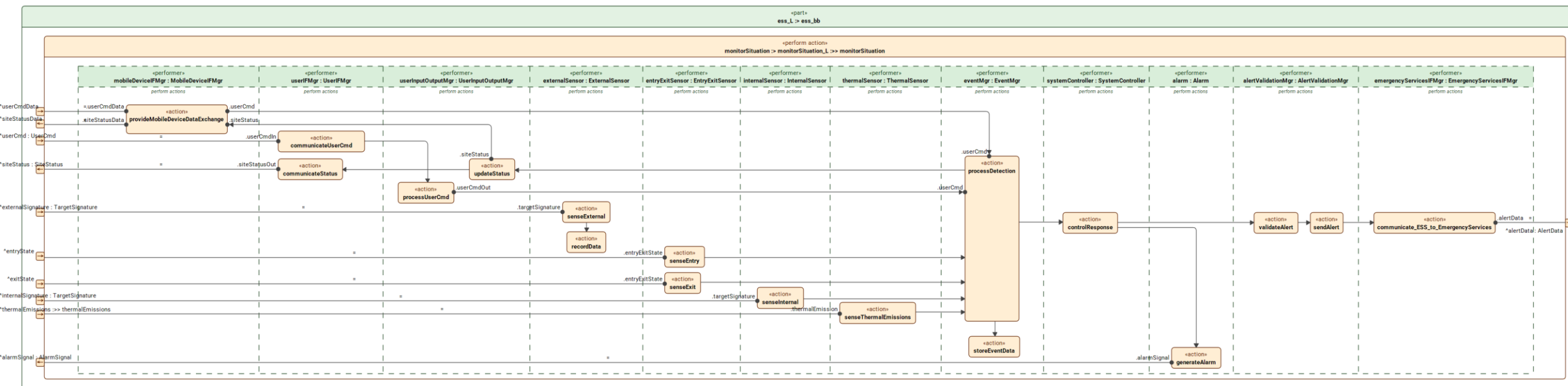


Define Action Flow (logical) for monitor situation (cont.)



more complete realization
of monitorSituation_L

Define Interaction between Logical Components with swim lanes

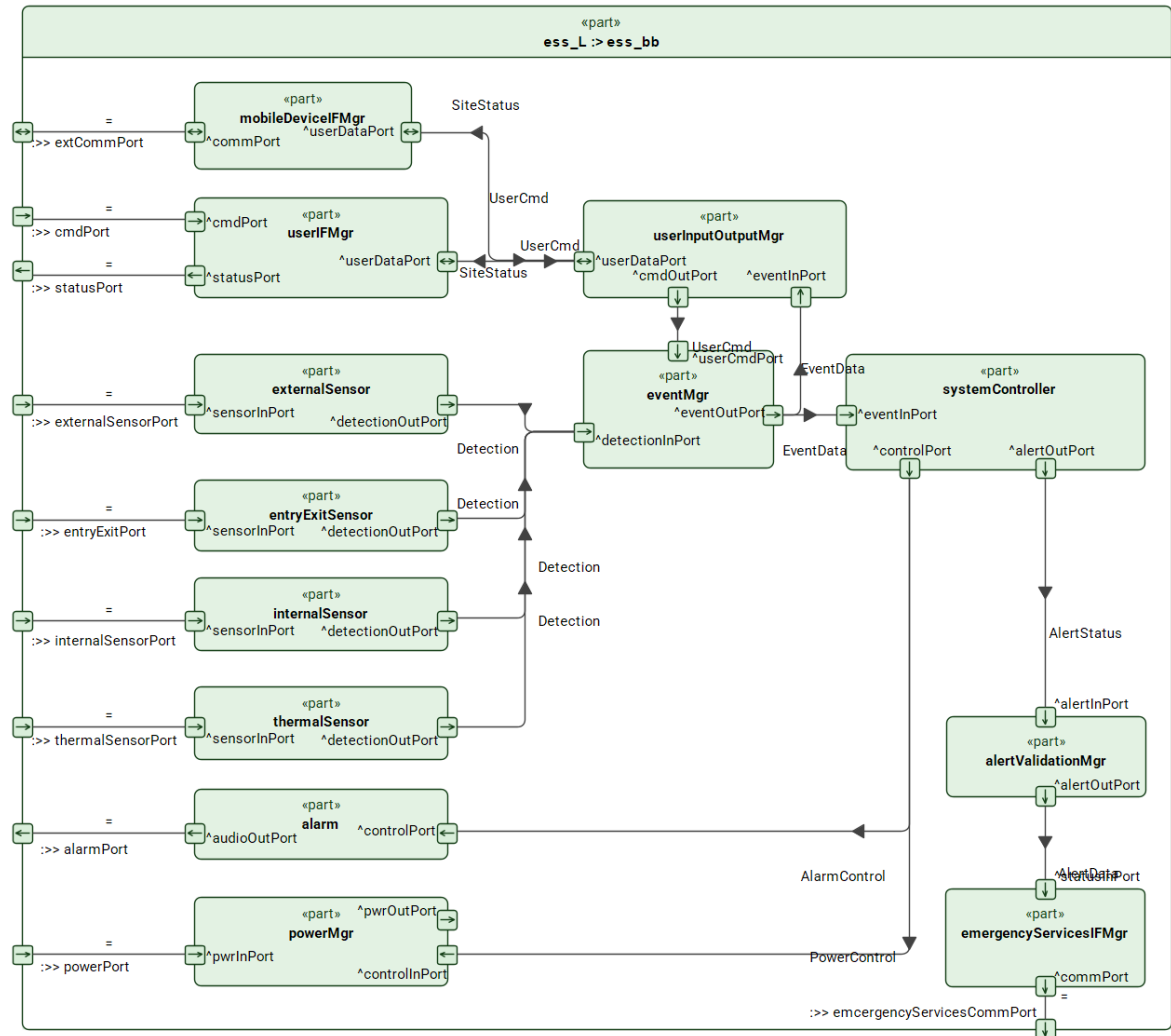


more complete realization
of monitorSituation_L
(logical-refactored)

applying case1 pattern
for perform actions

ESS Logical Interconnection

- Interconnection of logical components support interactions specified by the activity diagrams



Specify Logical Components

- Each logical component can be specified similar to the system black-box specification
 - Logical behavior captured as actions and states
 - Note: states that realize logical behavior control flows may be deferred to physical architecture
 - Logical connection points/flows captured as ports with directed features
 - Performance, physical, and quality characteristics as attributes
 - Persistent data, energy, material as store items

«part def» Internal Sensor
<i>ports</i> in sensor port detection port
<i>attributes</i> coverage area min signal relative cost
<i>actions</i> sense internal intruder
<i>exhibit states</i> sensor states

Summary

- The Logical Architecture is an abstraction level that aids in managing change, supports architecture trade-offs, and improved system understanding
- The system black box is decomposed into logical components
 - The logical decomposition approach provides a systematic method to identify the components needed to provide the system functionality
- Each black box system action is performed by the interaction between the logical components
- The logical interconnection view shows the how the logical components are connected to support the logical behavior

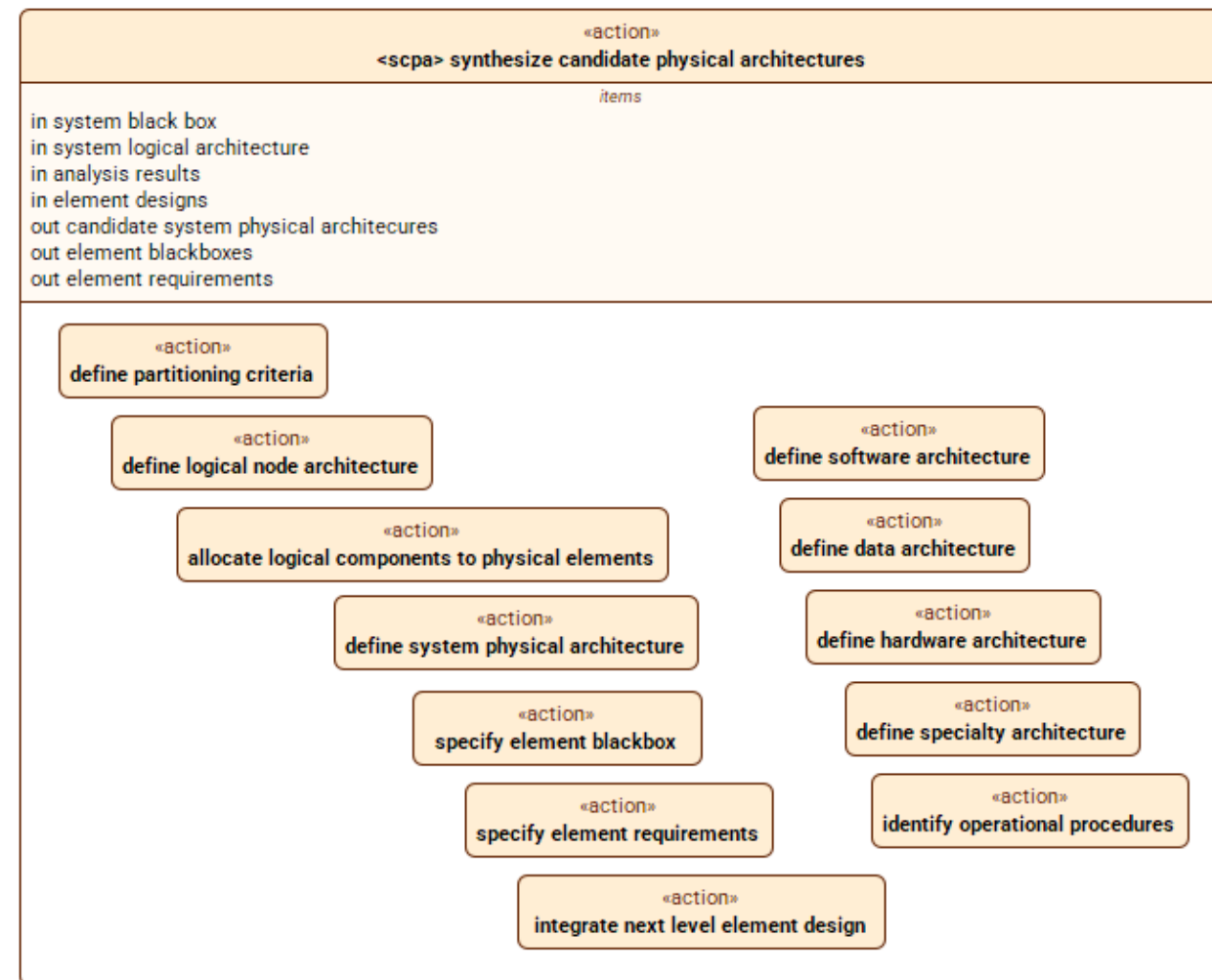
Synthesize Candidate Physical Architectures

Motivation

- Synthesize alternative physical architectures that realize the logical design to satisfy the system requirements
- Specify the physical components of the system including the hardware, software, data, and procedures
- Integrate the component designs into the system physical architecture

Synthesize Candidate Physical Architectures

- Define how logical components are distributed across nodes (if applicable)
- Allocate logical components, functions, and interfaces to physical elements, functions, and interfaces at each node
- Define the system physical architecture including the interconnection and interaction among physical elements (i.e., the gray box)
- Define hardware, software, data, and other cross-cutting views of the system architecture and specify the black box software, hardware, and data elements
- Integrate the next-level element design into the architecture (i.e., the white box)



Define Partitioning Criteria

- Define criteria to:
 - Partition functionality, data, and control among the logical and physical components
 - Partition the components among subsystems, nodes, and layers of the architecture
- Apply partitioning criteria to:
 - Result in designs that exhibit maximum cohesion and minimum coupling to reduce interface complexity
 - Reduce the impact of requirements and technology changes
 - Address key requirements such as performance, reliability, maintainability, and security
 - Leverage 'Design for X' techniques, (such as Design for Maintainability) to improve design quality

Partitioning Examples

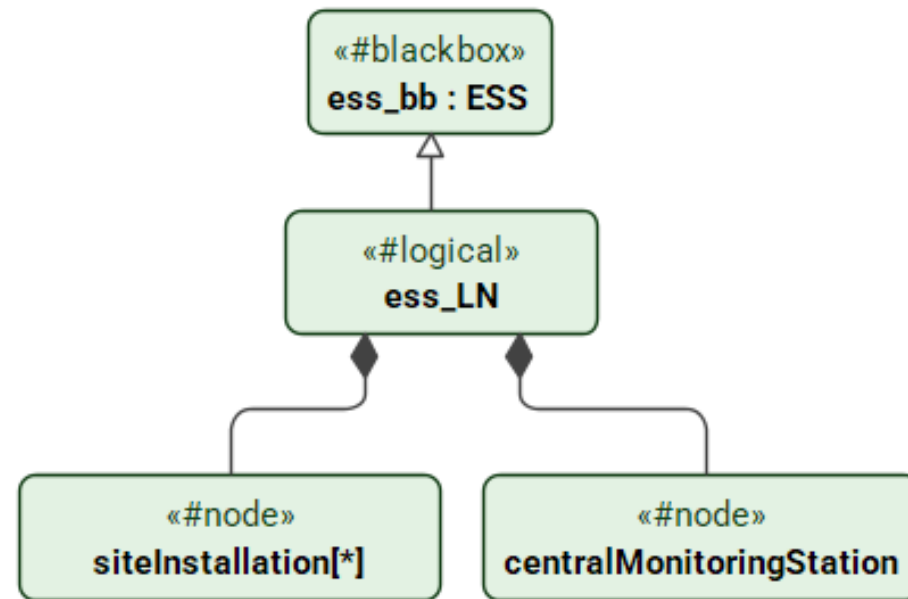
- ESS Example
 - Event Detection Manager and System Controller partition common functionality for detecting events and controlling actions in response to inputs from multiple sensors
 - Can add sensors (e.g. carbon monoxide) with minimal impact
- Other example
 - Higher failure rate parts or other parts that require maintenance should be physically partitioned so that they can readily be accessed and replaced

Partitioning Criteria

- Partitioning criteria may address:
 - Modularity (maximize cohesion, minimize coupling)
 - Component isolation (encapsulation, hiding)
 - Reuse, COTS, and design constraints
 - Common functions, inputs and outputs
 - Data (dynamic vs. static, shared)
 - Performance (accuracy, response time, throughput)
 - Temporal (concurrency, update rates, phasing, events)
 - Reliability/maintainability (failure rate, time to repair)
 - Producability (e.g. design for assembly)
 - Safety and security (access control)
 - Physical/environmental (e.g. location, power)
 - Design for change (dependencies)
 - Subcontractor/development responsibility
 - Life cycle phase or development increment

Define Nodes

- nodes used to partition components based on their location or other criteria
- ess_LN (i.e., Logical-Node) specializes the ess black box and contains the nodes
 - nodes include the site Installations and the central monitoring station



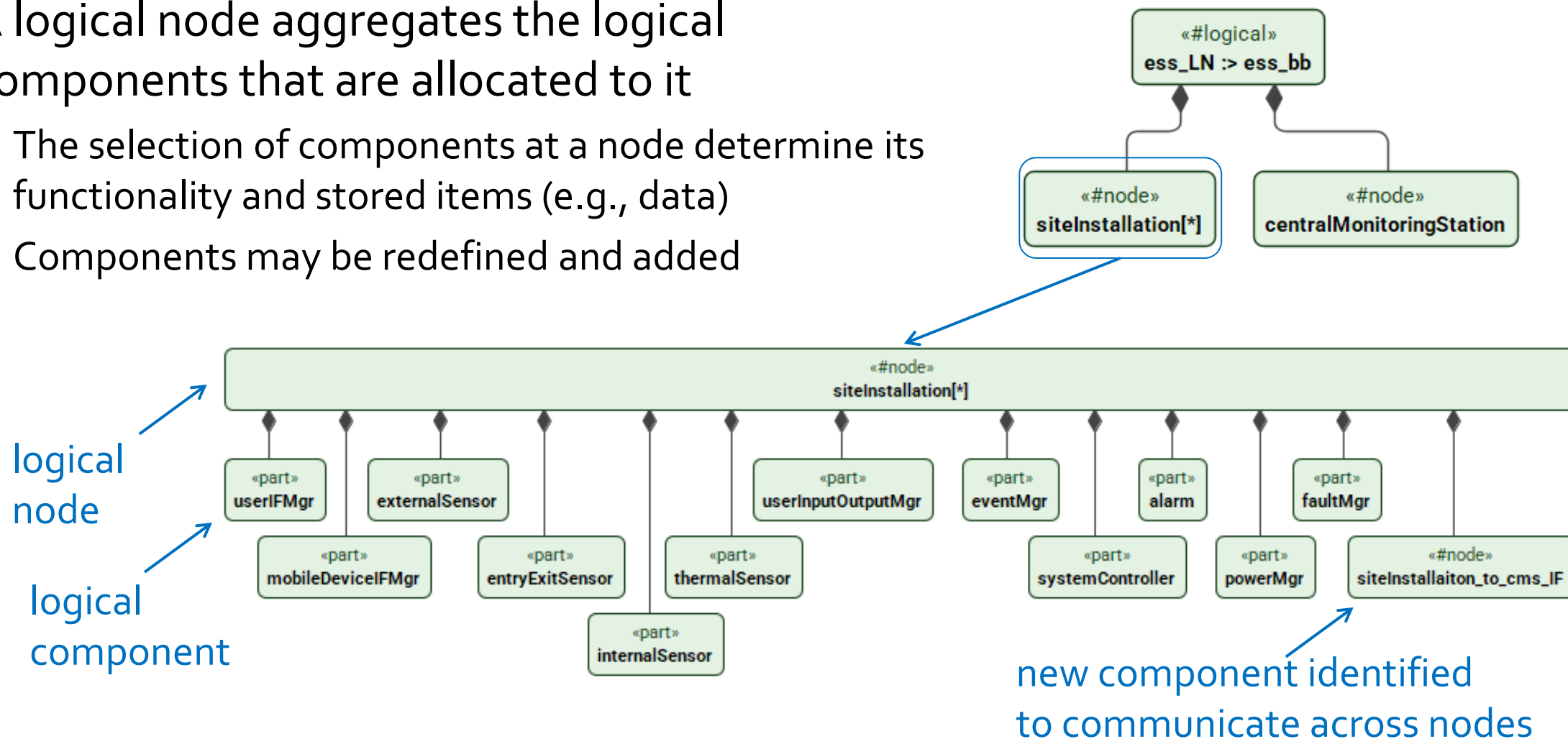
Define Logical Node Architecture

- Allocate logical components to nodes to distribute functionality, control, and data
 - Highly distributed such that each node can operate autonomously
 - Highly centralized where most functionality, control, and data reside at a central node, and local nodes just provide an interface at a particular location (e.g., central ac unit for a hotel with console i/f in each room)
 - Partially distributed, where each node performs a subset of the total functionality, control, and data
- Trade studies are performed to optimize the distribution based on performance, availability, security, and cost

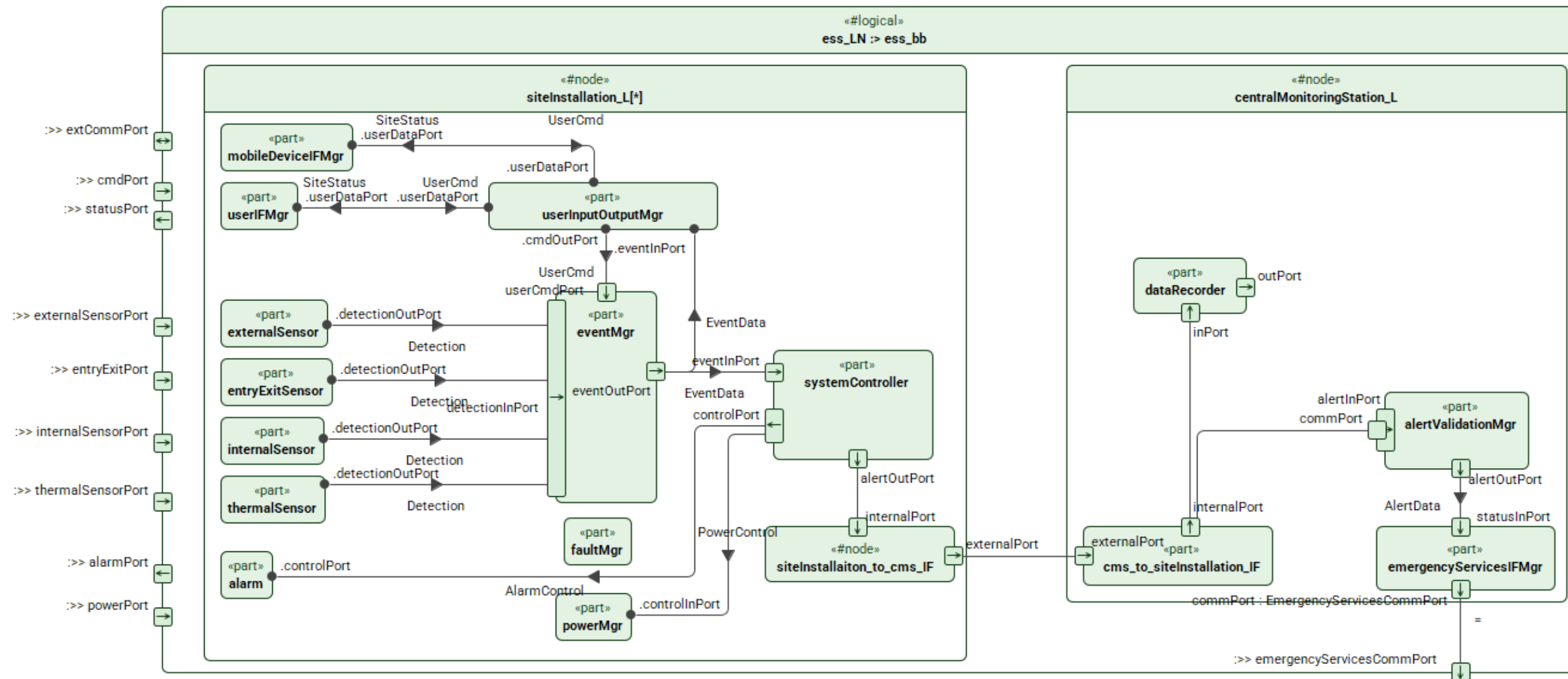


Define Logical Node Architecture

- A logical node aggregates the logical components that are allocated to it
 - The selection of components at a node determine its functionality and stored items (e.g., data)
 - Components may be redefined and added

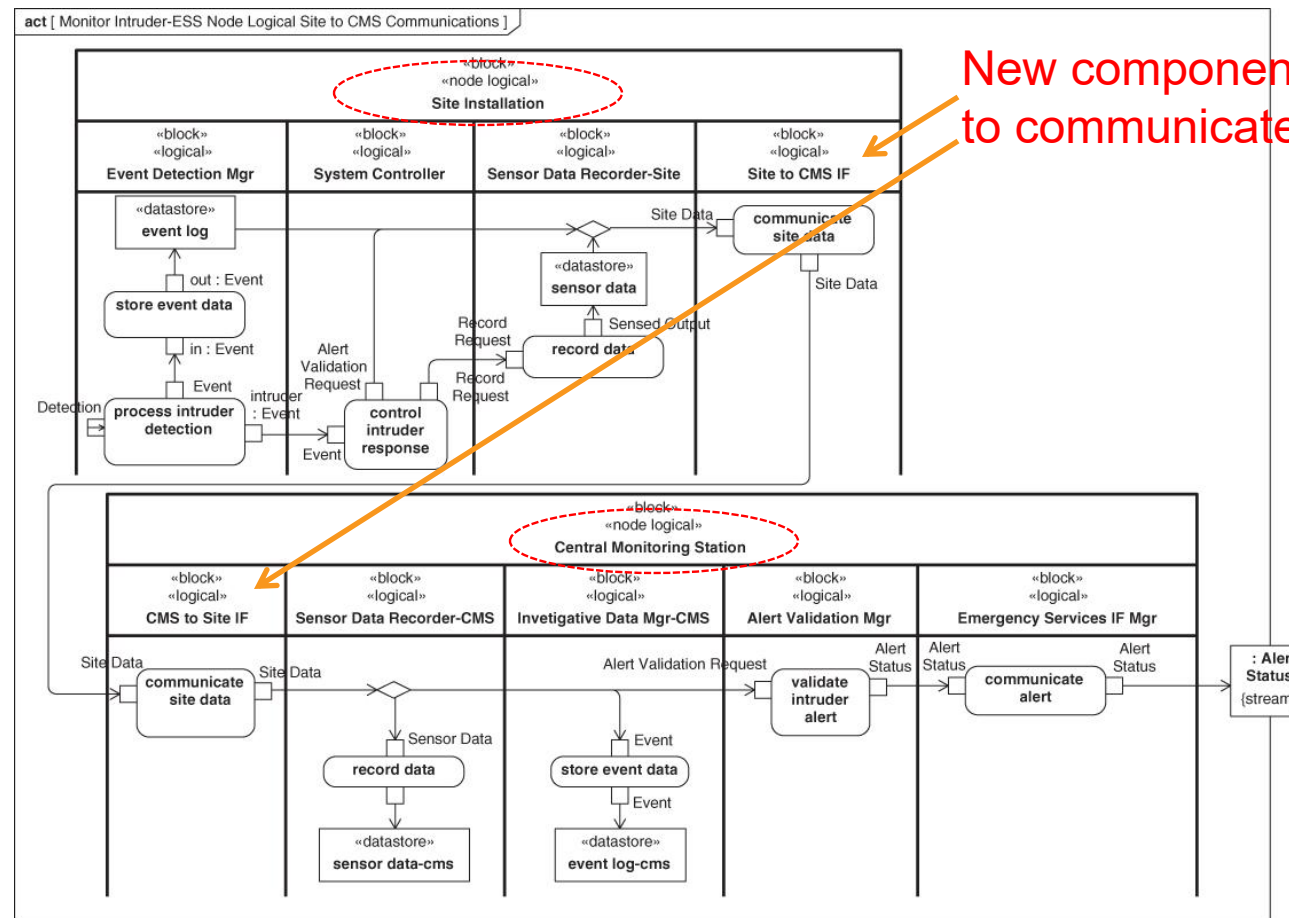


ESS Logical Node Interconnection



Define Logical Node Architecture (cont.)

- Interaction of logical components distributed across the logical nodes

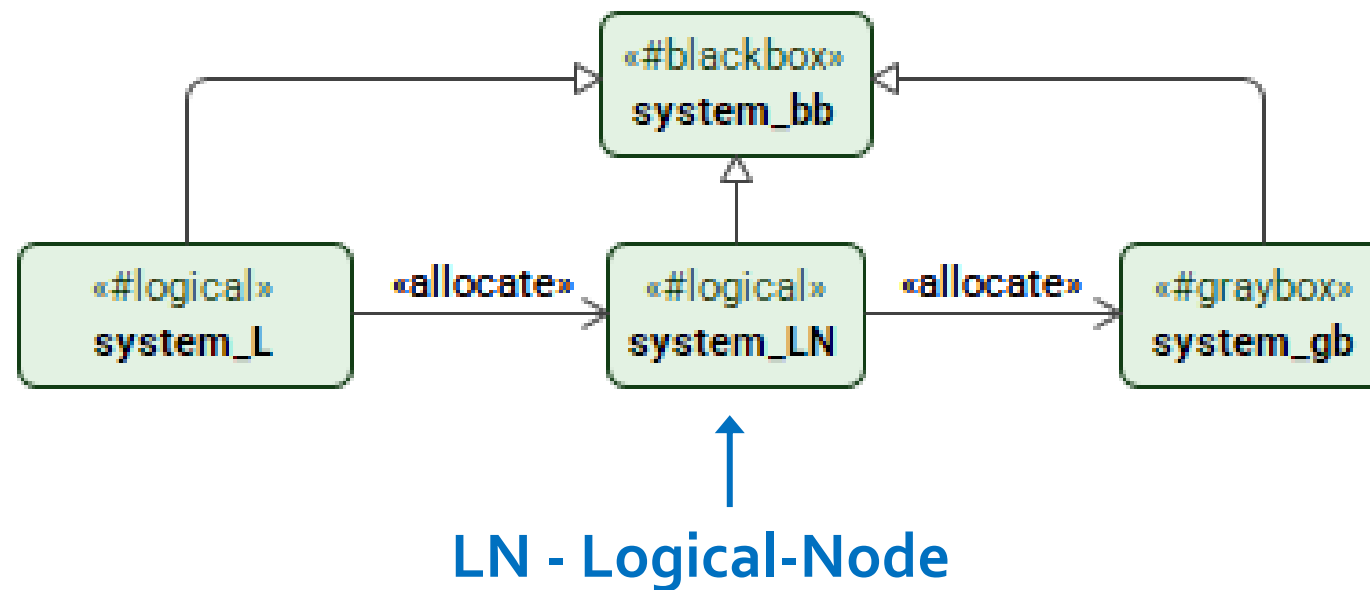


New components identified to communicate across nodes

From OOSEM v1 slides

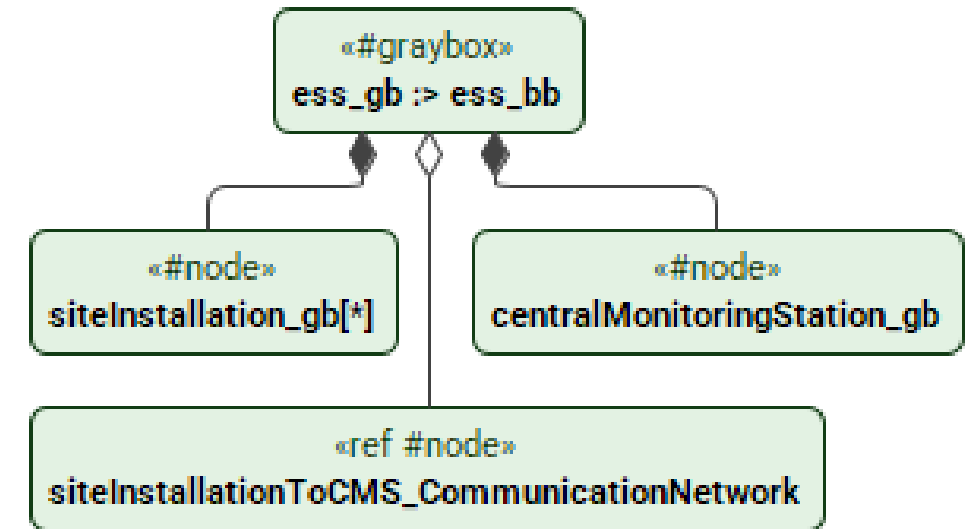
Decomposition

Logical to Gray Box



Define Physical Architecture

- gray box (ess_gb) subsets black box (ess_bb)
- Physical nodes
 - composed of physical components instead of logical components
 - may include additional node-specific features (e.g., location, interface, sizing)

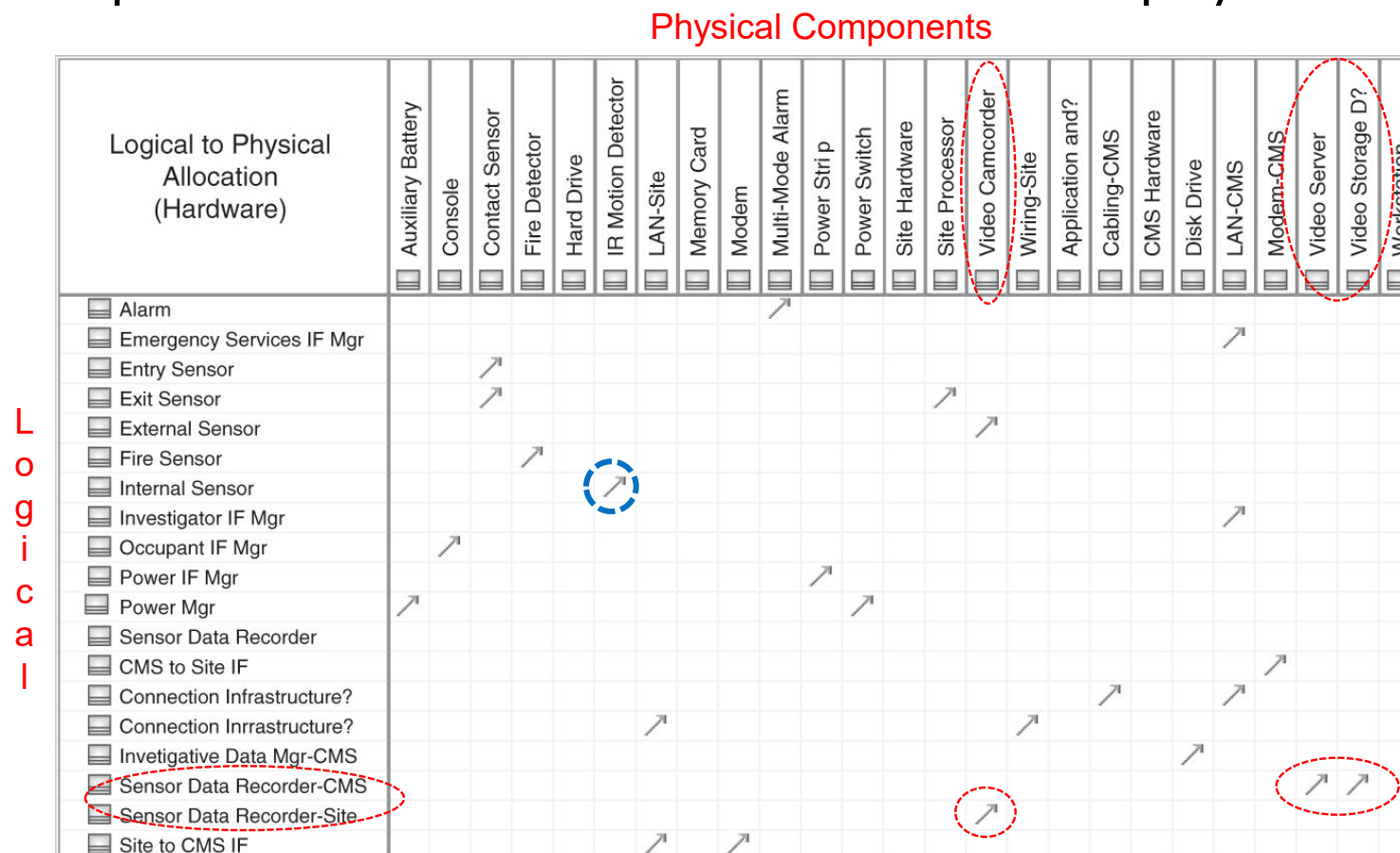


Define Physical Architecture (cont.)

- Logical components from each logical node are allocated to hardware, software, data, people, and facilities (e.g. physical components) at the corresponding physical node
- Allocation is based on trade-off analysis addressing technology and implementation concerns related to performance, reliability, sizing, security, cost, and other considerations
 - Capture rationale for allocate relationship
 - Can use variation with variants to identify alternative physical designs
- Design constraints identified from 'Analyze System Requirements' are imposed on the physical design
- A logical component allocated to software must then be allocated to a hardware component to execute it

Define Physical Node Architecture (cont.)

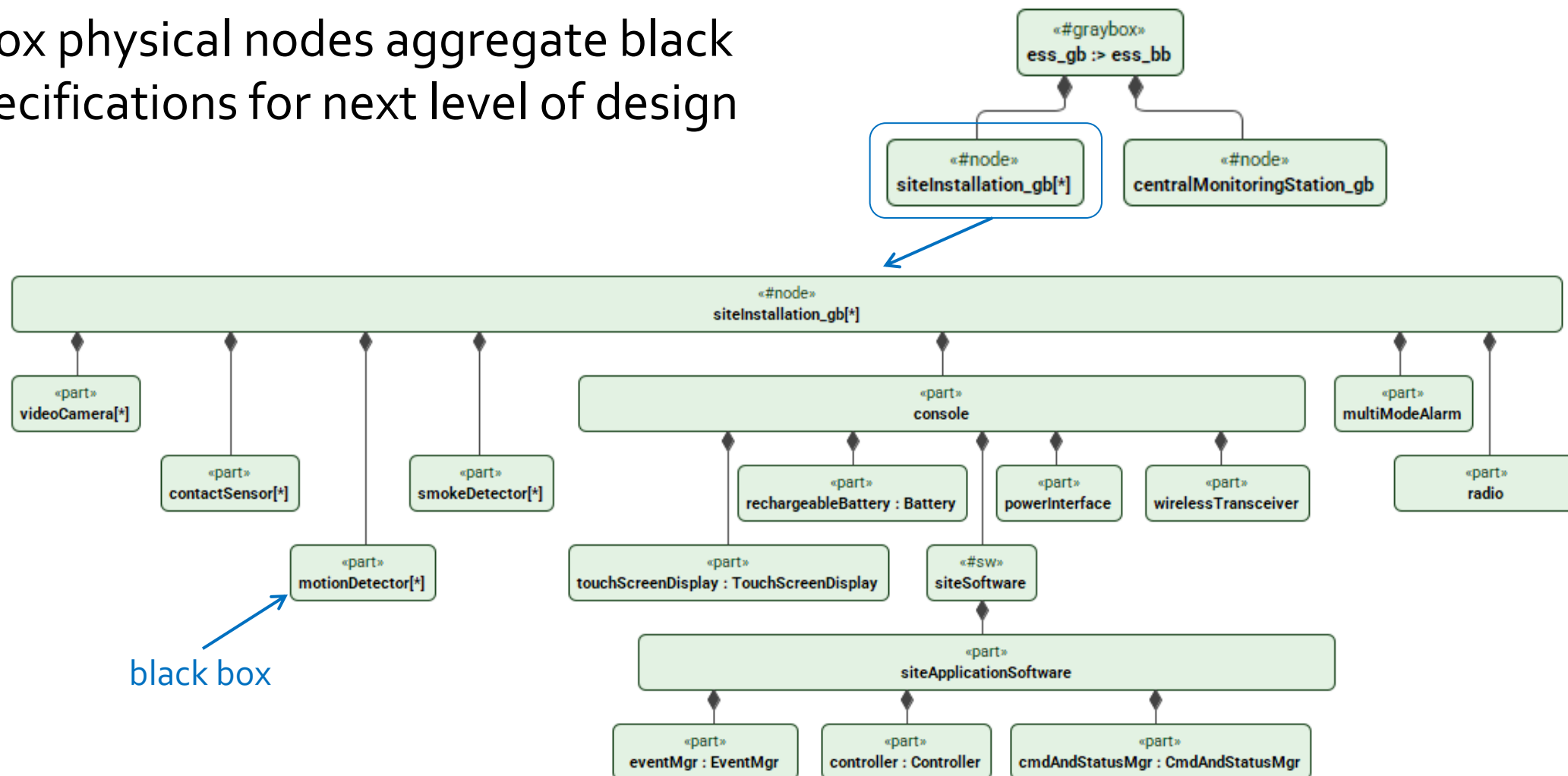
- Logical components at each node are allocated to physical components



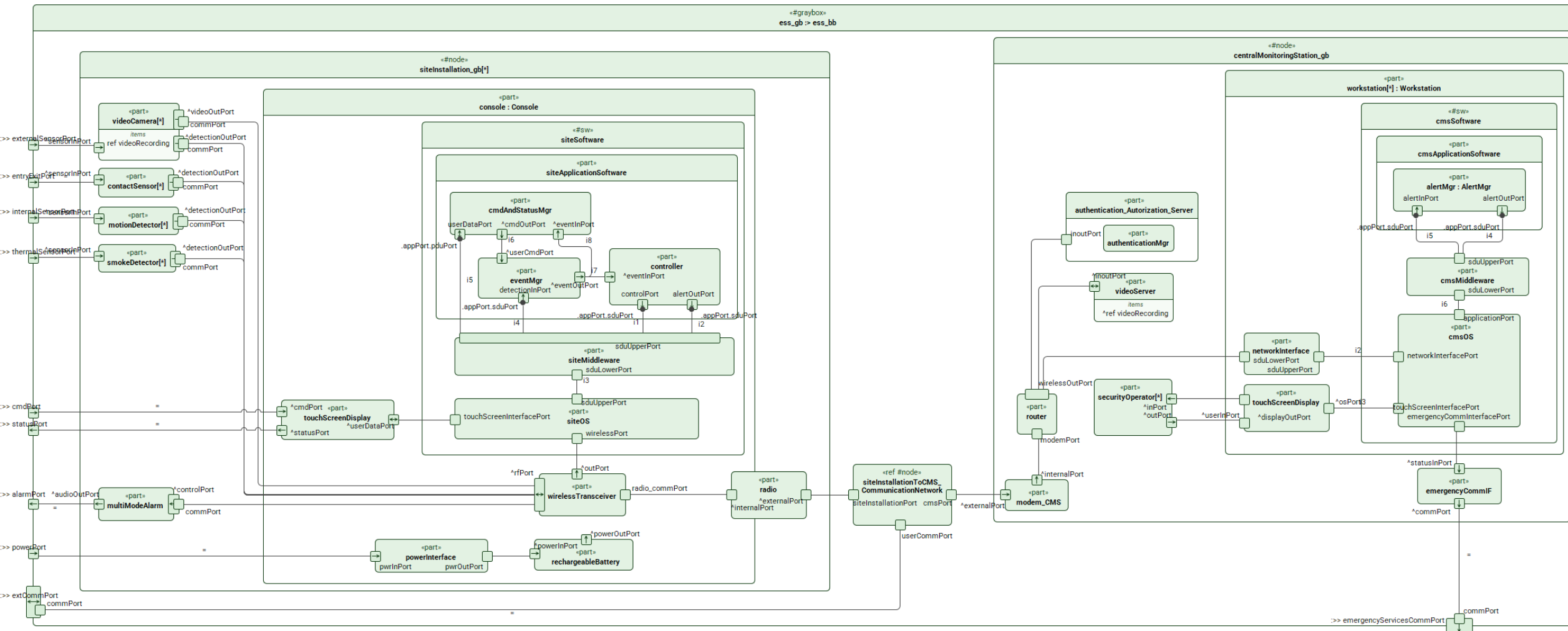
From OOSEM v1 slides

Define Physical Architecture (gray box)

- Gray box physical nodes aggregate black box specifications for next level of design

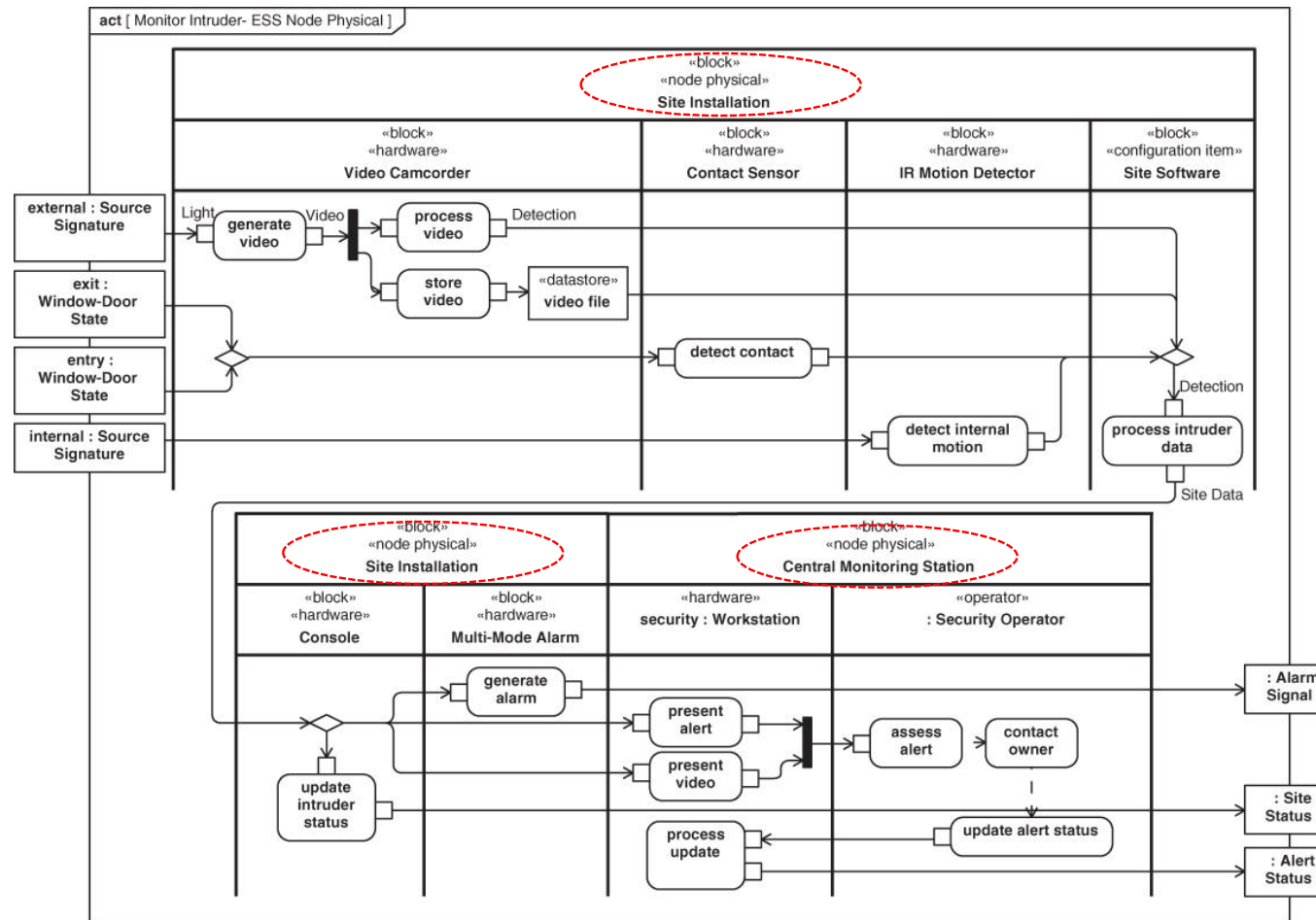


ESS Physical Architecture Interconnection View



Define Physical Architecture (cont.)

- Interaction of physical components across nodes



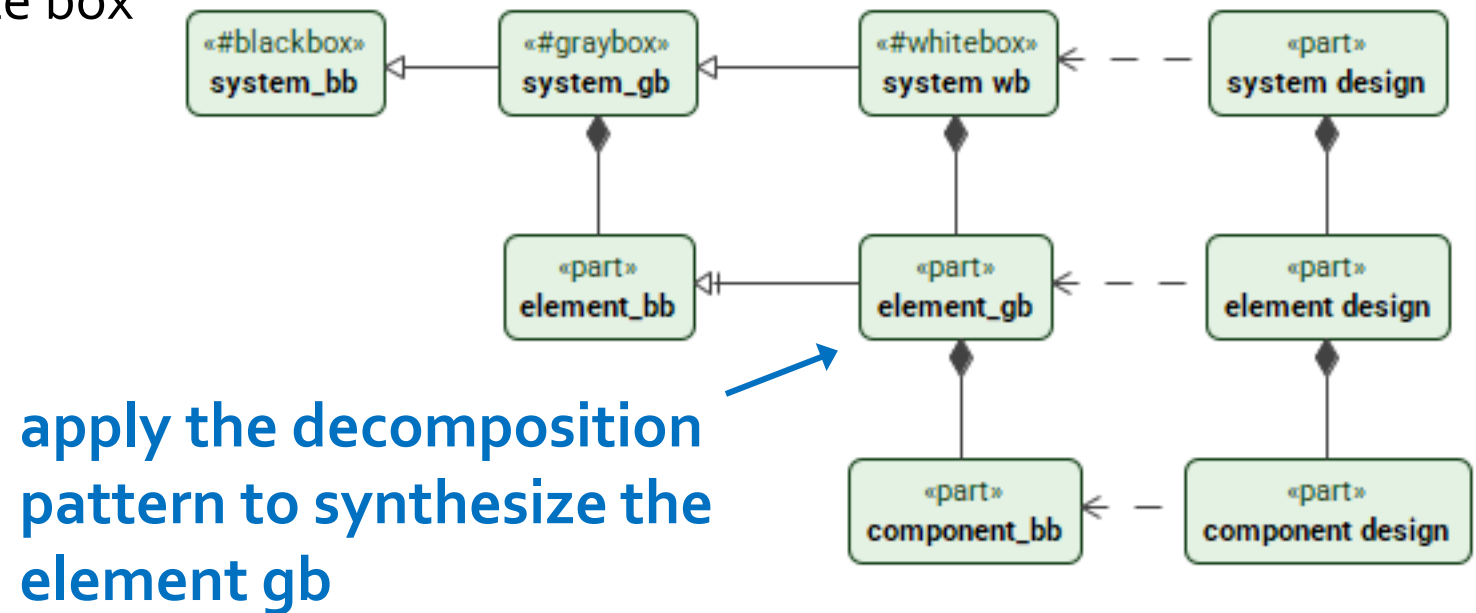
From OOSEM v1 slides

Synthesize Candidate Physical Architectures

Recursive Decomposition Pattern

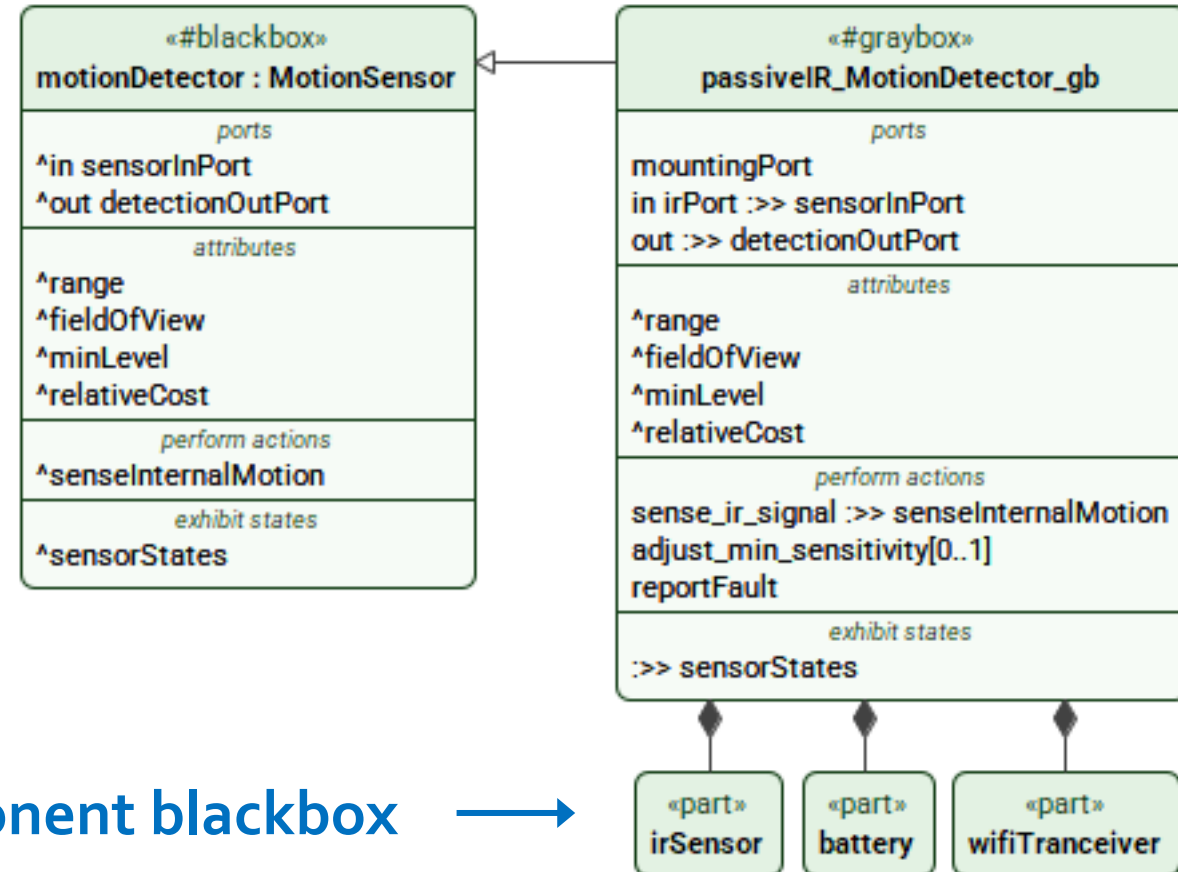
Separately Managed Models

- Black Box (BB) - Specification at level N
- Gray Box (GB) - Decomposed to next-level (N+1) black box specifications
- White Box (WB) - Integrates next-level (N+1) designs into top-level (N) design
 - Can allocate gray box to white box instead of specialize gray box
 - Can continue to refine white box



Element Specification & Design

- Apply recursive approach
 - Element black box
 - Element logical design
 - Element graybox
 - Element whitebox
 - Continued design refinement

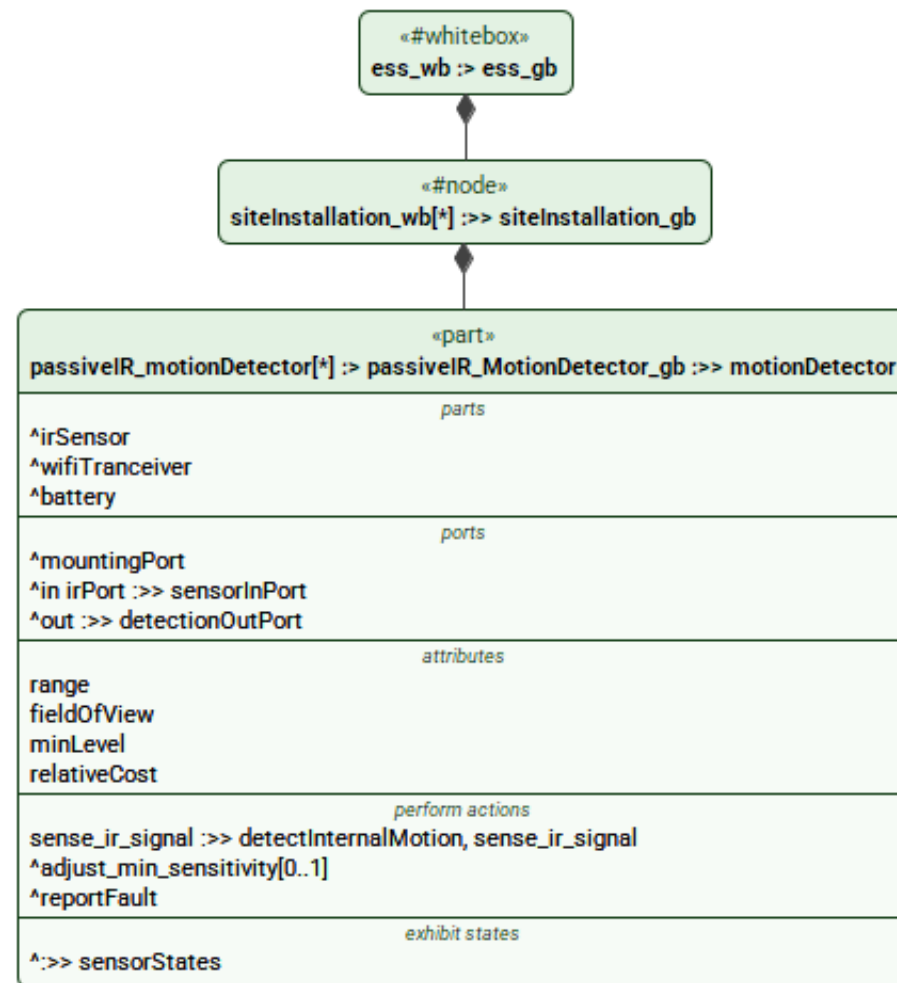


component blackbox →

Integrating Element Design into Higher Level System Design

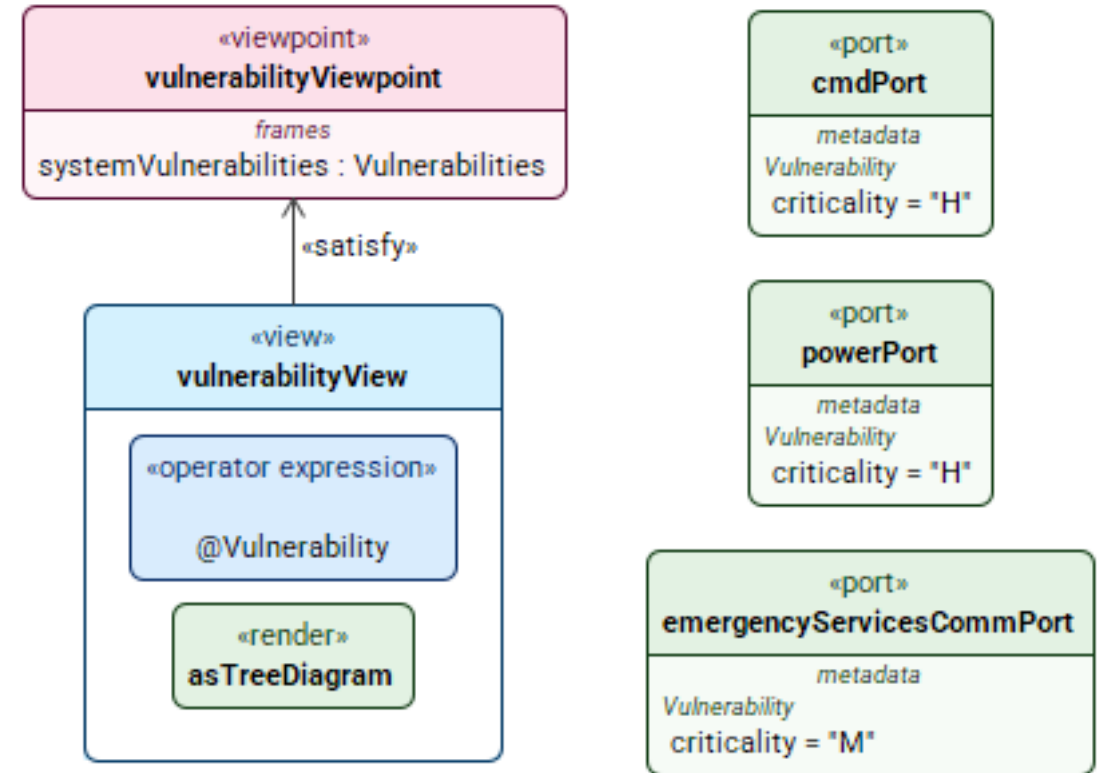
- Integrate next level design (e.g. element gray box) into system design
 - ess_wb subsets ess_gb
 - Graybox component redefines blackbox component

- **motionDetector is a blackbox**
- **passiveIR_MotionDetector_gb is the graybox**



Architecture Views

- Viewpoints and views can capture specific stakeholder perspectives such as a vulnerability view of the architecture
- A viewpoint specifies a set of stakeholder concerns (e.g., what the stakeholder cares about)
- A view renders a filtered subset of the model that satisfies the viewpoint
- A vulnerability view filters the system architecture based on *vulnerability markings*
- A view can be rendered as a diagram, table, or document (e.g. a risk report with a list of vulnerabilities, criticality, and mitigations)



Summary

- Establishing explicit partitioning criteria enables good architecting practice
- A logical node aggregates logical components at a location
- The logical components at each node are allocated to physical components at that node
- The hardware, software, and data architectures are views of the system physical architecture
- The hardware and software components are specified as a black box (e.g., functional, interface, performance)
- Other cross cutting views of the architecture can be defined to support specific concerns (e.g., subsystems, security, ..)

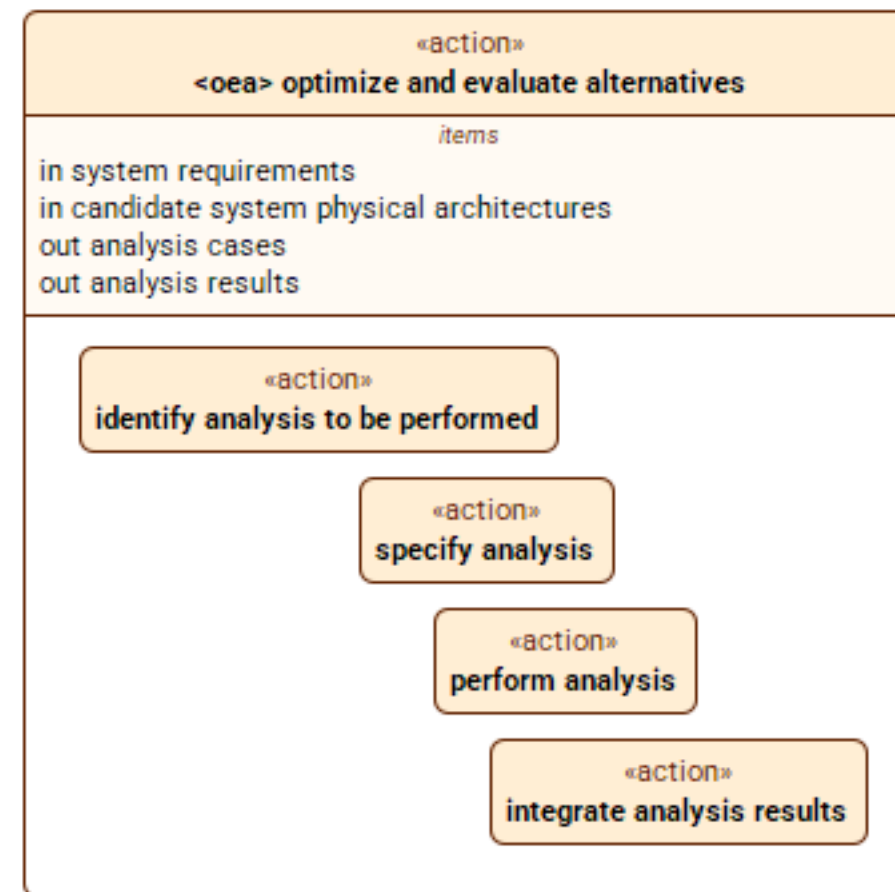
Optimize and Evaluate Alternatives

Motivation

- System model must be integrated with analysis models to support:
 - Trade studies
 - Identification, evaluation, and management of key parameters
 - Traceability and impact assessment
 - Verification via analysis
- Invoked throughout the process

Optimize & Evaluate Alternatives

- Identify the the analysis to be performed
- Specify the analysis via analysis cases
- Perform the analysis
- Integrate analysis results into system specification and design



Identify Analyses to be Performed

- Identify analysis to:
 - Evaluate and select a preferred solution from alternative designs
 - Characterize or predict some aspect of the system, such as performance, reliability, mass properties, or cost
 - Optimize the design through sensitivity analysis
 - Verify a design satisfies its requirements
 - Support risk analysis
- Capture as an analysis case or calc

«analysis» AvailabilityAnalysis
<i>subject</i> ess_under_analysis
<i>objective</i> doc determine availability of system including communications to central monitoring system
<i>attributes</i> in <mtbf> reliability in <mttr> maintainability out availability : Real

Entry Sensor Battery Lifetime Analysis

- Calculations can be used in analysis cases

«calc def» ContactSensorBatteryLife
<i>attributes</i> triggerConsumption :> electricCharge default = 0.02 [milli * A * h] dailyConsumption :> electricCharge = triggerConsumption * triggers_per_day + standbyConsumption batteryCapacity :> electricCharge default = 240 [milli * A * h] ...
<i>parameters</i> in triggers_per_day : Integer default = 10 in standbyConsumption :> electricCharge default = 0.05 [milli * A * h] out batteryLifeInYears : Real = batteryCapacity / (dailyConsumption * 365)

«attribute»
batteryLife_Years = ContactSensorBatteryLife(triggers_per_day = 20, standbyConsumption = 0.06)

Optimize & Evaluate Alternatives

Trade Study

- Perform Trade Study Analysis to evaluate and select a preferred solution that satisfies the system requirements and maximizes value based on the effectiveness measures
- For each alternative
 - Identify the analysis to be performed (e.g., performance, mass properties, reliability, and cost)
 - Specify each analysis as an analysis case
 - Perform the engineering analysis to determine the values of the system properties (Note: the analysis may be performed in various engineering analysis tools)
- Compute the overall effectiveness for each alternative (i.e., the evaluation function)

Criteria	Weight	Alt 1	Alt 2	Alt 3
Performance	0.5	7	5	5
Reliability	0.2	4	6	5
Cost	0.3	3	5	8
Effectiveness		5.2	4.2	5.9

Summary

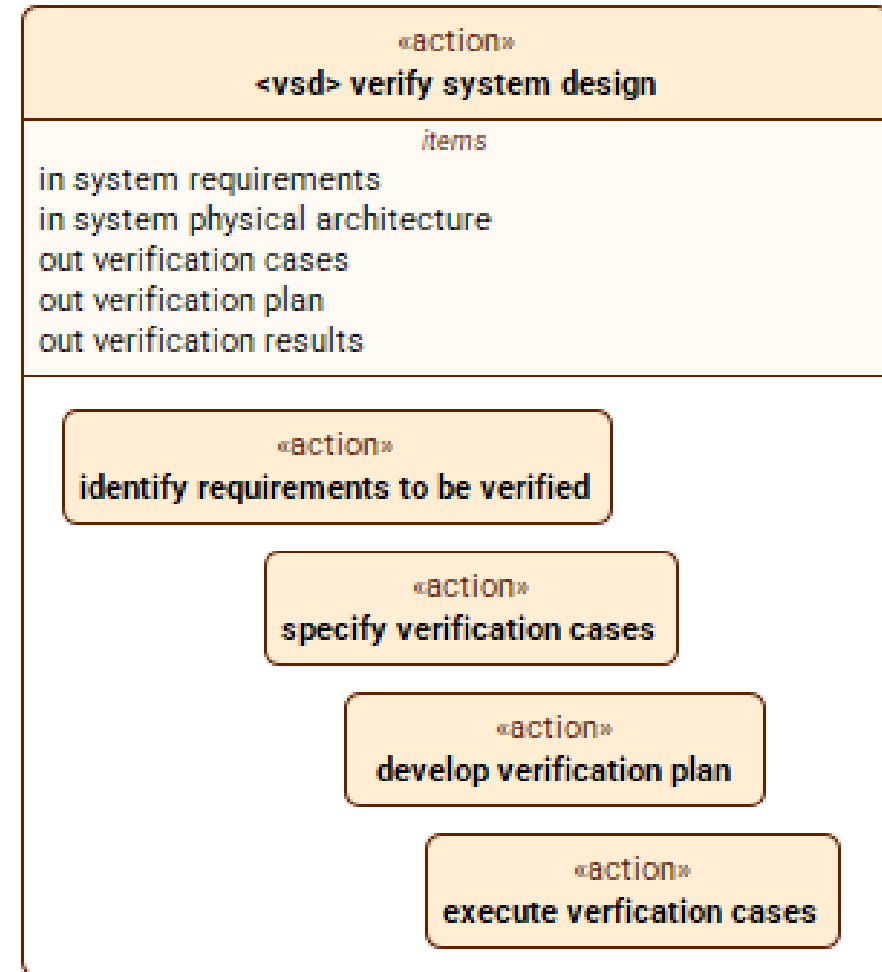
- Optimize and Evaluate Alternatives is invoked throughout the design process to support architecture optimization
- Each analysis model can be captured using an analysis case or calc and then executed by an analysis tool
- An analysis case can be used to compute top-level MOEs from critical system, element, and component properties

Integrate and Verify System

- A verification program is used to ensure requirements are satisfied
- A test program and the resulting hardware and software fixes are often a significant contributor to the development schedule and cost
- Surface verification issues early and improve test program effectiveness
 - Specify verifiable requirements that minimize ambiguity
 - Verify early through analysis
 - Design for test
 - Architect the verification system
 - Perform effective verification planning that includes continuous system integration and verification

Verify System Design

- Design verification is performed via analysis on the left side of the vee
 - Develop verification cases
 - Develop initial verification plan
 - Execute verification cases via analysis
 - Initiate development of test system architecture and test procedures
- Product verification is performed via test on the right side of the vee
 - Part of assembly, integration, & test process

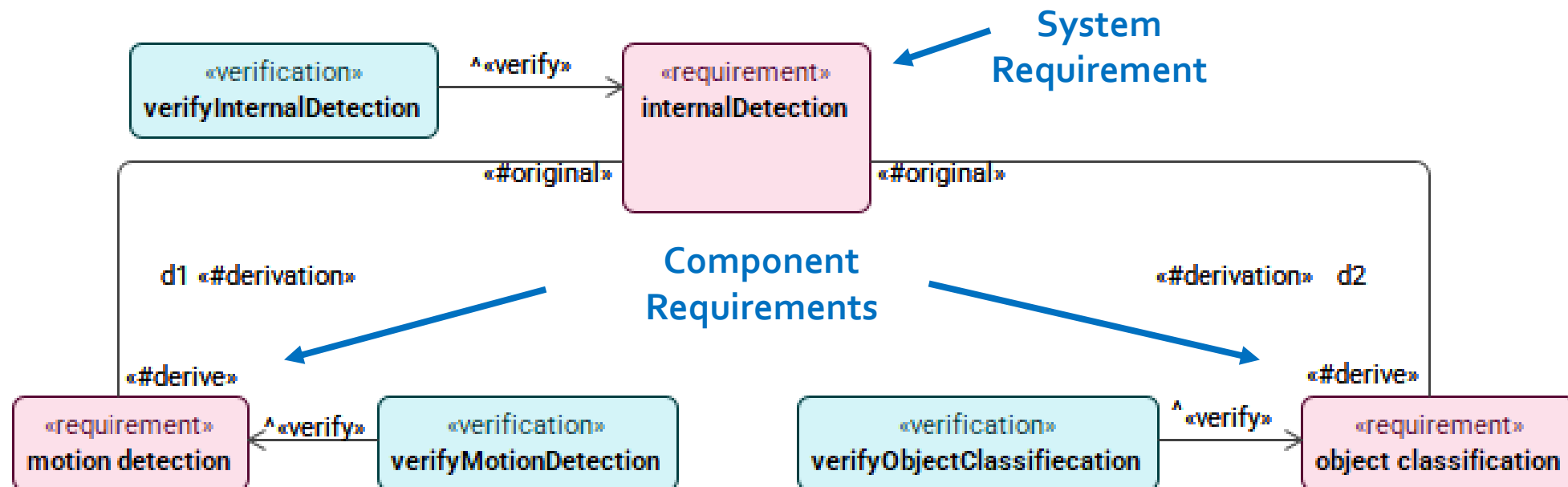


Verification Methods

- Analysis
 - Used to verify design changes before implementation
 - Can vary in fidelity to include preliminary analysis, detailed simulations, and hardware, software, operators in the loop
- Inspection, Demonstration, Test
 - Used to verify the product after implementation

Identifying Verification Cases

- A verification case can verify one or more requirements
- Identify derived requirements and dependent verification cases



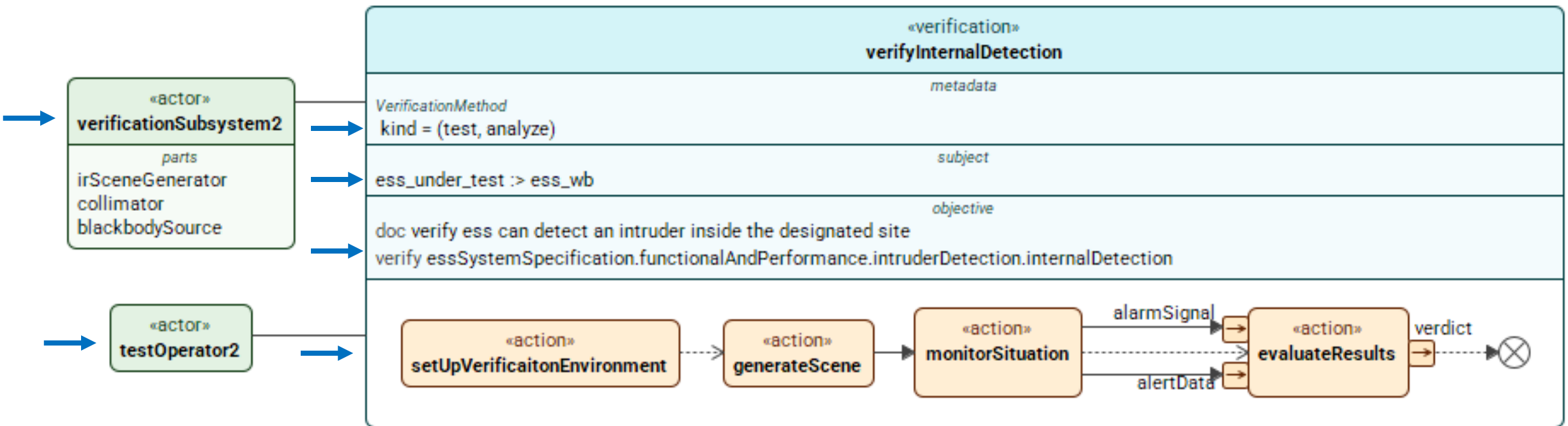
Verify Internal Detection Verifiable Requirements

- Specify requirements in terms of the inputs/conditions and expected outputs
 - Can either be a refined requirement or an update to the original requirement

<i>«requirement»</i> internalDetection
<i>doc</i> ESS shall generate an alert when motion is detected and the object is classified as a human, and when the system is activated.
<i>attributes</i> stateKind : StateKind detection : Boolean classificaiton : Boolean alert : Boolean
<i>require constraints</i> {alert == detection and classificaiton}
<i>assume constraints</i> {stateKind == activated}

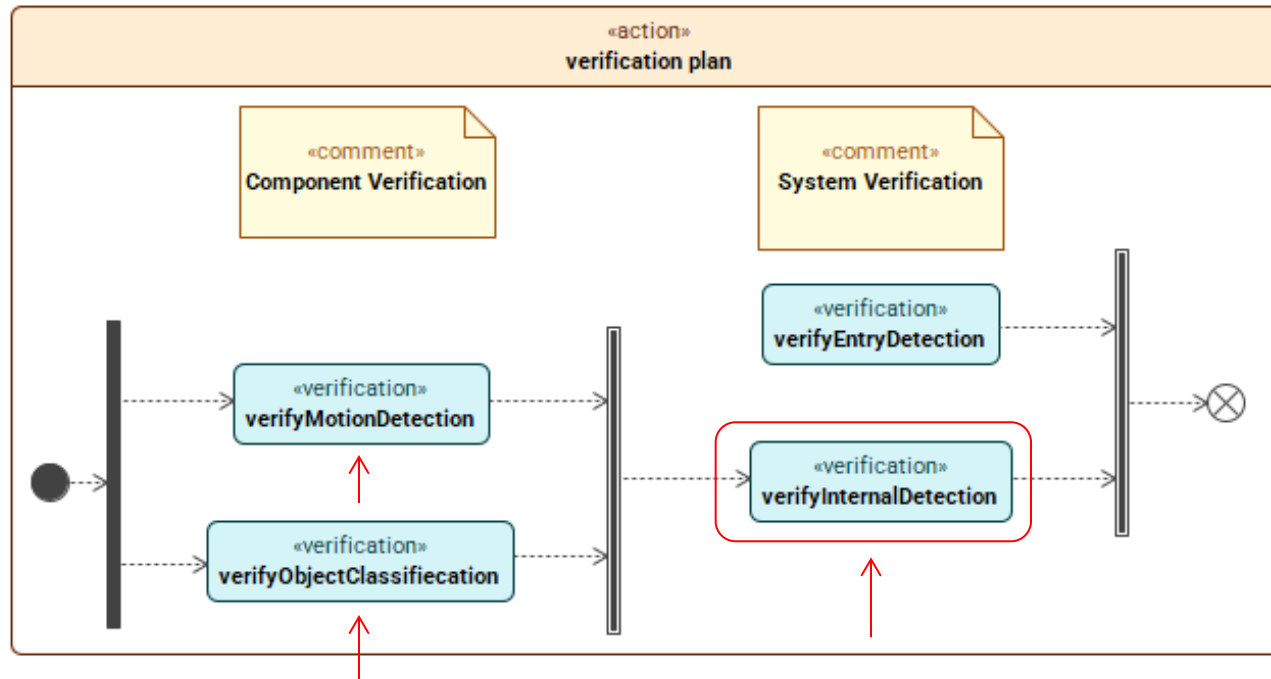
Define Verification Case

- Objective including requirements to be verified
- Subject – unit under test
- Actors – equipment and operators
- Verification methods
- Action flow



Verification Plan

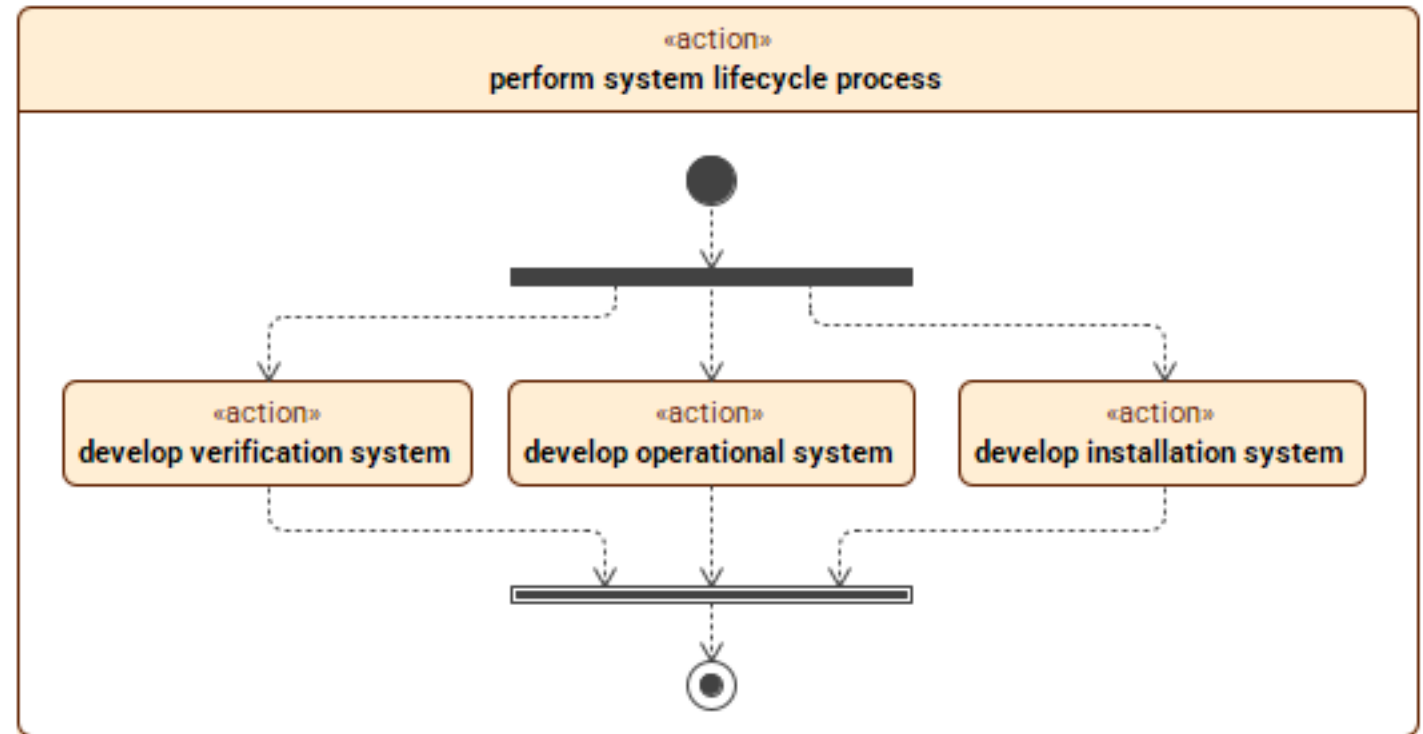
- Sequencing of verification cases can be captured in action flow



Verification cases from previous slide

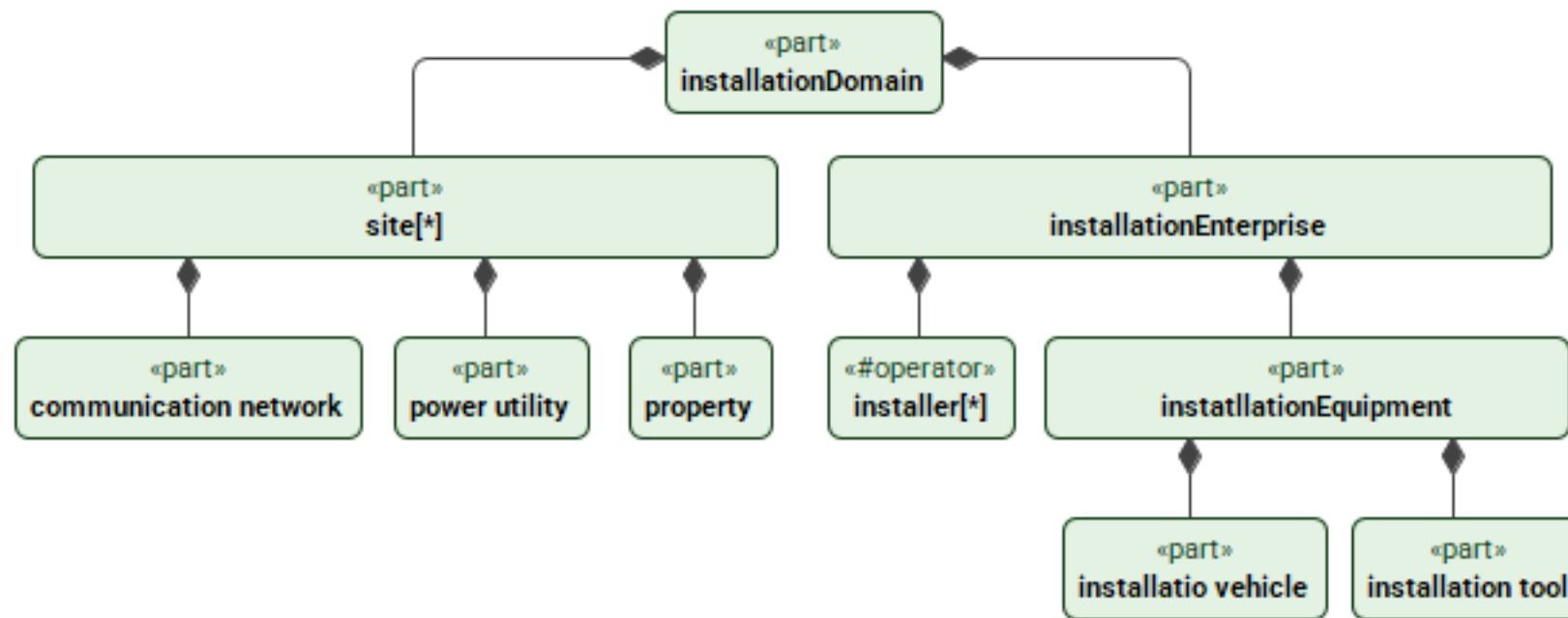
Applying OOSEM to Enabling Systems

- OOSEM can be applied to develop enabling systems that support the operational system throughout its life cycle
 - Verification system
 - Manufacturing system
 - Installation system
 - Support system
 - ...



Develop Installation System

- Installation Domain Model includes Installation Enterprise & Installation System
- Tailor OOSEM as needed to develop each domain



Summary

- OOSEM can support a more effective verification program by using the model to define verification cases, verification planning, architecting the environment, and maintaining requirements traceability
- OOSEM can be tailored and applied to enabling systems for each phase of the life cycle

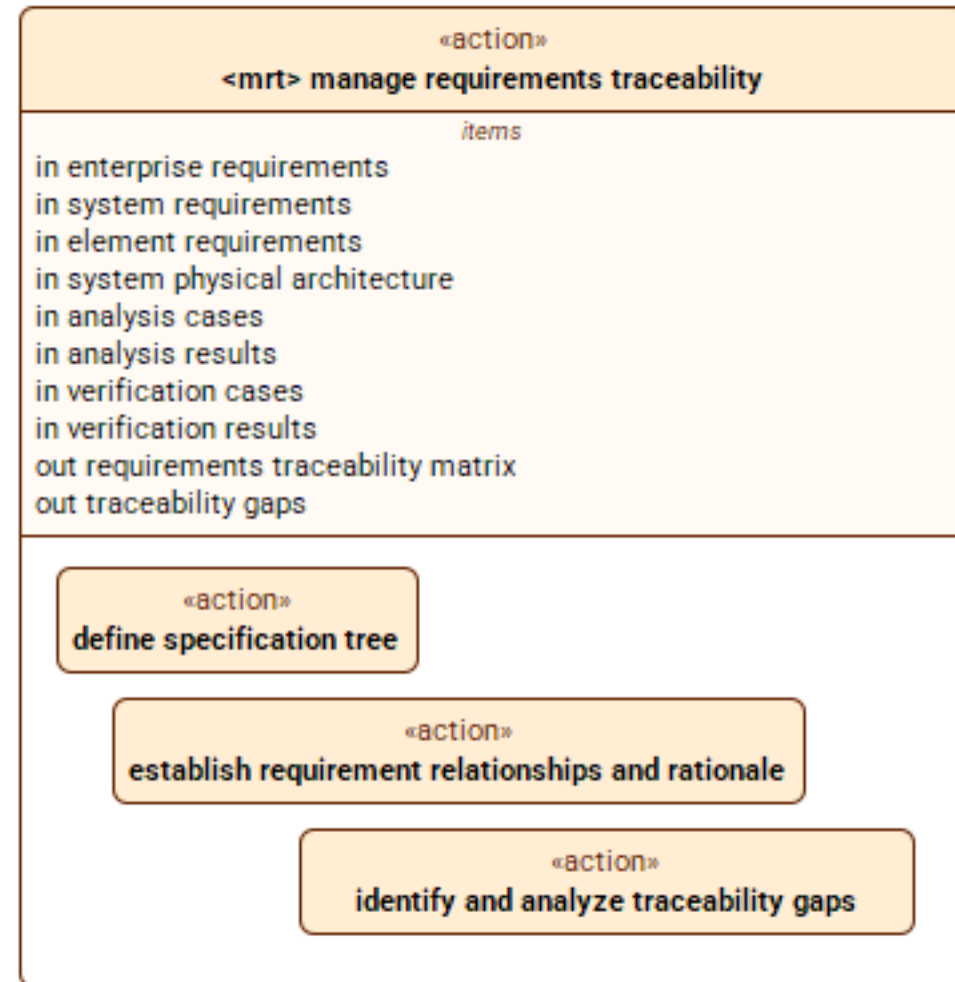
Manage Requirements Traceability

Motivation

- Requirements traceability is essential to understand:
 - How requirements are satisfied and verified
 - The impact of requirements and design changes

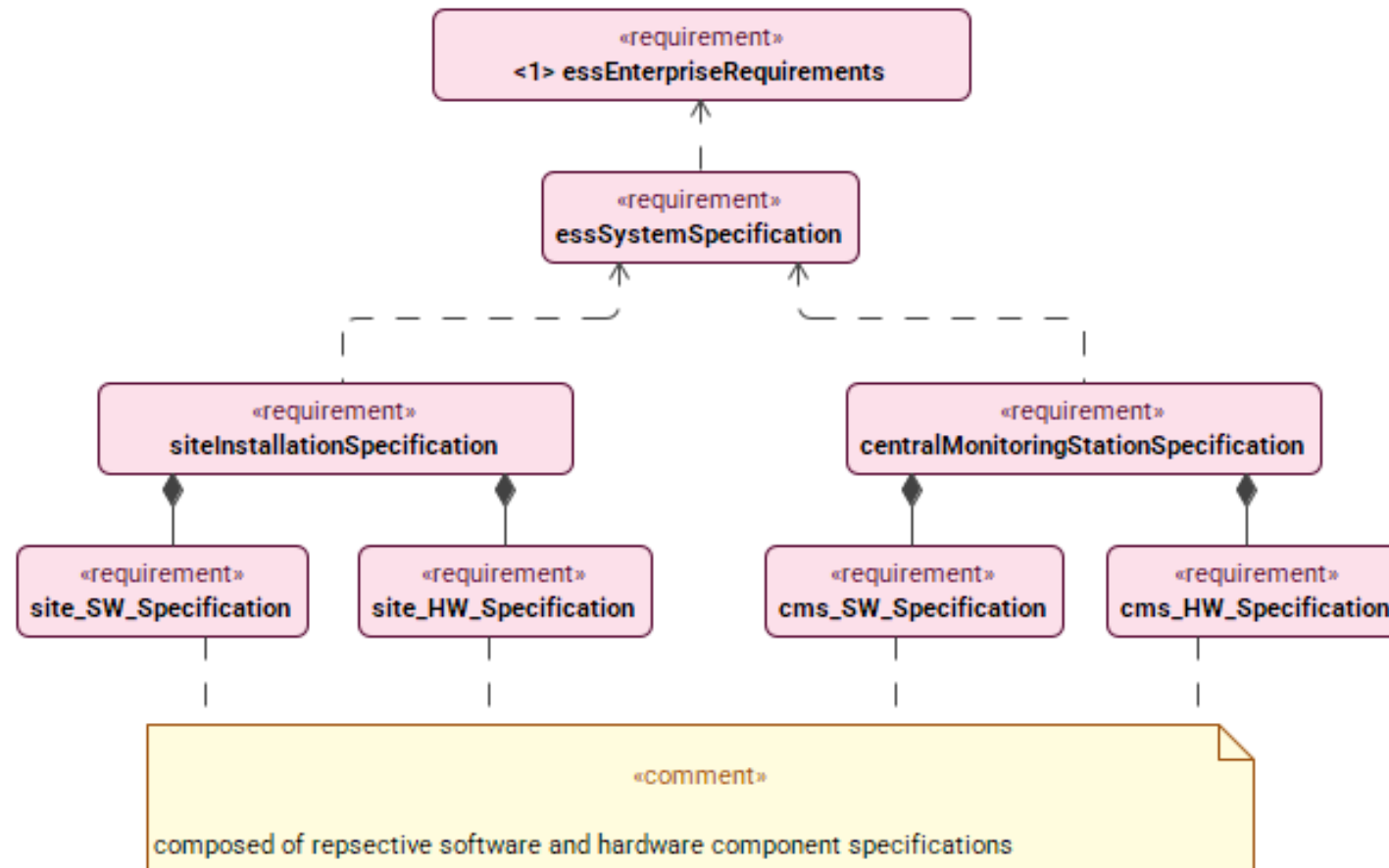
Manage Requirements Traceability

- Relate specifications in a specification tree
- Capture and maintain traceability between requirements
 - derived requirements
 - requirements satisfaction
 - requirements verification
- Identify and analyze traceability gaps between requirements, design, analysis, and verification



Define Specification Tree

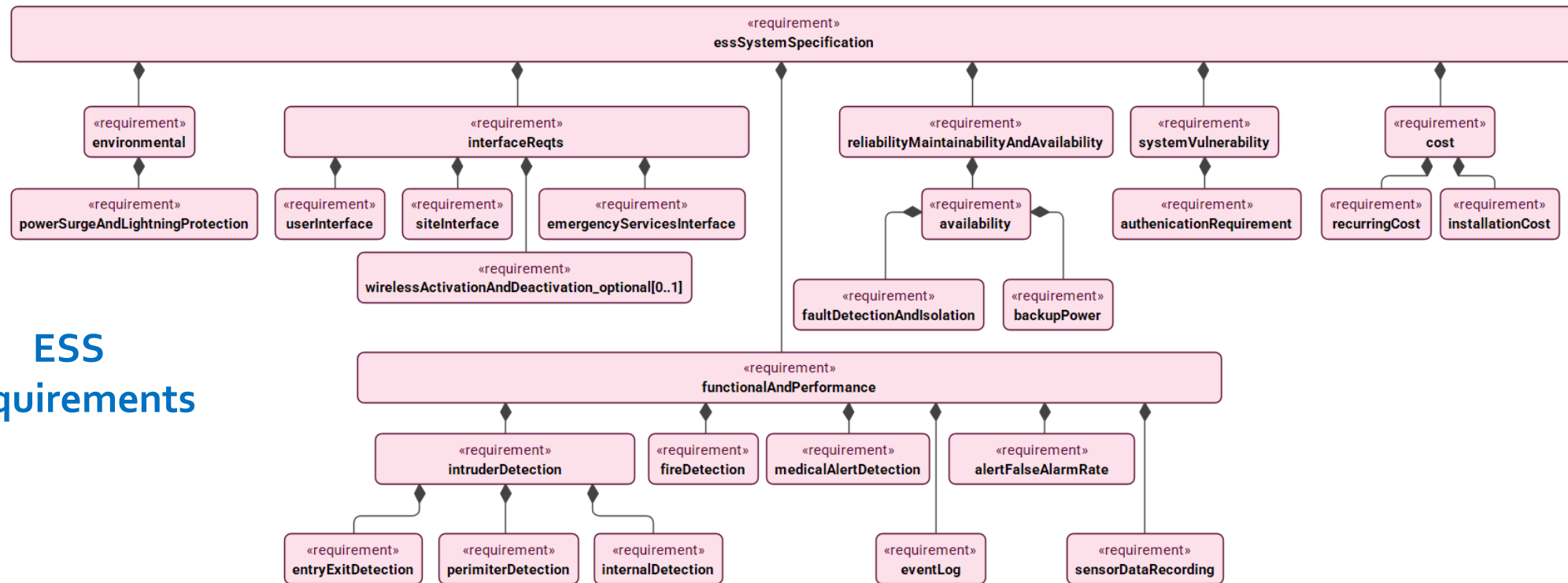
- The specification tree shows the specifications at each level of the system hierarchy



Requirements Hierarchy for Each Specification

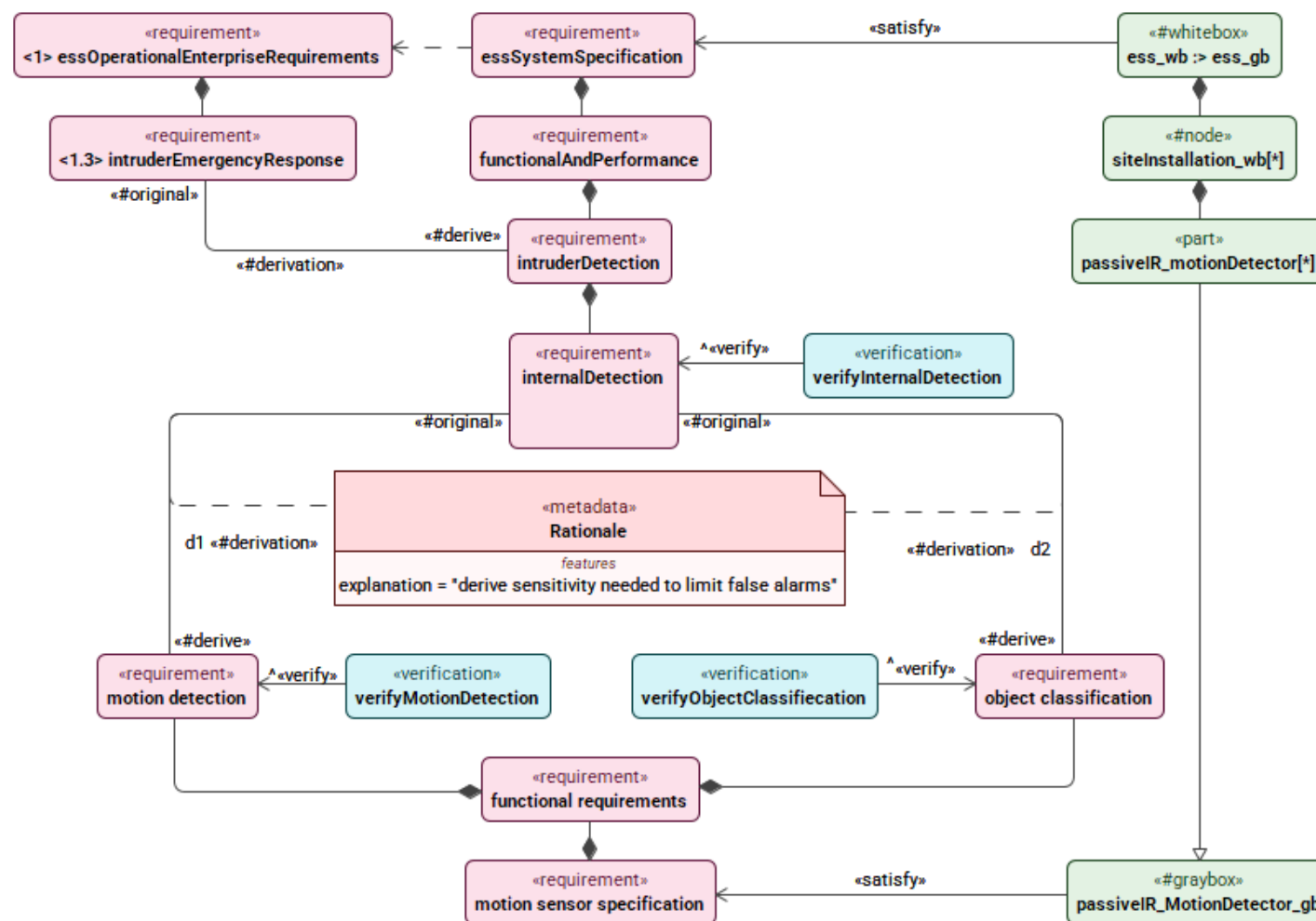
- A top-level requirement for each specification is defined
 - The hierarchy of requirements corresponds to the way the specification is organized

ESS Requirements



Establish Requirements Traceability

- Establish relationships between the text-based requirements and other model elements (including other requirements, design, analysis and verification cases)
- Capture rationale
- Render in requirements traceability matrix



Analyze Traceability Gaps

- Traceability reports can be generated from the model to analyze traceability gaps and assess how the system design satisfies the system requirements

1-Requirements	14
2 ESS System Specification	14
2.2 System Vulnerability	
2.6 Interface	3
2.6.5 Emergency Services Interface	✓
2.6.7 Site Interface	✓
2.6.8 User Interface	✓
2.6.4 Wireless Activation and Deactivation (optional)	
2.6.8 User Interface	
2.6.8.2 Displays and Controls Interface	
2.6.8.1 Web Interface	
2.3 Reliability, Maintainability, and Availability	2
2.3.1 Availability	2
2.3.1.1 Backup Power	✓
2.3.1.2 Fault Detection and Isolation	✓
2.1 Environmental	1
2.1.1 Power Surge and Lightning Protection	✓
2.4 Cost	1
2.4.1 Installation Cost	✓
2.4.2 Recurring Cost	
2.5 Functional and Performance	7
2.5.4 Data Storage and Validation	✓
2.5.2 Fire Detection and False Alarm Rate	✓
2.5.1 Intrusion Detection and False Alarm Rate	✓
2.5.3 Medical Alert and False Alarm Rate	✓
2.5.1 Intrusion Detection and False Alarm Rate	3
2.5.1.2 Entry-Exit Detection	✓
2.5.1.3 Internal Detection	✓
2.5.1.1 Perimeter Detection	✓

System
Satisfaction

satisfied

Manage Requirements Updates

- Manage updates to requirements via a defined requirements management process
- Requirements management tools can be used in conjunction with a systems modeling tool to manage the requirements
 - Tool integration required to synchronize requirements and their relationships across tools
- Requirements management process defines how changes are handled
 - Example: make changes to text requirements in the requirements management tool and update traceability relationships in the model
- Generate change impact reports by navigating model relationships to facilitate change process

Summary

- The specification tree identifies each specification and the traceability relationship between them
- Each specification contains a hierarchy of requirements
- Fine grained traceability can be maintained between the requirements in each specification and other requirements, design, analysis, and verification elements in the model
- Traceability reports can be generated from the model to analyze traceability gaps
- Updates to requirements are controlled by a management process and synchronization is maintained between the model and the requirements management tool

OOSEM Summary

OOSEM Summary

- OOSEM is a model-based systems engineering method that supports the system specification and design process
 - Produces a system model
 - Flow down of requirements, structure, behavior, critical properties, and verification cases
 - Provides a way to deal with system design and development complexity using separation of concerns as a fundamental tenet
 - Black box, white box
 - Logical, physical
 - Distribution
 - Operational system, Enabling systems
 - ...
 - Should be tailored to satisfy project objectives and constraints

OOSEM Summary (cont.)

- OOSEM Supports the following activities
 - Manage System Development
 - Analyze Stakeholder Needs
 - Analyze System requirements
 - Define Logical Architecture
 - Synthesize Candidate Physical Architectures
 - Optimize and Evaluate Alternatives
 - Verify System Design
 - Manage Requirements Traceability

OOSEM is a System Thinking Process